Red Hat Enterprise Linux 10

管理、监控和更新内核

在 Red Hat Enterprise Linux 10 上管理 Linux 内核的指南

Last Updated: 2025-12-05

Red Hat Enterprise Linux 10 管理、监控和更新内核

在 Red Hat Enterprise Linux 10 上管理 Linux 内核的指南

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at
http://creativecommons.org/licenses/by-sa/3.0/
. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

Java ® is a registered trademark of Oracle and/or its affiliates.

XFS ® is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL ® is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js ® is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack ® Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

作为系统管理员，您可以配置 Linux 内核以优化操作系统。对 Linux 内核进行修改可以提高系统性能、安全性和稳定性，并可以对系统进行审核并对问题进行故障排除。

Table of Contents

对红帽文档提供反馈 6

第 1 章 LINUX 内核 7
1.1. 内核是什么 7
1.2. RPM 软件包 7
1.3. LINUX 内核 RPM 软件包概述 7
1.4. 显示内核软件包的内容 8
1.5. 安装特定的内核版本 9
1.6. 更新内核 9
1.7. 将内核设置为默认 9

第 2 章 64K 页大小内核 11
2.1. 按系统构架确定内核页大小 11

第 3 章 管理内核模块 12
3.1. 内核模块简介 12
3.2. 内核模块依赖关系 12
3.3. 列出安装的内核 13
3.4. 列出当前载入的内核模块 13
3.5. 显示内核模块信息 14
3.6. 在系统运行时载入内核模块 15
3.7. 在系统运行时卸载内核模块 16
3.8. 在启动过程早期卸载内核模块 17
3.9. 在系统引导时自动载入内核模块 17
3.10. 防止在系统引导时自动载入内核模块 18
3.11. 编译自定义的内核模块 20

第 4 章 配置内核命令行参数 23
4.1. 什么是内核命令行参数 23
4.2. 了解引导条目 23
4.3. 为所有引导条目更改内核命令行参数 23
4.4. 为单一引导条目更改内核命令行参数 24
4.5. 在引导时临时更改内核命令行参数 25
4.6. 配置 GRUB 设置以启用串行控制台连接 26
4.7. 使用 GRUB 配置文件更改引导条目 26

第 5 章 在运行时配置内核参数 28
5.1. 什么是内核参数 28
5.2. 使用 SYSCTL 临时配置内核参数 29
5.3. 使用 SYSCTL 永久配置内核参数 29
5.4. 使用 /ETC/SYSCTL.D/ 中的配置文件调整内核参数 30
5.5. 通过 /PROC/SYS/ 临时配置内核参数 30

第 6 章 使用 RHEL 系统角色永久配置内核参数 32
6.1. 使用 KERNEL_SETTINGS RHEL 系统角色应用所选的内核参数 32

第 7 章 使用 RHEL 系统角色配置 GRUB 2 引导装载程序 34
7.1. 使用 BOOTLOADER RHEL 系统角色更新现有的引导装载程序条目 34
7.2. 使用 BOOTLOADER RHEL 系统角色，使用密码保护引导菜单 35
7.3. 使用 BOOTLOADER RHEL 系统角色为引导装载程序菜单设置超时 38
7.4. 使用 BOOTLOADER RHEL 系统角色收集引导装载程序配置信息 40

第 8 章 使用内核实时修补程序应用补丁 43

8.1. KPATCH 的限制 43
8.2. 对第三方实时补丁的支持 43
8.3. 获得内核实时补丁 43
8.4. 实时对内核打补丁的过程 44
8.5. 将当前安装的内核订阅到实时补丁流 45
8.6. 自动订阅将来的内核到实时补丁流 46
8.7. 禁用实时补丁流的自动订阅 48
8.8. 更新内核补丁模块 49
8.9. 删除 LIVE PATCHING 软件包 49
8.10. 卸载内核补丁模块 50
8.11. 禁用 KPATCH.SERVICE 51

第 9 章 在虚拟环境中保留内核 PANIC 参数 53
9.1. 什么是软锁定 53
9.2. 控制内核 PANIC 的参数 53
9.3. 在虚拟环境中有伪装的软锁定 54

第 10 章 为数据库服务器调整内核参数 55
10.1. 介绍 55
10.2. 影响数据库应用程序性能的参数 55

第 11 章 配置巨页 57
11.1. 可用的巨页功能 57
11.2. 在引导时保留 HUGETLB 页面的参数 57
11.3. 在引导时配置 HUGETLB 58
11.4. 在运行时保留 HUGETLB 页面的参数 60
11.5. 在运行时配置 HUGETLB 61
11.6. 管理透明巨页 62
11.7. 页面大小对转换后备缓冲区大小的影响 66
11.8. 其他资源 66

第 12 章 内核日志记录入门 67
12.1. 什么是内核环缓冲 67
12.2. 日志级别和内核日志记录上的 PRINTK 角色 67

第 13 章 重新安装 GRUB 69
13.1. 在基于 BIOS 的机器上重新安装 GRUB 69
13.2. 在基于 UEFI 的机器上重新安装 GRUB 69
13.3. 在 IBM POWER 机器上重新安装 GRUB 70
13.4. 重置 GRUB 70

第 14 章 安装 KDUMP 72
14.1. KDUMP 72
14.2. 使用 ANACONDA 安装 KDUMP 72
14.3. 在命令行中安装 KDUMP 72

第 15 章 在命令行中配置 KDUMP 74
15.1. 估算 KDUMP 大小 74
15.2. 配置 KDUMP 内存用量 74
15.3. 配置 KDUMP 目标 76
15.4. 配置 KDUMP 核心收集器 79
15.5. 配置 KDUMP 默认失败响应 80
15.6. KDUMP 的配置文件 81
15.7. 测试 KDUMP 配置 81
15.8. 系统崩溃后 KDUMP 生成的文件 82

15.9. 启用和禁用 KDUMP 服务 83
15.10. 防止内核驱动程序为 KDUMP 加载 84
15.11. 在使用加密磁盘的系统中运行 KDUMP 85

第 16 章 启用 KDUMP **86**
16.1. 为所有安装的内核启用 KDUMP 86
16.2. 为特定安装的内核启用 KDUMP 86
16.3. 禁用 KDUMP 服务 87

第 17 章 支持的 KDUMP 配置和目标 **89**
17.1. KDUMP 的内存要求 89
17.2. 自动内存保留的最小阈值 90
17.3. 支持的 KDUMP 目标 91
17.4. 支持的 KDUMP 过滤等级 92
17.5. 支持的默认故障响应 93
17.6. 使用 FINAL_ACTION 参数 93
17.7. 使用 FAILURE_ACTION 参数 93

第 18 章 固件支持的转储机制 **95**
18.1. IBM POWERPC 硬件支持转储固件 95
18.2. 启用固件支持的转储机制 95
18.3. IBM Z 硬件支持的固件转储机制 96
18.4. 在 FUJITSU PRIMEQUEST 系统中使用 SADUMP 97

第 19 章 分析内核转储 **98**
19.1. 安装 CRASH 工具 98
19.2. 运行和退出 CRASH 工具 98
19.3. 在 CRASH 工具中显示各种指示符 99
19.4. 使用 KERNEL OOPS ANALYZER 102
19.5. KDUMP HELPER 工具 102

第 20 章 使用早期 KDUMP 来捕获引导时间崩溃 **103**
20.1. 启用早期 KDUMP 103

第 21 章 为安全引导签名内核和模块 **105**
21.1. 先决条件 105
21.2. 什么是 UEFI 安全引导 106
21.3. UEFI 安全引导支持 106
21.4. 使用 X.509 密钥验证内核模块的要求 107
21.5. 公钥的源 108
21.6. 生成公钥和私钥对 109
21.7. 系统密钥环输出示例 110
21.8. 通过在 MOK 列表中添加公钥在目标系统中注册公钥 111
21.9. 使用私钥签名内核 112
21.10. 使用私钥签名 GRUB 构建 113
21.11. 使用私钥签名内核模块 114
21.12. 载入经过签名的内核模块 116

第 22 章 更新安全引导撤销列表 **118**
22.1. 安全引导撤销列表 118
22.2. 应用一个在线撤销列表更新 118
22.3. 应用一个离线撤销列表更新 119

第 23 章 使用内核完整性子系统提高安全性 **120**
23.1. 内核完整性子系统 120

23.2. 通过基于 IMA 的 APPRAISAL 启用内核的运行时完整性监控 120
23.3. 使用 IMA 测量启用远程测试 121

第 24 章 扩展、自定义并故障排除内核完整性子系统 122
24.1. 为 IMA APPRAISAL 生成良好的参考值 122
24.2. 编写自定义 IMA 策略 124
24.3. 使用 OPENSSL 创建自定义 IMA 密钥 125
24.4. 加载由自定义 IMA 密钥签名的 IMA 策略 126
24.5. 对 SYSTEMD 无法加载 IMA 策略的故障排除 128
24.6. 签名自定义构建的软件包 129
24.7. 在 IMA 和 FAPOLICYD 间选择 130

第 25 章 使用 SYSTEMD 管理应用程序使用的资源 131
25.1. 资源管理中的 SYSTEMD 角色 131
25.2. 系统源的分发模型 131
25.3. 使用 SYSTEMD 分配系统资源 132
25.4. CGROUPS 的 SYSTEMD 层次结构概述 132
25.5. 列出 SYSTEMD 单元 134
25.6. 查看 SYSTEMD CGROUPS 层次结构 135
25.7. 查看进程的 CGROUP 136
25.8. 监控资源消耗 137
25.9. 使用 SYSTEMD 单元文件为应用程序设置限制 138
25.10. 使用 SYSTEMCTL 命令将限制设置为应用程序 139
25.11. 通过管理器配置设置全局默认 CPU 关联性 139
25.12. 使用 SYSTEMD 配置 NUMA 策略 140
25.13. SYSTEMD 的 NUMA 策略配置选项 141
25.14. 使用 SYSTEMD-RUN 命令创建临时 CGROUP 141
25.15. 删除临时控制组群 142

第 26 章 使用控制组群为应用程序设置系统资源限制 143
26.1. 控制组简介 143
26.2. 内核资源控制器简介 144
26.3. 命名空间简介 145

第 27 章 使用 CGROUPFS 手动管理 CGROUP 147
27.1. 在 CGROUPS-V2 文件系统中创建 CGROUP 和启用控制器 147
27.2. 通过调整 CPU 权重来控制应用程序的 CPU 时间分布 150

第 28 章 使用 EBPF 分析系统性能 154
28.1. 使用 BPFTRACE 软件包 154
28.2. 安装 BCC-TOOLS 软件包 155
28.3. 使用所选 BCC-TOOLS 进行性能调整 155

对红帽文档提供反馈

我们感谢您对我们文档的反馈。让我们了解如何改进它。

通过 Jira 提交反馈（需要帐户）

1. 登录到 Jira 网站。
2. 在顶部导航栏中点 **Create**
3. 在 **Summary** 字段中输入描述性标题。
4. 在 **Description** 字段中输入您对改进的建议。包括文档相关部分的链接。
5. 点对话框底部的 **Create**。

第 1 章 LINUX 内核

了解由红帽（红帽内核）提供和维护的 Linux 内核和 Linux 内核 RPM 软件包。使红帽内核保持更新，这可以确保操作系统具有最新的程序错误修复、性能增强和补丁，并与新硬件兼容。

1.1. 内核是什么

内核是 Linux 操作系统的一个核心部分，其管理系统资源，并在硬件和软件应用程序之间提供一个接口。

红帽内核是一个基于上游 Linux 主线内核的定制内核，红帽工程师对其进行了进一步的开发和强化，专注于稳定性和与最新技术和硬件的兼容性。

红帽内核是以 RPM 格式打包的，以通过 **DNF** 软件包管理器进行升级和验证。

警告

红帽只支持红帽编译的内核。

1.2. RPM 软件包

RPM 软件包由用于安装和删除这些文件的文件和元数据的存档组成。具体来说，RPM 软件包包含以下部分：

- GPG 签名
  GPG 签名用于验证软件包的完整性。
- 标头（软件包元数据）
  RPM 软件包管理器使用此元数据来确定软件包依赖项、安装文件的位置及其他信息。
- payload
  有效负载是一个 **cpio** 归档，其中包含要安装到系统的文件。

RPM 软件包有两种类型。这两种类型都共享文件格式和工具，但内容不同，并实现不同的目的：

- 源 RPM（SRPM）
  SRPM 包含源代码和一个 **spec** 文件，该文件描述了如何将源代码构建为二进制 RPM。另外，SRPM 可以包含源代码的补丁。
- 二进制 RPM
  一个二进制 RPM 包含了根据源代码和补丁构建的二进制文件。

1.3. LINUX 内核 RPM 软件包概述

kernel RPM 是一个元数据软件包，它不包含任何文件，而是保证正确安装了以下子软件包：

kernel-core

提供 Linux 内核的二进制镜像(**vmlinuz**)。

kernel-modules-core

提供基本内核模块,以确保核心功能。这包括最常用硬件正常功能的基本模块。

kernel-modules

提供 **kernel-modules-core** 中不存在的其余内核模块。

kernel-core 和 **kernel-modules-core** 子软件包可以在虚拟和云环境中使用，以便提供一个具有快速引导时间和小磁盘空间的 RHEL 10 内核。对于此类部署，通常不需要 **kernel-modules** 子软件包。

例如，可选内核软件包：

kernel-modules-extra

为不常用的内核模块提供内核模块。在此软件包中加载模块默认被禁用。

kernel-debug

提供了启用了许多调试选项的内核，以便进行内核诊断，以降低性能为代价。

kernel-tools

提供了操作 Linux 内核和支持文档的工具。

kernel-devel

提供了内核标头和 makefile，来足以针对 **kernel** 软件包构建模块。

kernel-abi-stablelists

提供了与 RHEL 内核 ABI 相关的信息，包括外部 Linux 内核模块所需的内核符号的列表和用于辅助实施的 **dnf** 插件。

kernel-headers

包括指定 Linux 内核和用户空间库以及程序间接口的 C 标头文件。头文件定义构建大多数标准程序所需的结构和常量。

kernel-uki-virt

包含 RHEL 内核的统一内核镜像(UKI)。
UKI 将 Linux 内核、**initramfs** （初始 RAM 文件系统）和内核命令行合并到一个签名的二进制文件中，这个二进制文件可以直接从 UEFI 固件引导。

kernel-uki-virt 包含在虚拟化和云环境中运行所需的内核模块，并可以用来代替 **kernel-core** 子软件包。

其他资源

- 什么是 kernel-core、kernel-modules 和 kernel-modules-extras 软件包？

1.4. 显示内核软件包的内容

通过查询存储库，您可以看到内核软件包是否提供了一个特定的文件，如模块。不需要下载或安装软件包来显示文件列表。

使用 **dnf** 工具查询文件列表，例如 **kernel-core**、**kernel-modules-core** 或 **kernel-modules** 软件包的文件列表。请注意，**kernel** 软件包是一个不包含任何文件的元数据软件包。

流程

1. 列出软件包的可用版本：

```
$ dnf repoquery <package_name>
```

2. 显示软件包中的文件列表：

```
$ dnf repoquery -l <package_name>
```

其他资源

- 打包和分发软件

1.5. 安装特定的内核版本

使用 **dnf** 软件包管理器安装新内核。

流程

- 要安装特定的内核版本，请输入以下命令：

```
# dnf install kernel-<version>
```

其他资源

- Red Hat Enterprise Linux 发行日期

1.6. 更新内核

使用 **dnf** 软件包管理器更新内核。

流程

1. 要更新内核，请输入以下命令：

```
# dnf upgrade kernel
```

   此命令将内核以及所有依赖项更新至最新可用版本。

2. 重启您的系统以使更改生效。
   如需更多信息，请参阅您系统上的 **dnf (8)** 手册页。

其他资源

- 使用 DNF 工具管理软件

1.7. 将内核设置为默认

使用 **grubby** 命令行工具和 GRUB 将特定内核设置为默认。

流程

- 使用 **grubby** 工具将内核设置为默认。
  - 输入以下命令，使用 **grubby** 工具将内核设置为默认：

```
# grubby --set-default $kernel_path
```

- 使用 **version** 参数将内核设置为默认。
  - 使用 kernel 关键字列出引导条目，然后将预期的内核设置为默认值：

```
# select k in /boot/vmlinuz-*; do grubby --set-default=$k; break; done
```

> **注意**
>
> 要使用 **title** 参数列出引导条目，请输入 **# grubby --info=ALL | grep title**。

- 仅为下次引导设置默认内核。
  - 输入以下命令，来只为使用 **grub2-reboot** 命令的下一个重启设置默认内核：

```
# grub2-reboot <index|title|id>
```

> **警告**
>
> 小心地为下次启动设置默认内核。安装新的内核 RPM、自构建内核，以及手动将条目添加到 **/boot/loader/entries/** 目录中可能会更改索引值。

第 2 章 64K 页大小内核

kernel-64k 是一个支持 64k 页的额外的可选的 64 位 ARM 架构内核软件包。这个额外内核与 RHEL 10 for ARM 内核一起存在，它支持 4k 页。

最佳的系统性能与不同的内存配置要求直接相关。这些要求由内核的两个变体解决，每个变体适合不同的工作负载。因此，64 位 ARM 硬件上的 RHEL 10 提供两个 MMU 页大小：

- 用于在较小的环境中有效地使用内存的 4K 页内核，
- 用于具有大型连续内存工作集的工作负载的**kernel-64k**。

4k 页内核和 **kernel-64k** 在用户体验方面没有不同，因为用户空间是相同的。您可以选择最适合您情况的变体。

4K 页内核

在较小的环境中，使用 4k 页面来更有效地使用内存，比如在边缘和低成本、小型云实例中。在这些环境中，由于空间、电力和成本的约束，增加物理系统内存量并不实际。另外，并非所有的 64 位 ARM 架构处理器都支持 64k 页大小。
4k 页面内核支持使用 Anaconda 的图形安装，系统或基于云镜像的安装，以及使用 Kickstart 的高级安装。

kernel-64k

64k 页大小内核是 ARM 平台上大型数据集的一个有用的选项。**kernel-64k** 适用于内存密集型工作负载，因为它在整体系统性能方面有显著提升，即在大型数据库、HPC 和高网络性能方面。
您必须在安装时在 64 位 ARM 架构系统上选择页大小。您只能通过将 **kernel-64k** 软件包添加到 **Kickstart** 文件中的软件包列表来通过 Kickstart 安装 **kernel-64k**。

2.1. 按系统构架确定内核页大小

您可以确定不同系统构架的内核页大小。

流程

1. 确定系统架构：

```
# uname -r
6.12.0-55.9.1.el10_0.x86_64
```

在这个输出中，**x86_64** 表示 64 位 Intel 或 AMD 架构。

2. 检查默认页面大小：

```
# getconf PAGE_SIZE
4096
```

在 x86_64 系统上，输出为 4096 B，这意味着默认的页面大小为 4 KB。

在 ppc64le 系统中，输出为 65536 B，这意味着默认的页面大小为 64 KB。

第 3 章 管理内核模块

了解内核模块，包括如何获取模块信息，并执行基本的管理任务。

3.1. 内核模块简介

Red Hat Enterprise Linux 内核可以使用内核模块进行扩展，该模块提供可选的附加功能，而无需重启系统。在 Red Hat Enterprise Linux 10 上，内核模块是构建到压缩的 **<KERNEL_MODULE_NAME>.ko.xz** 对象文件中的额外的内核代码。

可加载的内核模块(LKM)

可以从正在运行的 Linux 内核动态加载和卸载 LKM。您可以添加设备驱动程序或文件系统支持，而无需重启系统或重新编译整个内核。

内核模块启用的最常见功能是：

- 添加用于支持新硬件的设备驱动程序
- 支持文件系统，如 GFS2 或者 NFS
- 系统调用

在现代系统中，在需要时会自动载入内核模块。但在某些情况下，需要手动加载或卸载模块。

与内核类似，模块也接受自定义其行为的参数。

您可以使用内核工具对模块执行以下操作：

- 检查当前运行的模块。
- 检查可用来加载到内核的模块。
- 检查模块接受的参数。
- 启用一种机制，来将内核模块加载和卸载到运行的内核中。

3.2. 内核模块依赖关系

某些内核模块有时依赖一个或多个内核模块。**/lib/modules/<KERNEL_VERSION>/modules.dep** 文件包含相应内核版本的内核模块依赖项的完整列表。

depmod

依赖项文件由 **depmod** 程序生成，包含在 **kmod** 软件包中。**kmod** 提供的很多工具会在执行操作时考虑模块依赖项。因此，很少需要 **手动** 跟踪依赖项。

警告

内核模块的代码在不受限制的模式下在内核空间中执行。请小心您正在加载的模块。

weak-modules

除了 **depmod**， Red Hat Enterprise Linux 还提供 **weak-modules** 脚本，它是 **kmod** 软件包的一部分。**weak-modules** 脚本决定与安装的内核兼容的 kABI 模块。在检查模块内核的兼容性时， **weak-modules** 按照它们构建的内核的从高到低版本处理符号依赖项。它独立于内核版本单独处理每个模块。

其他资源

- Red Hat Enterprise Linux 附带的 weak-modules 脚本的目的是什么？
- 什么是内核应用程序二进制接口(kABI)？（红帽知识库）

3.3. 列出安装的内核

grubby --info=ALL 命令显示 **BLS** 安装上已安装的内核的索引列表。

使用 Boot Loader Specification(BLS)，您可以标准化指定引导条目的方法。BLS 由 systemd-boot 原生支持，GRUB 也可以被配置为使用 BLS。

流程

- 列出已安装的内核：

```
# grubby --info=ALL | grep title
```

显示所有已安装的内核的列表：

```
title="Red Hat Enterprise Linux (6.12.0-55.9.1.el10_0.x86_64) 10.0"
title="Red Hat Enterprise Linux (0-rescue-0d772916a9724907a5d1350bcd39ac92) 10.0"
```

这是 GRUB 菜单中安装的 grubby-8.40-17 内核的列表。

3.4. 列出当前载入的内核模块

查看当前载入的内核模块。

先决条件

- 已安装 **kmod** 软件包。

流程

- 列出所有当前载入的内核模块：

```
$ lsmod

Module                  Size  Used by
fuse                  126976  3
uinput                 20480  1
xt_CHECKSUM            16384  1
ipt_MASQUERADE         16384  1
xt_conntrack           16384  1
ipt_REJECT             16384  1
```

```
nft_counter            16384  16
nf_nat_tftp            16384  0
nf_conntrack_tftp      16384  1 nf_nat_tftp
tun                    49152  1
bridge                 192512  0
stp                    16384  1 bridge
llc                    16384  2 bridge,stp
nf_tables_set          32768  5
nft_fib_inet           16384  1
…
```

在本例中：

a. **Module** 列提供了当前载入的模块的 **名称**。

b. **Size** 列显示每个模块的 **内存** 量（以 KB 为单位）。

c. **Used by** 列显示 **依赖于** 特定模块的模块的编号，以及名称（可选）。

3.5. 显示内核模块信息

使用 **modinfo** 命令显示指定内核模块的一些详细信息。

先决条件

- 已安装 **kmod** 软件包。

流程

- 显示有关任何内核模块的信息：

```
$ modinfo <KERNEL_MODULE_NAME>
```

例如：

```
$ modinfo virtio_net

filename:       /lib/modules/6.12.0-55.9.1.el10_0.x86_64/kernel/drivers/net/virtio_net.ko.xz
license:        GPL
description:    Virtio network driver
rhelversion:    9.0
srcversion:     8809CDDBE7202A1B00B9F1C
alias:          virtio:d00000001v*
depends:        net_failover
retpoline:      Y
intree:         Y
name:           virtio_net
vermagic:       6.12.0-55.9.1.el10_0.x86_64 SMP mod_unload modversions
…
parm:           napi_weight:int
parm:           csum:bool
parm:           gso:bool
parm:           napi_tx:bool
```

您可以查询所有可用模块的信息，无论它们是否已加载。**parm** 条目显示用户可以为模块设置的参数，以及它们预期的值类型。

注意

在输入内核模块的名称时，不要将 **.ko.xz** 扩展附加到名称的末尾。内核模块名称没有扩展。但是，它们相应的文件有扩展：

3.6. 在系统运行时载入内核模块

扩展 Linux 内核功能的最佳方法是加载内核模块。使用 **modprobe** 命令查找并将内核模块载入到当前运行的内核中。

重要

重启系统后，这个过程中描述的更改**不会保留**。有关如何在系统重启后将内核模块载入为**持久性** 的详情，请参考 在系统引导时自动载入内核模块。

先决条件

- 您在系统上具有 root 权限。
- **kmod** 软件包已安装。
- 相应的内核模块没有被载入。要确保这一点，请列出 列出当前加载的内核模块。

流程

1. 选择您要载入的内核模块。
   模块位于 **/lib/modules/$(uname -r)/kernel/<SUBSYSTEM>/** 目录中。
2. 载入相关内核模块：

```
# modprobe <MODULE_NAME>
```

注意

在输入内核模块的名称时，不要将 **.ko.xz** 扩展附加到名称的末尾。内核模块名称没有扩展名，它们对应的文件有。

验证

- （可选）验证相关的模块是否已载入：

```
$ lsmod | grep <MODULE_NAME>
```

如果模块被正确载入了，您可以显示它：

```
$ lsmod | grep serio_raw
serio_raw              16384  0
```

如需更多信息，请参阅系统中的 **modprobe (8)** 手册页。

3.7. 在系统运行时卸载内核模块

要从正在运行的内核中卸载某些内核模块，请使用 **modprobe** 命令，在系统运行时从当前加载的内核查找和卸载内核模块。

> **警告**
>
> 您不能卸载在运行的系统中活跃的内核模块。这可能导致系统不稳定或无法运行。

> **重要**
>
> 卸载不活跃的内核模块不会禁用为引导时自动载入配置的模块。系统重启时，这些模块将再次被自动载入。有关如何防止此结果的详情，请参考 防止内核模块在系统引导时被自动载入。

先决条件

- 您在系统上具有 root 权限。
- **kmod** 软件包已安装。

流程

1. 列出所有载入的内核模块：

```
# lsmod
```

2. 选择要卸载的内核模块。
   如果内核模块有依赖项，请在卸载内核模块前卸载它们。有关识别使用依赖项的模块的详情，请参阅列出当前载入的内核模块和内核模块依赖项。

3. 卸载相关内核模块：

```
# modprobe -r <MODULE_NAME>
```

   在输入内核模块的名称时，不要将 **.ko.xz** 扩展附加到名称的末尾。内核模块名称没有扩展名，它们对应的文件有。

验证

- （可选）验证相关模块是否已卸载：

```
$ lsmod | grep <MODULE_NAME>
```

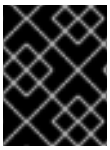
  如果模块被成功卸载，这个命令不显示任何输出。

  如需更多信息，请参阅系统中的 **modprobe (8)** 手册页。

3.8. 在启动过程早期卸载内核模块

在某些情况下，例如，当内核模块有导致系统变得无响应的代码时，用户无法达到永久禁用恶意内核模块的阶段时，您可能需要在引导过程的早期卸载内核模块。要临时阻止内核模块的加载，您可以使用引导装载程序。

您可以在引导序列继续前编辑相关的引导装载程序条目，来卸载所需的内核模块。



重要

此流程中描述的更改在系统重启后 **不会持久**。有关如何在 denylist 中添加内核模块的详情，请参考 防止内核模块在系统引导时被自动加载。

先决条件

- 您有一个您打算防止其载入的可加载内核模块。

流程

1. 将系统启动到引导装载程序中。
2. 使用光标键突出显示相关的引导装载程序条目。
3. 按 **e** 键编辑条目。
4. 使用光标键导航到以 **linux** 开头的行。
5. 将 **modprobe.blacklist=*module_name*** 附加到行尾。
   serio_raw 内核模块演示了一个要在引导过程早期卸载的恶意模块。
6. 按 **Ctrl**+**X** 使用修改后的配置启动。

验证

- 系统引导后，验证相关内核模块是否没有载入：

```
# lsmod | grep serio_raw
```

其他资源

- 管理内核模块

3.9. 在系统引导时自动载入内核模块

配置内核模块，来在引导过程中自动载入它。

先决条件

- 根权限
- **kmod** 软件包已安装。

步骤

1. 选择您要在引导过程中载入的内核模块。
   模块位于 **/lib/modules/$(uname -r)/kernel/<SUBSYSTEM>/** 目录中。

2. 为模块创建配置文件：

```
# echo <MODULE_NAME> > /etc/modules-load.d/<MODULE_NAME>.conf
```

注意

在输入内核模块的名称时，不要将 **.ko.xz** 扩展附加到名称的末尾。内核模块名称没有扩展名，它们对应的文件有。

验证

1. 重启后，验证相关模块是否已载入：

```
$ lsmod | grep <MODULE_NAME>
```

重要

重启系统后，这个过程中描述的更改**将会保留**。

详情请查看您系统上的 **modules-load.d (5)** 手册页。

3.10. 防止在系统引导时自动载入内核模块

您可以通过使用相应的命令在 **modprobe** 配置文件中列出模块，来防止系统在引导过程中自动载入内核模块。

先决条件

- 此流程中的命令需要 root 权限。使用 **su -** 切换到 root 用户，或在命令前使用 **sudo** 。
- **kmod** 软件包已安装。
- 确定您当前的系统配置不需要您计划拒绝的内核模块。

流程

1. 使用 **lsmod** 命令列出载入到当前运行的内核的模块：

```
$ lsmod
Module                  Size  Used by
tls                   131072  0
uinput                 20480  1
snd_seq_dummy          16384  0
snd_hrtimer            16384  1
…
```

在输出中，识别您要防止加载的模块。

- 或者，识别您要防止在 **/lib/modules/*<KERNEL-VERSION>*/kernel/*<SUBSYSTEM>*/** 目录中加载的而未加载的内核模块，例如：

```
$ ls /lib/modules/6.12.0-55.9.1.el10_0.x86_64/kernel/crypto/
ansi_cprng.ko.xz        chacha20poly1305.ko.xz  md4.ko.xz
serpent_generic.ko.xz
anubis.ko.xz            cmac.ko.xz…
```

2. 创建一个配置文件作为 denylist ：

```
# touch /etc/modprobe.d/denylist.conf
```

3. 在您选择的文本编辑器中，使用 **blacklist** 配置命令将您要从自动加载到内核中排除的模块的名称组合在一起，例如：

```
# Prevents <KERNEL-MODULE-1> from being loaded
blacklist <MODULE-NAME-1>
install <MODULE-NAME-1> /bin/false

# Prevents <KERNEL-MODULE-2> from being loaded
blacklist <MODULE-NAME-2>
install <MODULE-NAME-2> /bin/false
…
```

因为 **blacklist** 命令不会阻止模块作为不在 denylist 中的另一个内核模块的依赖项加载，所以您还必须定义 **install** 行。在这种情况下，系统运行 **/bin/false**，而不是安装模块。以哈希符号开头的行是注释，您可以用来使文件更易读。

注意

在输入内核模块的名称时，不要将 **.ko.xz** 扩展附加到名称的末尾。内核模块名称没有扩展名，它们对应的文件有。

4. 在重建前，创建当前初始 RAM 磁盘镜像的一个备份副本：

```
# cp /boot/initramfs-$(uname -r).img /boot/initramfs-$(uname -r).bak.$(date +%m-%d-%H%M%S).img
```

- 或者，创建与您要阻止内核模块自动载入的内核版本对应的初始 RAM 磁盘镜像的一个备份副本：

```
# cp /boot/initramfs-<VERSION>.img /boot/initramfs-<VERSION>.img.bak.$(date +%m-%d-%H%M%S)
```

5. 生成一个新的初始 RAM 磁盘镜像以应用更改：

```
# dracut -f -v
```

- 如果您为与您系统当前使用的内核版本不同的系统构建初始 RAM 磁盘镜像，请指定目标 **initramfs** 和内核版本：

```
# dracut -f -v /boot/initramfs-<TARGET-VERSION>.img <CORRESPONDING-TARGET-KERNEL-VERSION>
```

6. 重启系统：

```
$ reboot
```

重要

此流程中描述的更改将在重启后**生效并保留**。如果您在 denylist 中错误地列出了关键内核模块，您可以将系统切换到不稳定或无法正常工作的状态。

其他资源

- 如何防止内核模块自动加载？（红帽知识库）

3.11. 编译自定义的内核模块

您可以根据硬件和软件级别的各种配置的要求构建一个采样内核模块。

先决条件

- **kernel-devel**、**gcc** 和 **elfutils-libelf-devel** 软件包已安装。

```
# *dnf install kernel-devel-$(uname -r) gcc elfutils-libelf-devel*
```

- 您有 root 权限。
- 您创建了 **/root/testmodule/** 目录,在此编译自定义的内核模块。

步骤

1. 创建包含以下内容的 **/root/testmodule/test.c** 文件：

```
#include <linux/module.h>
#include <linux/kernel.h>

int init_module(void)
   { printk("Hello World\n This is a test\n"); return 0; }

void cleanup_module(void)
   { printk("Good Bye World"); }

MODULE_LICENSE("GPL");
```

test.c 文件是一个源文件，其向内核模块提供主要功能。出于组织需要，该文件已创建在专用的 **/root/testmodule/** 目录中。在模块编译后，**/root/testmodule/** 目录将包含多个文件。

test.c 文件包含来自系统库的文件：

- 示例代码中的 **printk（）** 函数需要 **linux/kernel.h** 头文件。
- **linux/module.h** 文件包含函数声明和用 C 编程语言编写的在多个源文件之间共享的宏定义。

2. 按照 **init_module （）** 和 **cleanup_module （）** 函数启动和结束内核日志记录函数 **printk （）**，后者会打印文本。

3. 使用以下内容创建 **/root/testmodule/Makefile** 文件。

```
obj-m := test.o
```

Makefile 包含编译器的说明，以生成名为 **test.o** 的对象文件。**obj-m** 指令指定生成的 **test.ko** 文件将编译为可加载的内核模块。或者，**obj-y** 指令可指示将 **test.ko** 作为内置内核模块构建。

4. 编译内核模块：

```
# make -C /lib/modules/$(uname -r)/build M=/root/testmodule modules
make: Entering directory '/usr/src/kernels/6.12.0-55.9.1.el10_0.x86_64'
  CC [M]  /root/testmodule/test.o
  MODPOST /root/testmodule/Module.symvers
  CC [M]  /root/testmodule/test.mod.o
  LD [M]  /root/testmodule/test.ko
  BTF [M] /root/testmodule/test.ko
Skipping BTF generation for /root/testmodule/test.ko due to unavailability of vmlinux
make: Leaving directory '/usr/src/kernels/6.12.0-55.9.1.el10_0.x86_64'
```

编译器将它们链接成最终内核模块(**test.ko**)之前，会为每个源文件(**test.c)**创建一个对象文件(**test.c**)来作为中间步骤。

成功编译后，**/root/testmodule/** 包含与编译的自定义内核模块相关的其他文件。已编译的模块本身由 **test.ko** 文件表示。

验证

1. 可选：检查 **/root/testmodule/** 目录的内容：

```
# ls -l /root/testmodule/
total 152
-rw-r—r--. 1 root root    16 Jul 26 08:19 Makefile
-rw-r—r--. 1 root root    25 Jul 26 08:20 modules.order
-rw-r—r--. 1 root root     0 Jul 26 08:20 Module.symvers
-rw-r—r--. 1 root root   224 Jul 26 08:18 test.c
-rw-r—r--. 1 root root 62176 Jul 26 08:20 test.ko
-rw-r—r--. 1 root root    25 Jul 26 08:20 test.mod
-rw-r—r--. 1 root root   849 Jul 26 08:20 test.mod.c
-rw-r—r--. 1 root root 50936 Jul 26 08:20 test.mod.o
-rw-r—r--. 1 root root 12912 Jul 26 08:20 test.o
```

2. 将内核模块复制到 **/lib/modules/$(uname -r)/** 目录中：

```
# cp /root/testmodule/test.ko /lib/modules/$(uname -r)/
```

3. 更新模块依赖项列表：

```
# depmod -a
```

4. 载入内核模块：

```
# modprobe -v test
insmod /lib/modules/6.12.0-55.9.1.el10_0.x86_64/test.ko
```

5. 验证内核模块是否成功载入：

```
# lsmod | grep test
test                   16384  0
```

6. 从内核环缓冲中读取最新的消息：

```
# dmesg
[74422.545004] Hello World
               This is a test
```

第 4 章 配置内核命令行参数

使用内核命令行参数，您可以在引导时更改 Red Hat Enterprise Linux 内核的某些方面的行为。作为系统管理员，您可以控制在引导时设置哪些选项。请注意，某些内核行为只能在引导时设置。

重要

通过修改内核命令行参数来更改系统的行为可能会对您的系统造成负面影响。在生产环境中部署前，始终测试更改。如需进一步指导，请联系红帽支持团队。

4.1. 什么是内核命令行参数

使用内核命令行参数，您可以覆盖默认值并设置特定的硬件设置。在引导时，您可以配置以下功能：

- Red Hat Enterprise Linux 内核
- 初始 RAM 磁盘
- 用户空间特性

默认情况下，使用 GRUB 引导装载程序的系统的内核命令行参数是在每个内核引导条目的引导条目配置文件中定义的。

您可以使用 **grubby** 工具操作引导装载程序配置文件。使用 **grubby**，您可以执行以下操作：

- 更改默认的引导条目。
- 从 GRUB 菜单条目中添加或删除参数。

4.2. 了解引导条目

引导条目是存储在配置文件中的选项的集合，并绑定到特定的内核版本。在实践中，您的引导条目至少与您所安装的系统数量相同。引导条目配置文件位于 **/boot/loader/entries/** 目录中：

```
d8712ab6d4f14683c5625e87b52b6b6e-6.12.0.el10_0.x86_64.conf
```

文件名由存储在 **/etc/machine-id** 文件中的一个机器 ID 和内核版本组成。

引导条目配置文件包含有关内核版本、初始 ramdisk 镜像和内核命令行参数的信息。引导条目配置的示例内容如下：

```
title Red Hat Enterprise Linux (6.12.0-0.el10_0.x86_64) 10.0
version 6.12.0-0.el10_0.x86_64
linux /vmlinuz-6.12.0-0.el10_0.x86_64
initrd /initramfs-6.12.0-0.el10_0.x86_64.img
options root=/dev/mapper/rhel_kvm--02--guest08-root ro crashkernel=2G-64G:256M,64G-:512M
resume=/dev/mapper/rhel_kvm--02--guest08-swap rd.lvm.lv=rhel_kvm-02-guest08/root
rd.lvm.lv=rhel_kvm-02-guest08/swap console=ttyS0,115200
grub_users $grub_users
grub_arg --unrestricted
grub_class kernel
```

4.3. 为所有引导条目更改内核命令行参数

更改系统上所有引导条目的内核命令行参数。

重要

当在 Red Hat Enterprise Linux 10 系统中安装较新版本的内核时，**grubby** 工具会传递来自之前内核版本的内核命令行参数。

先决条件

- **grubby** 工具已安装在您的系统上。
- **zipl** 工具已安装在 IBM Z 系统上。

流程

- 添加参数：

```
# grubby --update-kernel=ALL --args="<NEW_PARAMETER>"
```

对于使用 GRUB 引导装载程序的系统，在使用 zIPL 引导装载程序的 IBM Z 上，该命令会向每个 **/boot/loader/entries/<*ENTRY*>.conf** 文件添加新内核参数。

  - 在 IBM Z 上，更新引导菜单：

```
# zipl
```

- 删除参数：

```
# grubby --update-kernel=ALL --remove-args="<PARAMETER_TO_REMOVE>"
```

  - 在 IBM Z 上，更新引导菜单：

```
# zipl
```

注意

不需要使用 GRUB 引导装载程序更新系统的引导菜单。

其他资源

- 什么是内核命令行参数

4.4. 为单一引导条目更改内核命令行参数

对系统上单个引导条目的内核命令行参数进行更改。

先决条件

- **grubby** 和 **zipl** 工具已安装在您的系统上。

流程

- 添加参数：

```
# grubby --update-kernel=/boot/vmlinuz-$(uname -r) --args="<NEW_PARAMETER>"
```

  - 在 IBM Z 上，更新引导菜单：

```
# grubby --args="<NEW_PARAMETER> --update-kernel=ALL --zipl
```

- 删除参数：

```
# grubby --update-kernel=/boot/vmlinuz-$(uname -r) --remove-args="
<PARAMETER_TO_REMOVE>"
```

  - 在 IBM Z 上，更新引导菜单：

```
# grubby --args="<NEW_PARAMETER> --update-kernel=ALL --zipl
```

重要

- **grubby** 修改单个内核引导条目的内核命令行参数并将其存储在 **/boot/loader/entries/<*ENTRY*>.conf** 文件中。

4.5. 在引导时临时更改内核命令行参数

通过在单个引导过程中更改内核参数，对内核惨淡条目进行临时更改。这个流程只适用于单个引导，且在系统重启后不持久。

流程

1. 引导到 GRUB 引导菜单。
2. 选择您要启动的内核。
3. 按 **e** 键编辑内核参数。
4. 通过移动光标来找到内核命令行。
5. 将光标移至行末。
6. 根据需要编辑内核参数。例如，要在紧急模式下运行系统，请在 **linux** 行末尾添加 **emergency** 参数：

```
linux   ($root)/vmlinuz-6.12.0-0.el10_0.x86_64 root=/dev/mapper/rhel-root ro
crashkernel=2G-64G:256M,64G-:512M resume=/dev/mapper/rhel-swap rd.lvm.lv=rhel/root
rd.lvm.lv=rhel/swap rhgb quiet emergency
```

   要启用系统消息，请删除 **rhgb** 和 **quiet** 参数。

7. 按 **Ctrl**+**x** 使用所选内核以及修改的命令行参数进行引导。

重要

如果按 **Esc** 键离开命令行编辑，它将丢弃用户做的所有更改。

4.6. 配置 GRUB 设置以启用串行控制台连接

当您需要连接到无头服务器或嵌入式系统，且网络中断时，串行控制台非常有用。或者，当您需要避免安全规则并获得在不同系统上的登录访问权限时。

您需要配置一些默认的 GRUB 设置，以使用串行控制台连接。

先决条件

- 您在系统上具有 root 权限。

流程

1. 将下面两行添加到 **/etc/default/grub** 文件中：

```
GRUB_TERMINAL="serial"
GRUB_SERIAL_COMMAND="serial --speed=9600 --unit=0 --word=8 --parity=no --stop=1"
```

   第一行将禁用图形终端。**GRUB_TERMINAL** 键覆盖 **GRUB_TERMINAL_INPUT** 和 **GRUB_TERMINAL_OUTPUT** 键的值。

   第二行调整了波特率(**--speed**)，奇偶校验和其他值以适合您的环境和硬件。请注意，对于以下日志文件等任务，最好使用更高的波特率，如 115200。

2. 更新 GRUB 配置文件：

```
# grub2-mkconfig -o /boot/grub2/grub.cfg
```

   这适用于基于 BIOS 和 UEFI 的机器。

3. 重启系统以使更改生效。

4.7. 使用 GRUB 配置文件更改引导条目

/etc/default/grub GRUB 配置文件包含 **GRUB_CMDLINE_LINUX** 键，它列出了要添加到 Linux 内核的引导条目的内核命令行参数。例如：

```
GRUB_CMDLINE_LINUX="crashkernel=2G-64G:256M,64G-:512M resume=/dev/mapper/rhel-swap
rd.lvm.lv=rhel/root rd.lvm.lv=rhel/swap"
```

要更改引导条目，请使用 **GRUB_CMDLINE_LINUX** 值的内容覆盖 Boot Loader 规范(BLS)片断。

先决条件

- 全新的 Red Hat Enterprise Linux 10 安装。

流程

1. 使用 **grubby** 在安装后脚本中为各个内核添加或删除内核参数：

```
# grubby --update-kernel <PATH_TO_KERNEL> --args "<NEW_ARGUMENTS>"
```

   例如，在所选内核中添加 **noapic** 参数：

```
# grubby --update-kernel /boot/vmlinuz-6.12.0-0.el10_0.x86_64 --args "noapic"
```

参数传播到 BLS 代码片段，但不传播到 **/etc/default/grub** 文件中。

2. 使用 **/etc/default/grub** 文件中 **GRUB_CMDLINE_LINUX** 值的内容覆盖 BLS 片断：

```
# grub2-mkconfig -o /boot/grub2/grub.cfg --update-bls-cmdline
Generating grub configuration file …
Adding boot menu entry for UEFI Firmware Settings …
done
```

注意

其他更改，如对 **GRUB_TIMEOUT** 键的更改（也包含在 **/etc/default/grub** GRUB 配置文件中），通过执行 **grub2-mkconfig** 命令被传播到新的 **grub.cfg** 文件中。

验证

1. 重启您的系统。
2. 验证参数是否包含在 **/proc/cmdline** 文件中。
   例如，如果您添加了 **noapic** ：

```
BOOT_IMAGE=(hd0,gpt2)/vmlinuz-6.12.0-0.el10_0.x86_64 root=/dev/mapper/RHELCSB-
Root ro vconsole.keymap=us crashkernel=2G-64G:256M,64G-:512M
rd.lvm.lv=RHELCSB/Root rd.luks.uuid=luks-d8a28c4c-96aa-4319-be26-96896272151d rhgb
quiet noapic rd.luks.key=d8a28c4c-96aa-4319-be26-
96896272151d=/keyfile:UUID=c47d962e-4be8-41d6-8216-8cf7a0d3b911 ipv6.disable=1
```

第 5 章 在运行时配置内核参数

作为系统管理员，您可以修改 Red Hat Enterprise Linux 内核在运行时行为的很多方面。使用 **sysctl** 命令，并修改 **/etc/sysctl.d/** 和 **/proc/sys/** 目录中的配置文件来在运行时配置内核参数。

重要

在产品系统中配置内核参数需要仔细规划。非计划的更改可能导致内核不稳定，需要重启系统。在更改任何内核值之前，验证您是否正在使用有效选项。

有关在 IBM DB2 上调整内核的更多信息，请参阅 为 IBM DB2 调整 Red Hat Enterprise Linux 。

5.1. 什么是内核参数

内核参数是可调整的值，您可以在系统运行时调整。请注意，要使更改生效，您不需要重启系统或重新编译内核。

内核参数和内核命令行参数之间的区别是 ；内核参数可以使用所有选项配置 Linux 内核，而内核命令行参数是在引导时传递给内核的特定参数，允许运行时配置，而无需内核重新编译。

可以通过以下方法处理内核参数：

- **sysctl** 命令
- 挂载于 **/proc/sys/** 目录的虚拟文件系统
- **/etc/sysctl.d/** 目录中的配置文件

Tunables 被内核子系统划分为不同的类。Red Hat Enterprise Linux 有以下可调整类：

表 5.1. sysctl 类表

可调整类	子系统
abi	执行域和个人
crypto	加密接口
debug	内核调试接口
dev	特定于设备的信息
fs	全局和特定文件系统的 tunables
内核	全局内核 tunables
net	网络 tunables
sunrpc	Sun 远程过程调用 (NFS)

可调整类	子系统
user	用户命名空间限制
vm	调整和管理内存、缓冲和缓存

5.2. 使用 SYSCTL 临时配置内核参数

使用 **sysctl** 命令在运行时临时设置内核参数。命令也可用于列出和过滤可调项。

先决条件

- 您在系统上具有 root 权限。

流程

1. 列出所有参数及其值。

```
# sysctl -a
```

> **注意**
>
> **sysctl -a** 命令显示内核参数，这些参数可在运行时和引导时调整。

2. 要临时配置一个参数，请输入：

```
# sysctl <TUNABLE_CLASS>.<PARAMETER>=<TARGET_VALUE>
```

这个示例在系统运行时更改参数值。更改会立即生效，不需要系统重启。

> **注意**
>
> 在系统重启后，所在的改变会返回到默认状态。

其他资源

- 使用 /etc/sysctl.d/ 中的配置文件调整内核参数

5.3. 使用 SYSCTL 永久配置内核参数

使用 **sysctl** 命令永久设置内核参数。

先决条件

- 您在系统上具有 root 权限。

流程

1. 列出所有参数：

```
# sysctl -a
```

该命令显示所有可在运行时配置的内核参数。

2. 永久配置一个参数：

```
# sysctl -w <TUNABLE_CLASS>.<PARAMETER>=<TARGET_VALUE> >> /etc/sysctl.conf
```

示例命令会更改可调值，并将其写入 **/etc/sysctl.conf** 文件，该文件会覆盖内核参数的默认值。更改会立即并永久生效，无需重启。

注意

要永久修改内核参数，您还可以手动更改 **/etc/sysctl.d/** 目录中的配置文件。

其他资源

- 使用 /etc/sysctl.d/ 中的配置文件调整内核参数

5.4. 使用 /ETC/SYSCTL.D/ 中的配置文件调整内核参数

您必须手动修改 **/etc/sysctl.d/** 目录中的配置文件，来永久设置内核参数。

先决条件

- 您在系统上具有 root 权限。

流程

1. 在 **/etc/sysctl.d/** 中创建新配置文件：

```
# vim /etc/sysctl.d/<some_file.conf>
```

2. 包括内核参数，一行一个：

```
<TUNABLE_CLASS>.<PARAMETER>=<TARGET_VALUE>
<TUNABLE_CLASS>.<PARAMETER>=<TARGET_VALUE>
```

3. 保存配置文件。
4. 重启机器以使更改生效。
   - 或者，在不重启的情况下应用更改：

```
# sysctl -p /etc/sysctl.d/<some_file.conf>
```

使用此命令，您可以从之前创建的配置文件中读取值。

如需更多信息，请参阅系统中的 **sysctl (8)** 和 **sysctl.d (5)** man page。

5.5. 通过 /PROC/SYS/ 临时配置内核参数

通过 **/proc/sys/** 虚拟文件系统目录中的文件临时设置内核参数。

先决条件

- 您在系统上具有 root 权限。

流程

1. 识别您要配置的内核参数：

```
# ls -l /proc/sys/<TUNABLE_CLASS>/
```

命令返回的可写入文件可以用来配置内核。具有只读权限的文件提供了对当前设置的反馈。

2. 为 kernel 参数分配目标值：

```
# echo <TARGET_VALUE> > /proc/sys/<TUNABLE_CLASS>/<PARAMETER>
```

使用命令应用的配置更改不是永久的，系统重启后会消失。

验证

1. 验证新设置的内核参数的值：

```
# cat /proc/sys/<TUNABLE_CLASS>/<PARAMETER>
```

第 6 章 使用 RHEL 系统角色永久配置内核参数

您可以使用 **kernel_settings** RHEL 系统角色一次在多个客户端上配置内核参数。这个解决方案：

- 提供带有有效输入设置的友好接口。
- 保留所有预期的内核参数。

从控制计算机运行 **kernel_settings** 角色后，内核参数将立即应用于受管系统，并在重新启动后保留。

重要

请注意，通过 RHEL 渠道交付的 RHEL 系统角色作为默认 AppStream 存储库中的 RPM 软件包提供给 RHEL 客户。RHEL 系统角色也可以作为一个集合提供给具有通过 Automation Hub 的 Ansible 订阅的客户。

6.1. 使用 KERNEL_SETTINGS RHEL 系统角色应用所选的内核参数

您可以使用 **kernel_settings** RHEL 系统角色在多个受管操作系统上远程配置各种内核参数，并具有持久效果。例如，您可以配置：

- 透明巨页，通过减少管理较小页的开销来提高性能。
- 通过具有回环接口的网络传输的最大数据包大小。
- 对文件的限制，可以同时打开。

先决条件

- 您已准备好控制节点和受管节点
- 以可在受管主机上运行 playbook 的用户登录到控制节点。
- 用于连接到受管节点的帐户具有 **sudo** 权限。

流程

1. 创建一个包含以下内容的 playbook 文件，如 ~/**playbook.yml** ：

```
---
- name: Configuring kernel settings
  hosts: managed-node-01.example.com
  tasks:
    - name: Configure hugepages, packet size for loopback device, and limits on
simultaneously open files.
      ansible.builtin.include_role:
        name: redhat.rhel_system_roles.kernel_settings
      vars:
        kernel_settings_sysctl:
          - name: fs.file-max
            value: 400000
          - name: kernel.threads-max
            value: 65536
        kernel_settings_sysfs:
```

```
    - name: /sys/class/net/lo/mtu
      value: 65000
  kernel_settings_transparent_hugepages: madvise
  kernel_settings_reboot_ok: true
```

示例 playbook 中指定的设置包括以下内容：

kernel_settings_sysfs: *<list_of_sysctl_settings>*

sysctl 设置的 YAML 列表以及您要分配给这些设置的值。

kernel_settings_transparent_hugepages: *<value>*

控制内存子系统透明巨页(THP)设置。您可以禁用 THP 支持(**never**)、在系统范围内启用(**always**)或在 **MAD_HUGEPAGE** 区域内启用(**madvise**)。

kernel_settings_reboot_ok: *<true|false>*

默认值为 **false**。如果设置为 **true**，则系统角色将确定是否需要重启受管主机，以使请求的更改生效，并重启它。如果设置为 **false**，则角色将返回值为 **true** 的变量 **kernel_settings_reboot_required**，表示需要重启。在这种情况下，用户必须手动重启受管节点。
有关 playbook 中使用的所有变量的详情，请查看控制节点上的 **/usr/share/ansible/roles/rhel-system-roles.kdump/README.md** 文件。

2. 验证 playbook 语法：

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

请注意，这个命令只验证语法，不会防止错误但有效的配置。

3. 运行 playbook：

```
$ ansible-playbook ~/playbook.yml
```

验证

- 验证受影响的内核参数：

```
# ansible managed-node-01.example.com -m command -a 'sysctl fs.file-max
kernel.threads-max net.ipv6.conf.lo.mtu'
# ansible managed-node-01.example.com -m command -a 'cat
/sys/kernel/mm/transparent_hugepage/enabled'
```

第 7 章 使用 RHEL 系统角色配置 GRUB 2 引导装载程序

通过使用 **bootloader** RHEL 系统角色，您可以自动化与 GRUB2 引导装载程序相关的配置和管理任务。

此角色目前支持配置在以下 CPU 构架上运行的 GRUB2 引导装载程序：

- AMD 和 Intel 64 位构架(x86-64)
- 64 位 ARM 架构(ARMv8.0)
- IBM Power Systems, Little Endian(POWER9)

7.1. 使用 BOOTLOADER RHEL 系统角色更新现有的引导装载程序条目

您可以使用 **bootloader** RHEL 系统角色，以自动的方式更新 GRUB2 引导菜单中的现有条目。这样，您可以有效地传递可以优化系统性能或行为的特定的内核命令行参数。

例如，如果您使用不需要来自内核和 init 系统的详细引导消息的系统，请使用 **bootloader** 将 **quiet** 参数应用到受管节点上的现有引导装载程序条目中，以实现更干净、更整洁、更用户友好的引导体验。

先决条件

- 您已准备好控制节点和受管节点
- 以可在受管主机上运行 playbook 的用户登录到控制节点。
- 用于连接到受管节点的帐户具有 **sudo** 权限。
- 您确定了与您要更新的引导装载程序条目对应的内核。

流程

1. 创建一个包含以下内容的 playbook 文件，如 ~/**playbook.yml** ：

```
---
- name: Configuration and management of GRUB2 boot loader
  hosts: managed-node-01.example.com
  tasks:
    - name: Update existing boot loader entries
      ansible.builtin.include_role:
        name: redhat.rhel_system_roles.bootloader
      vars:
        bootloader_settings:
          - kernel:
              path: /boot/vmlinuz-6.12.0-0.el10_0.aarch64
            options:
              - name: quiet
                state: present
        bootloader_reboot_ok: true
```

示例 playbook 中指定的设置包括以下内容：

kernel

指定与您要更新的引导装载程序条目连接的内核。

options

指定要更新您所选的引导装载程序条目（内核）的内核命令行参数。

bootloader_reboot_ok: *true*

角色检测到需要重启才能使更改生效，并执行受管节点的重启。

有关 playbook 中使用的所有变量的详情，请查看控制节点上的 **/usr/share/ansible/roles/rhel-system-roles.bootloader/README.md** 文件。

2. 验证 playbook 语法：

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

请注意，这个命令只验证语法，不会防止错误但有效的配置。

3. 运行 playbook：

```
$ ansible-playbook ~/playbook.yml
```

验证

- 检查您指定的引导装载程序条目是否有更新的内核命令行参数：

```
# ansible managed-node-01.example.com -m ansible.builtin.command -a 'grubby --info=ALL'
managed-node-01.example.com | CHANGED | rc=0 >>
...
index=1
kernel="/boot/vmlinuz-6.12.0-0.el10_0.aarch64"
args="ro crashkernel=2G-4G:256M,4G-64G:320M,64G-:576M rd.lvm.lv=rhel/root
rd.lvm.lv=rhel/swap $tuned_params quiet"
root="/dev/mapper/rhel-root"
initrd="/boot/initramfs-6.12.0-0.el10_0.aarch64.img $tuned_initrd"
title="Red Hat Enterprise Linux (6.12.0-0.el10_0.aarch64) 10"
id="2c9ec787230141a9b087f774955795ab-6.12.0-0.el10_0.aarch64"
...
```

7.2. 使用 BOOTLOADER RHEL 系统角色，使用密码保护引导菜单

您可以使用 **bootloader** RHEL 系统角色，以自动的方式为 GRUB2 引导菜单设置密码。这样，您可以有效地防止未经授权的用户修改引导参数，并更好地控制系统引导。

先决条件

- 您已准备好控制节点和受管节点
- 以可在受管主机上运行 playbook 的用户登录到控制节点。
- 用于连接到受管节点的帐户具有 **sudo** 权限。

步骤

1. 将您的敏感变量存储在一个加密文件中：

a. 创建 vault ：

```
$ ansible-vault create ~/vault.yml
New Vault password: <vault_password>
Confirm New Vault password: <vault_password>
```

b. 在 **ansible-vault create** 命令打开编辑器后，以 ***<key>: <value>*** 格式输入敏感数据：

```
pwd: <password>
```

c. 保存更改，并关闭编辑器。Ansible 加密 vault 中的数据。

2. 创建一个包含以下内容的 playbook 文件，如 ~/**playbook.yml** ：

```
---
- name: Configuration and management of GRUB2 boot loader
  hosts: managed-node-01.example.com
  vars_files:
    - ~/vault.yml
  tasks:
    - name: Set the bootloader password
      ansible.builtin.include_role:
        name: redhat.rhel_system_roles.bootloader
      vars:
        bootloader_password: "{{ pwd }}"
        bootloader_reboot_ok: true
```

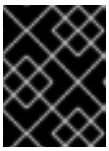
示例 playbook 中指定的设置包括以下内容：

bootloader_password: *"{{ pwd }}"*

变量确保使用密码保护引导参数。

bootloader_reboot_ok: *true*

角色检测到需要重启才能使更改生效，并执行受管节点的重启。



重要

更改引导装载程序密码不是幂等事务。这意味着，如果您再次应用同样的 Ansible playbook，则结果将不一样，并且受管节点的状态将改变。

有关 playbook 中使用的所有变量的详情，请查看控制节点上的 **/usr/share/ansible/roles/rhel-system-roles.bootloader/README.md** 文件。

3. 验证 playbook 语法：

```
$ ansible-playbook --syntax-check --ask-vault-pass ~/playbook.yml
```

请注意，这个命令只验证语法，不会防止错误但有效的配置。

4. 运行 playbook：

```
$ ansible-playbook --ask-vault-pass ~/playbook.yml
```

验证

验证

1. 在 GRUB2 引导菜单屏幕期间，在受管节点上按 **e** 键进行编辑。

2. 系统将提示您输入用户名和密码：

输入 username: root

引导装载程序用户名始终是 **root**，您不需要在 Ansible playbook 中指定它。

输入 password: *<password>*

引导装载程序密码对应于您在 **vault.yml** 文件中定义的 **pwd** 变量。

3. 您可以查看或编辑特定引导装载程序条目的配置：

其他资源

- Ansible vault

7.3. 使用 BOOTLOADER RHEL 系统角色为引导装载程序菜单设置超时

您可以使用 **bootloader** RHEL 系统角色，以自动的方式为 GRUB2 引导装载程序菜单配置超时。这样，您可以有效地更新一段时间，在此期间，您可以出于不同目的进行干预并选择一个非默认引导条目。

先决条件

- 您已准备好控制节点和受管节点
- 以可在受管主机上运行 playbook 的用户登录到控制节点。
- 用于连接到受管节点的帐户具有 **sudo** 权限。

流程

1. 创建一个包含以下内容的 playbook 文件，如 ~/**playbook.yml** ：

```
---
- name: Configuration and management of GRUB2 boot loader
  hosts: managed-node-01.example.com
  tasks:
    - name: Update the boot loader timeout
      ansible.builtin.include_role:
        name: redhat.rhel_system_roles.bootloader
      vars:
        bootloader_timeout: 10
```

示例 playbook 中指定的设置包括以下内容：

bootloader_timeout: 10

输入一个整数，以控制在引导默认条目之前 GRUB2 引导装载程序菜单显示多长时间。

有关 playbook 中使用的所有变量的详情，请查看控制节点上的 **/usr/share/ansible/roles/rhel-system-roles.bootloader/README.md** 文件。

2. 验证 playbook 语法：

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

请注意，这个命令只验证语法，不会防止错误但有效的配置。

3. 运行 playbook：

```
$ ansible-playbook ~/playbook.yml
```

验证

1. 远程重启受管节点：

```
# ansible managed-node-01.example.com -m ansible.builtin.reboot
managed-node-01.example.com | CHANGED => {
    "changed": true,
    "elapsed": 21,
    "rebooted": true
}
```

2. 在受管节点上，观察 GRUB2 引导菜单屏幕。

突出显示的将在 10s 后自动执行的条目

在 GRUB2 自动使用默认条目之前，此引导菜单显示多长时间。

- 替代方案：您可以远程查询受管节点的 **/boot/grub2/grub.cfg** 文件中的"timeout"设置：

```
# ansible managed-node-01.example.com -m ansible.builtin.command -a "grep
'timeout' /boot/grub2/grub.cfg"
managed-node-01.example.com | CHANGED | rc=0 >>
if [ x$feature_timeout_style = xy ] ; then
  set timeout_style=menu
  set timeout=10
# Fallback normal timeout code in case the timeout_style feature is
  set timeout=10
if [ x$feature_timeout_style = xy ] ; then
    set timeout_style=menu
    set timeout=10
    set orig_timeout_style=${timeout_style}
    set orig_timeout=${timeout}
      # timeout_style=menu + timeout=0 avoids the countdown code keypress check
      set timeout_style=menu
      set timeout=10
      set timeout_style=hidden
      set timeout=10
if [ x$feature_timeout_style = xy ]; then
  if [ "${menu_show_once_timeout}" ]; then
    set timeout_style=menu
    set timeout=10
    unset menu_show_once_timeout
    save_env menu_show_once_timeout
```

7.4. 使用 BOOTLOADER RHEL 系统角色收集引导装载程序配置信息

您可以使用 **bootloader** RHEL 系统角色，以自动的方式收集有关 GRUB2 引导装载程序条目的信息。这样，您可以快速确定您的系统是否被设置为正确引导，所有条目是否都指向正确的内核和初始 RAM 磁盘镜像。

因此，您可以，例如：

- 防止引导失败。
- 在故障排除时恢复到已知的良好状态。
- 确保与安全相关的内核命令行参数被正确配置。

先决条件

- 您已准备好控制节点和受管节点
- 以可在受管主机上运行 playbook 的用户登录到控制节点。
- 用于连接到受管节点的帐户具有 **sudo** 权限。

流程

1. 创建一个包含以下内容的 playbook 文件，如 ~/**playbook.yml** ：

```
---
- name: Configuration and management of GRUB2 boot loader
  hosts: managed-node-01.example.com
  tasks:
    - name: Gather information about the boot loader configuration
      ansible.builtin.include_role:
        name: redhat.rhel_system_roles.bootloader
      vars:
        bootloader_gather_facts: true

    - name: Display the collected boot loader configuration information
      debug:
        var: bootloader_facts
```

有关 playbook 中使用的所有变量的详情，请查看控制节点上的 **/usr/share/ansible/roles/rhel-system-roles.bootloader/README.md** 文件。

2. 验证 playbook 语法：

```
$ ansible-playbook --syntax-check ~/playbook.yml
```

请注意，这个命令只验证语法，不会防止错误但有效的配置。

3. 运行 playbook：

```
$ ansible-playbook ~/playbook.yml
```

验证

- 在控制节点上运行之前的 playbook 后，您会看到类似如下所示的命令行输出：

```
...
    "bootloader_facts": [
        {
            "args": "ro crashkernel=1G-4G:256M,4G-64G:320M,64G-:576M rd.lvm.lv=rhel/root
rd.lvm.lv=rhel/swap $tuned_params quiet",
            "default": true,
            "id": "2c9ec787230141a9b087f774955795ab-6.12.el10_0.aarch64",
            "index": "1",
            "initrd": "/boot/initramfs-6.12.0.el10_0.aarch64.img $tuned_initrd",
            "kernel": "/boot/vmlinuz-6.12.0-0.el10_0.aarch64",
            "root": "/dev/mapper/rhel-root",
            "title": "Red Hat Enterprise Linux (6.12.0-0.el10_0.aarch64) 10"
        }
    ]
...
```

命令行输出显示以下有关引导条目的值得注意的配置信息：

args

在引导过程中由 GRUB2 引导装载程序传递给内核的命令行参数。它们配置内核、initramfs 和其他引导时组件的各种设置和行为。

id

在引导装载程序菜单中分配给每个引导条目的唯一标识符。它由机器 ID 和内核版本组成。

root

内核要挂载的根文件系统，并在启动过程中作为主文件系统使用 。

第 8 章 使用内核实时修补程序应用补丁

您可以使用 Red Hat Enterprise Linux 内核实时打补丁解决方案，来在不重启或重启任何进程的情况下对运行的内核打补丁。

使用这个解决方案，系统管理员需要：

- 可以在内核中立即应用重要的安全补丁。
- 不必等待长时间运行的任务完成、关闭或调度停机时间。
- 可以控制系统的正常运行时间，且不会牺牲安全性和稳定性。

通过使用内核实时打补丁，您可以减少安全补丁所需的重启数。但请注意，您无法解决所有关键或重要的 CVE。有关实时打补丁的范围的详情，请查看红帽知识库解决方案 Red Hat Enterprise Linux 中是否支持实时内核补丁(kpatch)？

警告

内核实时补丁和其它内核子组件之间存在一些不兼容。在使用内核实时补丁前，请仔细阅读 kpatch 的限制。

注意

有关内核实时补丁更新支持节奏的详情，请参考：

- 内核实时补丁支持节奏更新
- 内核实时补丁生命周期

8.1. KPATCH 的限制

- 通过使用 **kpatch** 功能，您可以应用简单的不需要立即重启系统的安全更新和 bug 修复更新。
- 在载入补丁过程中或之后，您不能使用 **SystemTap** 或 **kprobe** 工具。在探测被删除后，补丁可能才会生效。

8.2. 对第三方实时补丁的支持

kpatch 实用程序是红帽通过红帽软件仓库提供的 RPM 模块支持的唯一内核实时补丁程序。红帽不支持第三方提供的实时补丁。

有关第三方软件支持策略的更多信息，请参阅 作为客户，在使用第三方组件时红帽如何支持我？

8.3. 获得内核实时补丁

内核模块(**kmod**)实现内核实时打补丁功能，并作为一个 RPM 软件包提供。

为您提供对内核实时补丁的访问，这些补丁通过标准渠道提供。但是，如果您没有订阅延长支持服务，则

当下一个次发行版本可用时，您会失去对当前次发行版本的新补丁的访问权限。例如，在标准订阅中，您可以在 RHEL 10.2 内核发布前实时给 RHEL 10.1 内核打补丁。RHEL 10.2 发布后，RHEL 10.1 的实时补丁不可用。

内核实时补丁的组件如下：

内核补丁模块

- 内核实时补丁的交付机制。
- 为内核构建的一个内核模块打修补。
- 补丁模块包含内核所需修复的代码。
- 补丁模块注册到 **livepatch** 内核子系统，并指定要替换的原始功能，以及指向替换功能的指针。内核补丁模块以 RPM 的形式提供。
- 命名规则为 **kpatch_<kernel version>_<kpatch version>_<kpatch release>**。名称中"kernel version"部分的*点*被 *下划线*替代。

kpatch 工具

用于管理补丁模块的命令行工具。

kpatch 服务

multiuser.target 所需的 **systemd** 服务。这个目标会在引导时载入内核补丁模块。

kpatch-dnf 软件包

以 RPM 软件包的形式提供的 DNF 插件。此插件管理内核实时补丁的自动订阅。

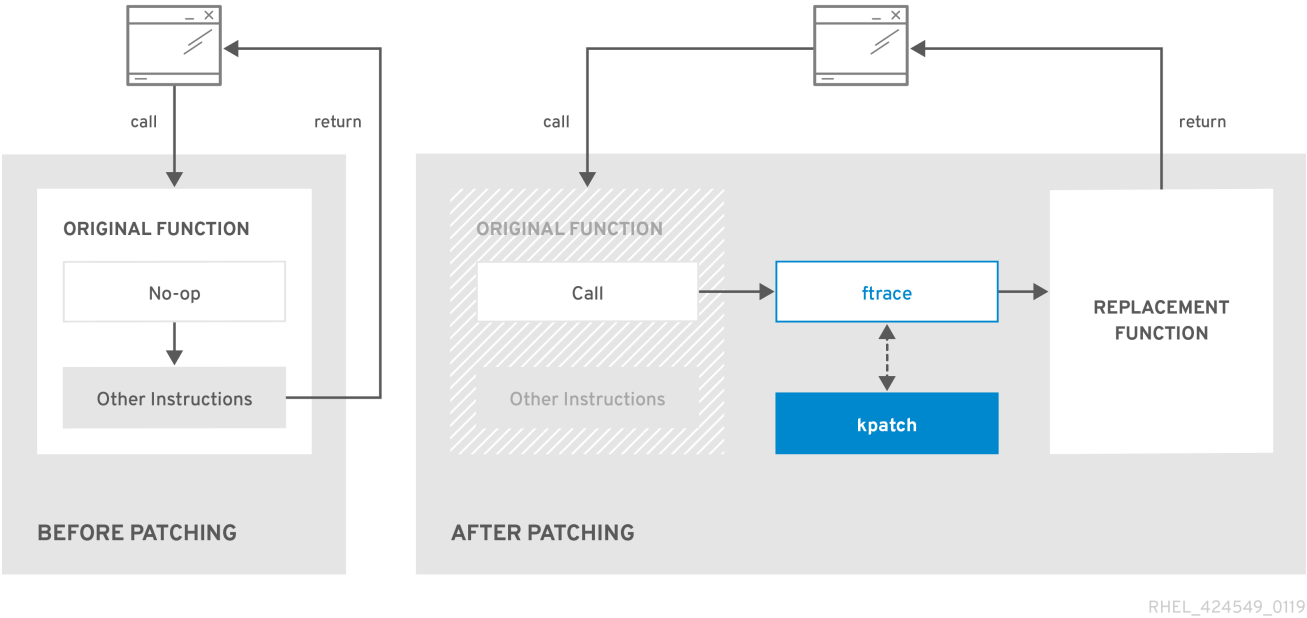
8.4. 实时对内核打补丁的过程

kpatch 内核打补丁解决方案使用 **livepatch** 内核子系统将过时的功能重定向到更新的功能。将实时内核补丁应用到系统会触发以下进程：

1. 内核补丁模块复制到 **/var/lib/kpatch/** 目录中，并在下次引导时由 **systemd** 注册以重新应用到内核。
2. **kpatch** 模块加载到运行的内核中，新的功能使用指向新代码在内存中的位置的指针被注册到 **ftrace** 机制中。

当内核访问打了补丁的功能时，**ftrace** 机制会重定向它，绕过原始功能，并将内核引导到打了补丁的函数版本。

图 8.1. 内核实时补丁如何工作



8.5. 将当前安装的内核订阅到实时补丁流

内核补丁模块在 RPM 软件包中提供，具体取决于被修补的内核版本。每个 RPM 软件包将随着时间不断累积更新。

以下流程解释了如何订阅以后为给定内核的所有累积实时补丁更新。因为实时补丁是累计的，所以您无法选择为一个特定的内核部署哪些单独的补丁。

> **警告**
>
> 红帽不支持任何适用于红帽支持的系统的第三方实时补丁。

先决条件

- 您在系统上具有 root 权限。

流程

1. 可选：检查您的内核版本：

```
# uname -r
6.12.0-55.9.1.el10_0.x86_64
```

2. 搜索与内核版本对应的实时补丁软件包：

```
# dnf search $(uname -r)
```

3. 安装实时补丁（live patching）软件包：

```
# dnf install kpatch
```

这个命令只为特定的内核安装并应用最新累积的实时补丁。

如果实时补丁软件包的版本是 1-1 或更高版本，则软件包将包含补丁模块。在这种情况下，内核会在安装 live patching 软件包期间自动修补。

内核补丁模块也安装到 **/var/lib/kpatch/** 目录中，供 **systemd** 系统和服务管理器以后重启时载入。

注意

当给定内核没有可用的实时补丁时，将安装空的实时补丁软件包。一个空的实时打补丁软件包会有一个 0-0 的 *kpatch_version-kpatch_release*，如 **kpatch-patch-6_12_0-1-0-0.x86_64.rpm**。空 RPM 安装会将系统订阅到以后为给定内核提供的所有实时补丁。

验证

- 验证是否所有安装的内核都已打了补丁：

```
# kpatch list
Loaded patch modules:
kpatch_6_12_0_1_0_1 [enabled]

Installed patch modules:
kpatch_6_12_0_1_0_1 (6.12.0.el10_0.x86_64)
…
```

输出显示内核补丁模块已加载到现在已使用 **kpatch-patch-6_12_0-0.el10_0.x86_64.rpm** 软件包的最新修复打了补丁的内核中。

如需更多信息，请参阅系统中的 **kpatch (1)** 手册页。

注意

输入 **kpatch list** 命令不会返回一个空的实时补丁软件包。使用 **rpm -qa | grep kpatch** 命令替代。

```
# rpm -qa | grep kpatch
kpatch-dnf-0.4-3.el10.noarch
kpatch-0.9.7-2.el10.noarch
kpatch-patch-6_12_0-0.el10_0.x86_64
```

8.6. 自动订阅将来的内核到实时补丁流

您可以使用 **kpatch-dnf** DNF 插件订阅系统，从而修复内核补丁模块（也称为内核实时补丁）提供的修复。该插件为系统当前使用的任何内核启用**自动**订阅，以及**在以后安装**地内核。

先决条件

- 您在系统上具有 root 权限。

流程

1. 可选：检查所有安装的内核和您当前运行的内核：

```
# dnf list installed | grep kernel
Updating Subscription Management repositories.
Installed Packages
...
kernel-core.x86_64          6.12.0-55.9.1.el10          @beaker-BaseOS
kernel-core.x86_64          6.12.0-55.9.1.el10          @@commandline
...

# uname -r
6.12.0-55.9.1.el10_0.x86_64
```

2. 安装 **kpatch-dnf** 插件：

```
# dnf install kpatch-dnf
```

3. 启用自动订阅内核实时补丁：

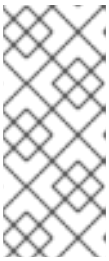
```
# dnf kpatch auto
Updating Subscription Management repositories.
Last metadata expiration check: 1:38:21 ago on Fri 17 Sep 2021 07:29:53 AM EDT.
Dependencies resolved.
==================================================
 Package                      Architecture
==================================================
Installing:
 kpatch-patch-6_12_0-1            x86_64
 kpatch-patch-6_12_0-2            x86_64

Transaction Summary
===================================================
Install  2 Packages
…
```

这个命令订阅所有当前安装的内核，以接收内核实时补丁。命令还会为所有安装的内核安装并应用最新的累积实时补丁（如果有）。

当您更新内核时，会在新的内核安装过程中自动安装实时补丁。

内核补丁模块也安装到 **/var/lib/kpatch/** 目录中，它由 **systemd** 系统和服务管理器以后重启时载入。



注意

当给定内核没有可用的实时补丁时，将安装空的实时补丁软件包。一个空的实时打补丁软件包会有一个 0-0 的 *kpatch_version-kpatch_release*，如 **kpatch-patch-6_12_0-1-0-0.el10.x86_64.rpm**。

空 RPM 安装会将系统订阅到以后为给定内核提供的所有实时补丁。

验证

- 验证是否所有安装的内核都已打了补丁：

```
# kpatch list
Loaded patch modules:
kpatch_6_12_0_2_0_1 [enabled]

Installed patch modules:
kpatch_6_12_0_1_0_1 (6.12.0-0.el10.x86_64)
kpatch_6_12_0_2_0_1 (6.12.0-0.el10.x86_64)
```

输出显示，您正在运行的内核以及其它安装的内核已使用 **kpatch-patch-6_12_0-1-0-1.el10.x86_64.rpm** 和 **kpatch-patch-6_12_0-2-0-1.el10.x86_64.rpm** 软件包中的修复打了补丁。

注意

输入 **kpatch list** 命令不会返回一个空的实时补丁软件包。使用 **rpm -qa | grep kpatch** 命令替代。

```
# rpm -qa | grep kpatch
kpatch-dnf-0.9.7_0.4-4.el10.noarch
kpatch-0.9.7-4.el10.noarch
kpatch-patch-6_12_0_1-0-0.el10_0.x86_64
```

8.7. 禁用实时补丁流的自动订阅

当您向内核补丁模块提供的修复订阅系统时，您的订阅是 **自动的**。您可以禁用此功能，以禁用自动安装 **kpatch-patch** 软件包。

先决条件

- 您在系统上具有 root 权限。

流程

1. 可选：检查所有安装的内核和您当前运行的内核：

```
# dnf list installed | grep kernel
Updating Subscription Management repositories.
Installed Packages
...
kernel-core.x86_64          6.12.0-0.el10          @beaker-BaseOS
kernel-core.x86_64          6.12.0-0.el10          @@commandline
...

# uname -r
6.12.0-0.el10_0.x86_64
```

2. 禁用向内核实时补丁的自动订阅：

```
# dnf kpatch manual
Updating Subscription Management repositories.
```

如需更多信息，请参阅 **kpatch (1)** 和 **dnf-kpatch (8)** 手册页。

验证

- 您可以检查成功的结果：

```
# yum kpatch status
...
Updating Subscription Management repositories.
Last metadata expiration check: 0:30:41 ago on Tue Jun 14 15:59:26 2022.
Kpatch update setting: manual
```

8.8. 更新内核补丁模块

内核补丁模块通过 RPM 软件包提供并应用。更新累积的内核补丁模块的过程与更新任何其他 RPM 软件包类似。

先决条件

- 系统已订阅实时补丁流，如将当前安装的内核订阅到实时补丁流中所述。

流程

- 更新至当前内核的新累计版本：

```
# dnf update kpatch
```

  这个命令会自动安装并应用当前运行的内核可用的更新。包括将来发布的所有实时补丁。

- 另外，更新所有安装的内核补丁模块：

```
# dnf update kpatch
```

注意

当系统重启到同一内核时，**kpatch.service** systemd 服务会再次对内核进行补丁。

其他资源

- 更新软件包

8.9. 删除 LIVE PATCHING 软件包

通过删除实时补丁软件包来禁用 Red Hat Enterprise Linux 内核实时补丁解决方案。

先决条件

- 您在系统上具有 root 权限。
- 已安装 live patching 软件包。

流程

1. 选择 live patching 软件包：

```
# dnf list installed | grep kpatch-patch
kpatch-patch-6.12.0-0.el10_0.x86_64        0-0.el10        @@commandline
…
```

示例输出列出了您安装的实时补丁软件包。

2. 删除 live patching 软件包：

```
# dnf remove kpatch-patch-6.12.0-0.el10_0.x86_64
```

删除实时补丁软件包后，内核将保持补丁，直到下次重启为止，但内核补丁模块会从磁盘中删除。将来重启时，对应的内核将不再被修补。

3. 重启您的系统。

4. 验证实时打补丁软件包是否已删除：

```
# dnf list installed | grep kpatch-patch
```

如果软件包已被成功删除，命令不会显示任何输出。

验证

1. 验证内核实时打补丁解决方案是否已禁用：

```
# kpatch list
Loaded patch modules:
```

示例输出显示内核没有补丁，实时补丁解决方案没有激活，因为目前没有加载补丁模块。

重要

目前，红帽不支持在不重启系统的情况下还原实时补丁。如有任何问题，请联系我们的支持团队。

其它资源

- 删除安装的软件包

8.10. 卸载内核补丁模块

防止 Red Hat Enterprise Linux 内核实时补丁解决方案在以后的引导时应用内核补丁模块。

先决条件

- 您在系统上具有 root 权限。
- 已安装实时补丁软件包。
- 已安装并载入内核补丁模块。

流程

1. 选择内核补丁模块：

```
# kpatch list
Loaded patch modules:
kpatch_6_12_0_1_0_1 [enabled]

Installed patch modules:
kpatch_6_12_0_1_0_1 (6.12.0.el10_0.x86_64)
…
```

2. 卸载所选的内核补丁模块。

```
# kpatch uninstall kpatch_6_12_0_1_0_1
uninstalling kpatch_6_12_0_1_0_1 (6.12.0.el10_0.x86_64)
```

   - 请注意，卸载的内核补丁模块仍然被加载：

```
# kpatch list
Loaded patch modules:
kpatch_6_12_0_1_0_1 [enabled]

Installed patch modules:
<NO_RESULT>
```

   卸载所选模块后，内核将保持补丁，直到下次重启为止，但已从磁盘中删除内核补丁模块。

3. 重启您的系统。

验证

1. 验证内核补丁模块是否已卸载：

```
# kpatch list
Loaded patch modules:
…
```

   这个示例输出显示没有加载或安装的内核补丁模块，因此内核没有被打补丁，且内核实时打补丁解决方案没有激活。

8.11. 禁用 KPATCH.SERVICE

防止 Red Hat Enterprise Linux 内核实时补丁解决方案在以后的引导中全局应用所有内核补丁模块。

先决条件

- 您在系统上具有 root 权限。
- 已安装实时补丁软件包。
- 已安装并载入内核补丁模块。

流程

1. 验证 **kpatch.service** 是否已启用。

```
# systemctl is-enabled kpatch.service
enabled
```

2. 禁用 **kpatch.service** ：

```
# systemctl disable kpatch.service
Removed /etc/systemd/system/multi-user.target.wants/kpatch.service.
```

- 请注意，应用的内核补丁模块仍然被载入：

```
# kpatch list
Loaded patch modules:
kpatch_6_12_0_1_0_1 [enabled]

Installed patch modules:
kpatch_6_12_0_1_0_1 (6.12.0.el10_0.x86_64)
```

3. 重启您的系统。

4. 可选：验证 **kpatch.service** 的状态。

```
# systemctl status kpatch.service
● kpatch.service - "Apply kpatch kernel patches"
   Loaded: loaded (/usr/lib/systemd/system/kpatch.service; disabled; vendor preset: disabled)
   Active: inactive (dead)
```

示例输出测试 **kpatch.service** 是否已禁用。因此，内核实时修补解决方案不活跃。

5. 验证内核补丁模块是否已卸载。

```
# kpatch list
Loaded patch modules:

Installed patch modules:
kpatch_6_12_0_1_0_1 (6.12.0.el10_0.x86_64)
```

示例输出显示内核补丁模块已安装，但内核没有打补丁。

重要

目前，红帽不支持在不重启系统的情况下将实时补丁恢复原样。如有任何问题，请联系我们的支持团队。

其他资源

- 管理 systemd

第 9 章 在虚拟环境中保留内核 PANIC 参数

在 Red Hat Enterprise Linux 10 中配置虚拟机时，不得启用 **softlockup_panic** 和 **nmi_watchdog** 内核参数。这导致虚拟机可能会遭受应该不需要内核 panic 的软锁定。

本建议背后的原因如下。

9.1. 什么是软锁定

当任务在不重新调度的情况下在 CPU 上的内核空间中执行时，软锁定通常是由程序错误造成的。该任务也不允许任何其他任务在特定 CPU 上执行。因此，用户通过系统控制台会显示警告信息。这个问题也被称为软锁定触发。

其他资源

- 什么是 CPU 软锁定？

9.2. 控制内核 PANIC 的参数

可设置以下内核参数来控制当检测到软锁定时的系统行为。

softlockup_panic

控制当检测到软锁定时内核是否 panic。

类型	值	效果
整数	0	内核在软锁定时不 panic
整数	1	软锁定中的内核 panics

默认情况下，在 RHEL 10 上这个值为 0。

系统需要首先检测硬锁定才能 panic。检测由 **nmi_watchdog** 参数控制。

nmi_watchdog

控制锁定检测机制 (**watchdogs**) 是否处于活动状态。这个参数是整数类型。

值	效果
0	禁用锁定检测器
1	启用锁定检测器

硬锁定检测器会监控每个 CPU 是否有响应中断的能力。

watchdog_thresh

控制 watchdog **hrtimer** 的频率、NMI 事件和软或硬锁定阈值。

默认阈值	软锁定阈值
10 秒	2 * **watchdog_thresh**

将此参数设置为 0 可禁用锁定检测。

其他资源

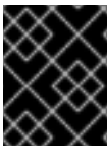
- 软锁定检测器和硬锁定检测器
- 内核 sysctl

9.3. 在虚拟环境中有伪装的软锁定

在物理主机上触发的 软锁定 通常代表内核或硬件 bug 。在虚拟环境中的客户机操作系统上发生的同样的现象可能代表假的警告。

主机上的重工作负载或对某些特定资源（如内存）的竞争可能导致错误的软锁定的触发，因为主机可能会调度客户端 CPU 的时间超过 20 秒。当客户机 CPU 再次被调度到在主机上运行时，它会经历一个触发 到期计时器的 *时间跳跃*。计时器还包括 **hrtimer** watchdog，其可报告客户机 CPU 上的软锁定。

虚拟化环境中的软锁定可能是假的。当报告对客户机 CPU 的一个软锁定时，您不必启用触发系统 panic 的内核参数。



重要

若要了解客户机中的软锁定，必须了解，主机会作为一个任务调度客户机，客户机然后会调度自己的任务。

其他资源

- 虚拟机组件及其交互

第 10 章 为数据库服务器调整内核参数

为确保数据库服务器和数据库的有效操作，您必须配置所需的内核参数集。

10.1. 介绍

数据库服务器是一种提供数据库管理系统 (DBMS) 功能的服务。DBMS 提供数据库管理的实用程序，并与最终用户、应用程序和数据库交互。

Red Hat Enterprise Linux 10 提供以下数据库管理系统：

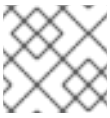
- **MariaDB 10.11**
- **MySQL 8.4**
- **PostgreSQL 16**
- **Valkey 7.2**

10.2. 影响数据库应用程序性能的参数

以下内核参数会影响数据库应用程序的性能。

fs.aio-max-nr

定义系统可在服务器中处理的异步 I/O 操作的最大数目。



注意

增加 **fs.aio-max-nr** 参数不会在增加 aio 限制外产生任何变化。

fs.file-max

定义系统在任何实例上支持的最大文件句柄数（临时文件名或者分配给打开文件的 ID）。内核会在应用程序请求文件句柄时动态分配文件。但是，当这些文件被应用程序释放时，内核不会释放这些文件句柄。而是回收这些文件句柄。分配的文件句柄的总数将随时间而增加，即使当前使用的文件句柄的数可能较少。

kernel.shmall

定义可用于系统范围的共享内存页面总数。要使用整个主内存，**kernel.shmall** 参数的值应当为主内存大小总计。

kernel.shmmax

定义 Linux 进程在其虚拟地址空间中可分配的单个共享内存段的最大字节大小。

kernel.shmmni

定义数据库服务器能够处理的共享内存段的最大数量。

net.ipv4.ip_local_port_range

系统将此端口范围用于连接到数据库服务器的程序，而无需指定端口号。

net.core.rmem_default

通过传输控制协议 (TCP) 定义默认接收套接字内存。

net.core.rmem_max

通过传输控制协议 (TCP) 定义最大接收套接字内存。

net.core.wmem_default

通过传输控制协议 (TCP) 定义默认发送套接字内存。

net.core.wmem_max

通过传输控制协议 (TCP) 定义最大发送套接字内存。

vm.dirty_bytes / vm.dirty_ratio

定义以脏内存百分比为单位的字节/阈值，在该阈值中，生成脏数据的进程会在 **write()** 函数中启动。

注意

一个 vm.dirty_bytes 或 vm.dirty_ratio 可以在同一时间被指定。

vm.dirty_background_bytes / vm.dirty_background_ratio

定义以脏内存百分比为单位的字节/阈值，达到此阈值时内核会尝试主动将脏数据写入硬盘。

注意

一个 vm.dirty_background_bytes 或 vm.dirty_background_ratio 可以一次指定。

vm.dirty_writeback_centisecs

定义负责将脏数据写入硬盘的内核线程定期唤醒之间的时间间隔。
这个内核参数以 100 分之一秒为单位。

vm.dirty_expire_centisecs

定义将变旧的脏数据写入硬盘的时间。
这个内核参数以 100 分之一秒为单位。

其他资源

- 脏页缓存回写和 vm.dirty 参数

第 11 章 配置巨页

物理内存以固定大小的块的形式进行管理，称为页。在 x86_64 架构中，由 Red Hat Enterprise Linux 支持，默认的内存页大小为 **4 KB**。这个默认页面大小适用于通用的操作系统，如支持许多工作负载的 Red Hat Enterprise Linux。

但是，在某些情况下，特定应用程序可能会从使用更大的页面中受益。例如，在使用 **4 KB** 页面时，如果应用程序需要处理大量的、相对固定的数据集合时（有几百兆字节甚至数千兆字节数据）可能会遇到性能问题。这种数据集可能需要大量的 **4 KB** 页面，这可以增加操作系统和 CPU 的资源使用量。

本节提供有关 RHEL 10 中提供的巨页的信息，以及如何配置它们。

11.1. 可用的巨页功能

使用 Red Hat Enterprise Linux，您可以将巨页用于处理大数据集的应用程序，并改进此类应用程序的性能。

以下是 RHEL 支持的巨页方法：

HugeTLB 页

HugeTLB 页面也称为静态巨页。有两种方法可以保留 HugeTLB 页面：

- **在引导时** ：这会增加成功的可能性，因为内存还没有很大的碎片。但是，在 NUMA 机器上，页面数量会自动在不同的 NUMA 节点间进行分隔。
  有关在引导时影响 HugeTLB 页行为的参数的更多信息，请参阅 在引导时保留 HugeTLB 页的参数，以及如何使用这些参数在引导时配置 HugeTLB 页，请参见 在引导时配置 HugeTLB 页。

- **在运行时** ：它允许您为每个 NUMA 节点保留巨页。如果在引导过程中以早期方式进行运行时保留，则可能会降低内存碎片的可能性。

有关在运行时影响 HugeTLB 页行为的参数的更多信息，请参阅 在运行时保留 HugeTLB 页的参数，以及如何使用这些参数在运行时配置 HugeTLB 页，请参阅 在运行时配置 HugeTLB 页。

透明巨页 (THP)

使用 THP 时，内核会自动将巨页分配给进程，因此不需要手动保留静态巨页。以下是 THP 中的两种操作模式：

- **系统范围** ：内核会在可以分配巨页时尝试为进程分配巨页，进程会使用一个大型连续的虚拟内存区域。

- **针对每个进程** ：内核仅将巨页分配给单个进程的内存区域，您可以使用 **madvise** () 系统调用来指定。

注意

THP 功能只支持 **2 MB** 页面。

有关在引导时影响 HugeTLB 页行为的参数的更多信息，请参阅 管理透明巨页。

11.2. 在引导时保留 HUGETLB 页面的参数

使用以下参数来在引导时影响 HugeTLB 页面行为。

有关如何在引导时使用这些参数配置 HugeTLB 页的更多信息，请参阅 在引导时配置 HugeTLB。

表 11.1. 用于在引导时配置 HugeTLB 页面的参数

参数	描述	默认值
hugepages	定义在引导时在内核中配置的持久性巨页数量。 在 NUMA 系统中，定义了这个参数的巨页在节点间平均划分。 您可以通过更改 **/sys/devices/system/node/node_id/hugepages/hugepages-size/nr_hugepages** 文件中的节点值，在运行时为特定节点分配巨页。	默认值为 **0**。 要在引导时更新这个值，在 **/proc/sys/vm/nr_hugepages** 文件中更改这个参数的值。
hugepagesz	定义在引导时在内核中配置的持久性巨页的大小。	有效值为 **2 MB** 和 **1 GB**。默认值为 **2 MB**。
default_hugepagesz	定义在引导时在内核中配置的持久性巨页的默认大小。	有效值为 **2 MB** 和 **1 GB**。默认值为 **2 MB**。

11.3. 在引导时配置 HUGETLB

HugeTLB 通过在引导时保留巨页来启用使用巨页，从而尽量减少内存碎片并确保有足够的资源可用于较大的内存页面。

11.3.1. 使用内核命令行参数配置 HugeTLB

您可以使用内核命令行参数在引导过程的最早阶段保留 Huge Translation Lookaside Buffer (HugeTLB)页面。这个方法提供了在内核启动过程中成功保留所需数量的巨页和大小的最机，因为内存是在内核启动期间分配的。

首选使用内核引导参数保留 HugeTLB 页面，因为此方法可确保与使用 systemd 单元相比，分配更大的连续内存区域。

注意

该流程中的示例演示了如何使用命令行选项配置 HugeTLB 页面。这些示例不一定适用于您的系统配置。在在您的环境中应用这些设置前，请查看您的系统要求和目标。

先决条件

- 在您的系统中必须具有 root 权限。

流程

- 更新内核命令行使其包含 HugeLTB 选项。
  - 要为您的架构保留默认大小的 HugeTLB 页面，请输入：

    ```
    # grubby --update-kernel=ALL --args="hugepages=10"
    ```

    这个命令保留默认池大小的 10 HugeTLB 页面。例如，在 **x86_64** 上，默认的池大小为 **2 MB**。在具有非统一内存访问(NUMA)的系统上，分配在 NUMA 节点上均匀分布。如果系统有两个 NUMA 节点，每个节点保留五个页面。

  - 要保留不同的 HugeTLB 页面大小，在内核命令行中指定 **hugepagesz** 和 **hugepages** 选项，请输入：

    ```
    # grubby --update-kernel=ALL --args="hugepagesz=2M hugepages=10 hugepagesz=1G
    hugepages=1"
    ```

    该命令保留 10 页（共 **2 MB** ）和 1 页(**1 GB**)。

    系统在引导时保留 HugeTLB 页面的指定数量和大小，确保在操作系统开始正常操作前分配内存。

    重要

    选项的顺序非常重要。每个 **hugepagesz=** 选项必须紧接其对应的 **hugepages=** 选项。

11.3.2. 使用 systemd 服务单元配置 HugeTLB

您可以使用 systemd 服务单元在用户空间引导过程中配置 Huge Translation Lookaside Buffer (HugeTLB)页面。这个方法在内核初始化后保留大型内存页面，但在启动大多数用户空间服务前。虽然这种方法不是作为内核命令行配置早期的，但可以确保应用程序在系统操作期间可以访问所需的巨页。

先决条件

- 在您的系统中必须具有 root 权限。

流程

1. 在 **/usr/lib/systemd/system/** 目录中创建一个名为 **hugetlb-gigantic-pages.service** 的新文件，并添加以下内容：

```
[Unit]
Description=HugeTLB Gigantic Pages Reservation
DefaultDependencies=no
Before=dev-hugepages.mount
ConditionPathExists=/sys/devices/system/node
ConditionKernelCommandLine=hugepagesz=1G

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/lib/systemd/hugetlb-reserve-pages.sh
```

```
[Install]
WantedBy=sysinit.target
```

2. 在 **/usr/lib/systemd/** 目录中创建一个名为 **hugetlb-reserve-pages.sh** 的新文件，并添加以下内容：

```
#!/bin/sh

nodes_path=/sys/devices/system/node/
if [ ! -d $nodes_path ]; then
    echo "ERROR: $nodes_path does not exist"
    exit 1
fi

reserve_pages()
{
    echo $1 > $nodes_path/$2/hugepages/hugepages-1048576kB/nr_hugepages
}

reserve_pages <number_of_pages> <node>
```

将 ***<number_of_pages***> 替换为您要保留的 1GB 页面数，将 ***&*** lt;node> 替换为您要保留的节点名称。例如：要在 **node0** 上保留两个 1 GB 页面，在 **node1** 上保留 1GB 页面，将 &lt ***;number_of_pages&*** gt；替换为 **node0**，**1** 代表 **node1**。

3. 创建可执行脚本：

```
# chmod +x /usr/lib/systemd/hugetlb-reserve-pages.sh
```

4. 启用早期引导保留：

```
# systemctl enable hugetlb-gigantic-pages.service
```

注意

- 您可以在运行时尝试保留更多 1 GB 页面，方法是写入 **nr_hugepages** 属性。但是，为了避免因为内存碎片导致失败，在引导过程中在早期保留 1 GB 页面。
- 保留静态巨页可以有效地减少系统可用的内存量，并阻止它使用内存容量。虽然正确大小的保留巨页池对使用它的应用程序有用，但保留巨页的过度或未使用的池最终会降低整体系统性能。当设置保留的巨页池时，请确定系统可以正确地使用其完整的内存容量。

11.4. 在运行时保留 HUGETLB 页面的参数

在运行时使用以下参数来影响 HugeTLB 页面的行为。

有关如何在运行时使用这些参数配置 HugeTLB 页的更多信息，请参阅 在运行时配置 HugeTLB。

表 11.2. 在运行时配置 HugeTLB 页面的参数

参数	描述	文件名
nr_hugepages	定义分配给指定 NUMA 节点的指定大小的巨页数量。	**/sys/devices/system/node/node_id/hugepages/hugepages-size/nr_hugepages**
nr_overcommit_hugepages	定义系统通过过量使用内存来创建和使用的最大额外巨页数。 在此文件中写入任何非零值表示，如果持久的巨页池耗尽，系统会从内核的正常页面池中获取巨页数量。随着这些多余的巨页变得未使用，它们会被释放并返回到内核的正常页面池。	**/proc/sys/vm/nr_overcommit_hugepages**

11.5. 在运行时配置 HUGETLB

这个步骤描述了如何在 *node2* 中添加 *20* 个 *2048 kB* 巨页。

要根据您的要求保留页面，请替换：

- *20*，使用您要保留的巨页数，
- *2048kB*，使用巨页的大小，
- *node2*，使用您要保留页面的节点。

流程

1. 显示内存统计信息：

```
# numastat -cm | egrep 'Node|Huge'
                 Node 0 Node 1 Node 2 Node 3  Total add
AnonHugePages         0      2      0      8     10
HugePages_Total       0      0      0      0      0
HugePages_Free        0      0      0      0      0
HugePages_Surp        0      0      0      0      0
```

2. 将指定大小的巨页数量添加到节点：

```
# echo 20 > /sys/devices/system/node/node2/hugepages/hugepages-2048kB/nr_hugepages
```

验证

- 确保添加了巨页数量：

```
# numastat -cm | egrep 'Node|Huge'
                 Node 0 Node 1 Node 2 Node 3  Total
AnonHugePages         0      2      0      8     10
```

```
HugePages_Total      0     0    40     0    40
HugePages_Free       0     0    40     0    40
HugePages_Surp       0     0     0     0     0
```

如需更多信息，请参阅 **numastat (8)** 手册页。

11.6. 管理透明巨页

在 Red Hat Enterprise Linux 10 中默认启用透明巨页(THP)。但是，您可以启用、禁用透明巨页，或使用运行时配置、TuneD 配置文件、内核命令行参数或 systemd 单元文件将透明巨页设置为 **madvise**。

11.6.1. 使用运行时配置管理透明巨页

可在运行时管理透明巨页(THP)，以优化内存用率。系统重启后，运行时配置不是持久的。

流程

1. 检查 THP 的状态：

```
$ cat /sys/kernel/mm/transparent_hugepage/enabled
```

2. 配置 THP.

- 启用 THP：

```
$ echo always > /sys/kernel/mm/transparent_hugepage/enabled
```

- 禁用 THP：

```
$ echo never > /sys/kernel/mm/transparent_hugepage/enabled
```

- 将 THP 设置为 **madvise** ：

```
$ echo madvise > /sys/kernel/mm/transparent_hugepage/enabled
```

要防止应用程序分配过多的内存资源，请禁用系统范围的透明巨页，并只为通过 **madvise** 系统调用明确请求它的应用程序启用它们。

注意

有时，为短期分配提供低延迟的优先级比立即实现长时间分配的性能要高。在这种情况下，您可以在启用 THP 时禁用直接压缩。

直接压缩是在巨页分配过程中同步的内存压缩。禁用直接压缩功能无法保证保存内存，但可能会降低频繁页面错误期间延迟更高的风险。另外，禁用直接压缩只允许 **madvise** 中突出显示的虚拟内存区域(VMAs)的异步压缩。请注意，如果工作负载从 THP 有很大的好处，则性能会降低。禁用直接压缩：

```
$ echo never > /sys/kernel/mm/transparent_hugepage/defrag
```

如需更多信息，请参阅系统中的 **madvise (2)** 手册页。

11.6.2. 使用 TuneD 配置文件管理透明巨页

您可以使用 TuneD 配置文件管理透明巨页(THP)。**tuned.conf** 文件提供 TuneD 配置文件的配置。这个配置在系统重启后保持不变。

先决条件

- **TuneD** 软件包已安装。
- **TuneD** 服务已启用。

流程

1. 将活跃配置文件复制到同一目录中：

```
$ sudo cp -R /usr/lib/tuned/my_profile /usr/lib/tuned/my_copied_profile
```

2. 编辑 **tune.conf** 文件：

```
$ sudo vi /usr/lib/tuned/my_copied_profile/tuned.conf
```

   - 要启用 THP，请添加行：

```
[bootloader]

cmdline = transparent_hugepage=always
```

   - 要禁用 THP，请添加行：

```
[bootloader]

cmdline = transparent_hugepage=never
```

   - 要将 THP 设置为 **madvise**，请添加行：

```
[bootloader]

cmdline = transparent_hugepage=madvise
```

3. 重启 **TuneD** 服务：

```
$ sudo systemctl restart tuned
```

4. 将新配置文件设置为活跃：

```
$ sudo tuned-adm profile my_copied_profile
```

验证

1. 验证新配置文件是否处于活跃状态：

```
$ sudo tuned-adm active
```

2. 验证是否设置了所需的 THP 模式：

```
$ cat /sys/kernel/mm/transparent_hugepage/enabled
```

11.6.3. 使用内核命令行参数管理透明巨页

您可以通过修改内核参数，来在引导时管理透明巨页(THP)。这个配置在系统重启后保持不变。

先决条件

- 您在系统上具有 root 权限。

流程

1. 获取当前的内核命令行参数：

```
# grubby --info=$(grubby --default-kernel)
kernel="/boot/vmlinuz-6.12.X-XXX.XX.X.el10_0.x86_64"
args="ro crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume=UUID=XXXXXXXX-
XXXX-XXXX-XXXX-XXXXXXXXXXXX console=tty0 console=ttyS0"
root="UUID=XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX"
initrd="/boot/initramfs-6.12.X-XXX.XX.X.el10_0.x86_64.img"
title="Red Hat Enterprise Linux (6.12.X-XXX.XX.X.el10_0.x86_64) 10.0"
id="XXXXXXXXXXXXXXXXXXXXXXXXXXXXXX-6.12.X-XXX.XX.X.el10_0.x86_64"
```

2. 通过添加内核参数来配置 THP。
   - 要启用 THP：

     ```
     # grubby --args="transparent_hugepage=always" --update-kernel=DEFAULT
     ```

   - 要禁用 THP：

     ```
     # grubby --args="transparent_hugepage=never" --update-kernel=DEFAULT
     ```

   - 要将 THP 设置为 **madvise** ：

     ```
     # grubby --args="transparent_hugepage=madvise" --update-kernel=DEFAULT
     ```

3. 重启系统以使更改生效：

```
# reboot
```

验证

- 要验证 THP 的状态，请查看以下文件：

```
# cat /sys/kernel/mm/transparent_hugepage/enabled
always madvise [never]
```

```
# grep AnonHugePages: /proc/meminfo
AnonHugePages:         0 kB
```

```
# grep nr_anon_transparent_hugepages /proc/vmstat
nr_anon_transparent_hugepages 0
```

11.6.4. 使用 systemd 单元文件管理透明巨页

您可以使用 systemd 单元文件，在系统启动时管理透明巨页(THP)。通过创建一个 systemd 服务，您可以在系统重启后获得一致的 THP 配置。

先决条件

- 您在系统上具有 root 权限。

流程

1. 创建新的 systemd 服务文件，以启用、禁用 THP ，并将 THP 设置为 **madvise**。例如：**/etc/systemd/system/disable-thp.service**。

2. 通过向新的 systemd 服务文件中添加以下内容来配置 THP。

   - 要启用 THP，请将以下内容添加到 **<new_thp_file>.service** 文件中：

     ```
     [Unit]
     Description=Enable Transparent Hugepages
     After=local-fs.target
     Before=sysinit.target

     [Service]
     Type=oneshot
     RemainAfterExit=yes
     ExecStart=/bin/sh -c 'echo always > /sys/kernel/mm/transparent_hugepage/enabled

     [Install]
     WantedBy=multi-user.target
     ```

   - 要禁用 THP，请将以下内容添加到 **<new_thp_file>.service** 文件中：

     ```
     [Unit]
     Description=Disable Transparent Hugepages
     After=local-fs.target
     Before=sysinit.target

     [Service]
     Type=oneshot
     RemainAfterExit=yes
     ExecStart=/bin/sh -c 'echo never > /sys/kernel/mm/transparent_hugepage/enabled

     [Install]
     WantedBy=multi-user.target
     ```

   - 要将 THP 设置为 **madvise**，请将以下内容添加到 **<new_thp_file>.service** 文件中：

     ```
     [Unit]
     Description=Madvise Transparent Hugepages
     After=local-fs.target
     ```

```
Before=sysinit.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/bin/sh -c 'echo madvise > /sys/kernel/mm/transparent_hugepage/enabled

[Install]
WantedBy=multi-user.target
```

3. 启用并启动服务：

```
# systemctl enable <new_thp_file>.service
```

```
# systemctl start <new_thp_file>.service
```

验证

- 要验证 THP 的状态，请查看以下文件：

```
$ cat /sys/kernel/mm/transparent_hugepage/enabled
```

11.7. 页面大小对转换后备缓冲区大小的影响

从页表中读取地址映射非常耗时且需要消耗大量资源，因此 CPU 会对最近使用的地址进行缓存，这称为 Translation Lookaside Buffer (TLB) 。但是，默认的 TLB 只能缓存特定数量的地址映射。

如果请求的地址映射不在 TLB 中，称为 TLB miss（TLB未命中），系统仍然需要读取页表以确定到虚拟地址映射的物理。由于应用程序内存要求和用于缓存地址映射的页面大小的关系，具有大内存要求的应用程序可能会比内存要求小的应用程序造成性能下降。因此，务必要尽可能避免 TLB 丢失。

HugeTLB 和 Transparent Huge Page 功能可让应用程序使用大于 **4 KB** 的页面。这允许存储在 TLB 中的地址引用更多内存，这可以降低 TLB 未命中的情况并改进应用程序性能。

11.8. 其他资源

- RHEL 提供的调优配置文件
- TuneD 插件

第 12 章 内核日志记录入门

日志文件提供有关系统的消息，包括内核、服务及其上运行的应用程序。**syslog** 服务为登录 Red Hat Enterprise Linux 提供了原生支持。各种实用程序使用此系统记录事件并将其整理到日志文件中。这些文件在审计操作系统和故障排除问题时很有用。

12.1. 什么是内核环缓冲

在引导过程中，控制台提供有关系统启动的初始阶段的重要信息。为避免丢失早期的消息，内核使用环缓冲。此缓冲区会保存由内核代码中的 **printk()** 函数所产生的所有消息（包括引导消息）。来自内核环缓冲的消息随后由 **syslog** 服务读取并存储在永久存储上的日志文件中。

环缓冲是一个具有固定大小的循环数据结构，并硬编码到内核中。用户可以通过 **dmesg** 命令或 **/var/log/boot.log** 文件显示存储在内核环缓冲中的数据。当环形缓冲区满时，新数据将覆盖旧数据。

如需更多信息，请参阅系统中的 **syslog (2)** 和 **dmesg (1)** 手册页。

12.2. 日志级别和内核日志记录上的 PRINTK 角色

内核报告的每条消息都有一个与它关联的日志级别，用于定义消息的重要性。如什么是内核环缓冲区中所述，内核环缓冲会收集所有日志级别的内核消息。**kernel.printk** 参数用于定义缓冲区中哪些消息打印到控制台中。

日志级别值按以下顺序划分：

0
: 内核紧急情况。系统不可用。

1
: 内核警报。必须立即采取行动。

2
: 内核情况被视为是关键的。

3
: 常见内核错误情况。

4
: 常见内核警告情况。

5
: 内核注意到一个正常但严重的情况。

6
: 内核信息性消息.

7
: 内核调试级别消息。

默认情况下，RHEL 10 中的 **kernel.printk** 有以下值：

```
# sysctl kernel.printk
kernel.printk = 7 4 1 7
```

四个值按顺序定义了以下情况：

1. 控制台日志级别，定义打印到控制台的消息的最低优先级。
2. 消息没有显式附加日志级别的默认日志级别。
3. 为控制台日志级别设置最低的日志级别配置。
4. 在引导时为控制台日志级别设置默认值。
   这些值的每一个都定义不同的处理错误消息的规则。

重要

默认的 **7 4 1 7 printk** 值可以更好地调试内核活动。但是，当与串行控制台耦合时，这个 **printk** 设置可能会导致大量 I/O 突发，这可能会导致 RHEL 系统变得暂时不响应。为避免这种情况，把 **printk** 值设置为 **4 4 1 7** 通常可以正常工作，但其代价是丢失额外的调试信息。

另请注意，某些内核命令行参数（如 **quiet** 或 **debug**）会更改默认的 **kernel.printk** 值。

如需更多信息，请参阅系统中的 **syslog (2)** 手册页。

第 13 章 重新安装 GRUB

您可以重新安装 GRUB 引导装载程序来修复某些问题，通常是由于 GRUB 的安装不正确、缺少文件或损坏的系统造成的。您可以通过恢复缺少的文件并更新引导信息来解决这些问题。

重新安装 GRUB 的原因：

- 升级 GRUB 引导装载程序软件包。
- 将引导信息添加到另一个驱动器中。
- 用户需要 GRUB 引导装载程序来控制安装的操作系统。但是，一些操作系统是使用自己的引导装载程序安装的，重新安装 GRUB 会将控制权返回给这些操作系统。

注意

只有在文件没有损坏时，GRUB 才能恢复这些文件。

13.1. 在基于 BIOS 的机器上重新安装 GRUB

您可以在基于 BIOS 的系统上重新安装 GRUB 引导装载程序。在更新 GRUB 软件包后，总是重新安装 GRUB。

重要

这会覆盖现有的 GRUB ，以安装新的 GRUB。确定系统在安装过程中不会导致数据损坏或引导崩溃。

流程

1. 安装 **grub2-pc**:

```
# grub2-pc
```

2. 在安装了它的设备上重新安装 GRUB。例如：如果 **sda** 是您的设备：

```
# grub2-install /dev/sda
```

3. 重启您的系统以使更改生效：

```
# reboot
```

如需更多信息，请参阅系统中的 **grub2-pc (8)** 手册页。

13.2. 在基于 UEFI 的机器上重新安装 GRUB

您可以在基于 UEFI 的系统上重新安装 GRUB 引导装载程序。

重要

确定系统在安装过程中不会导致数据损坏或引导崩溃。

流程

1. 重新安装 **grub2-efi** 和 **shim** 引导装载程序文件：

```
# dnf reinstall grub2-efi-x64 shim
```

2. 重启您的系统以使更改生效：

```
# reboot
```

13.3. 在 IBM POWER 机器上重新安装 GRUB

您可以在 IBM Power 系统的 Power PC Reference Platform (PReP)引导分区上重新安装 GRUB 引导装载程序。在更新 GRUB 软件包后，总是重新安装 GRUB。

重要

这会覆盖现有的 GRUB，以安装新的 GRUB。确定系统在安装过程中不会导致数据损坏或引导崩溃。

流程

1. 列出存储 GRUB 的磁盘分区：

```
# bootlist -m normal -o
sda1
```

2. 在磁盘分区上重新安装 GRUB：

```
# grub2-install partition
```

使用标识的 GRUB 分区（如 **/dev/sda1**） 替换 ***partition***。

3. 重启您的系统以使更改生效：

```
# reboot
```

详情请查看您系统中的 **grub-install (1)** 手册页。

13.4. 重置 GRUB

重置 GRUB 会完全删除所有 GRUB 配置文件和系统设置，并重新安装引导装载程序。您可以将所有配置设置重置回其默认值，因此可以修复由文件损坏和无效配置导致的故障。

重要

以下流程将删除用户所做的所有自定义。

流程

1. 删除配置文件：

```
# rm /etc/grub.d/*
# rm /etc/sysconfig/grub
```

2. 重新安装软件包。
   - 在基于 BIOS 的机器上：

     ```
     # dnf reinstall grub2-pc grub2-tools
     ```

   - 在基于 UEFI 的机器上：

     ```
     # dnf reinstall grub2-efi shim grub2-tools grub2-common
     ```

3. 重建 **grub.cfg** 文件，以使更改生效：

   ```
   # grub2-mkconfig -o /boot/grub2/grub.cfg
   ```

   这适用于基于 BIOS 和 UEFI 的系统。

   警告

   对于基于 BIOS 和 UEFI 的机器，重建 **grub.cfg** 的路径是一样的。实际的 **grub.cfg** 只在 BIOS 路径中存在。UEFI 路径有一个 stub 文件，不能使用 **grub2-mkconfig** 命令进行修改或重新创建。

4. 按照 重新安装 GRUB 流程，来在 **/boot/** 分区上恢复 GRUB。

第 14 章 安装 KDUMP

kdump 服务默认在 Red Hat Enterprise Linux 安装上安装并激活。

14.1. KDUMP

kdump 是一个提供崩溃转储机制的服务，并生成一个崩溃转储或 **vmcore** 转储文件。**vmcore** 包括系统内存的内容，以用于分析和故障排除。**kdump** 使用 **kexec** 系统调用引导到第二个内核 *捕获内核*，而不重启。这个内核捕获崩溃的内核的内存内容，并将其保存到一个文件中。第二个内核位于系统内存的保留部分。

重要

当系统出现故障时，内核崩溃转储是唯一可用的信息。因此，在关键任务环境中运行 **kdump** 非常重要。您必须在正常的内核更新周期中定期更新并测试 **kexec-tools**、**kdump-utils** 和 **makedumpfile** 软件包。这在安装新内核功能时非常重要。

如果您在机器上有多个内核，则可以为所有安装的内核或只为指定的内核启用 **kdump**。安装 **kdump** 时，系统会创建一个默认的 **/etc/kdump.conf** 文件。**/etc/kdump.conf** 包含默认最小 **kdump** 配置，您可以编辑它来自定义 **kdump** 配置。

14.2. 使用 ANACONDA 安装 KDUMP

Anaconda 安装程序在交互安装过程中为 **kdump** 配置提供了一个图形界面屏幕。您可以启用 **kdump**，并保留所需的内存量。

流程

1. 在 Anaconda 安装程序上，点 KDUMP 并启用 **kdump**。
2. 在 Kdump Memory Reservation 中，如果您必须自定义内存保留，请选择 Manual'。
3. 在 KDUMP > Memory for Reserved (MB)中，设置 **kdump** 所需的内存保留。

14.3. 在命令行中安装 KDUMP

安装选项，如自定义 Kickstart 安装，在某些情况下 **不会** 默认安装或启用 **kdump**。以下流程帮助您在这种情况下启用 **kdump**。

先决条件

- 一个有效的 Red Hat Enterprise Linux 订阅。
- 包含用于您的系统 CPU 架构的 **kexec-tools**、**kdump-utils** 和 **makedumpfile** 软件包的存储库。
- 满足 **kdump** 配置和目标的要求。详情请查看支持的 kdump 配置和目标。

流程

1. 检查您的系统上是否已安装了 **kdump** ：

```
# rpm -q kexec-tools kdump-utils makedumpfile
```

2. 如果没有安装 **kdump**，请安装所需的软件包：

```
# dnf install kexec-tools kdump-utils makedumpfile
```

第 15 章 在命令行中配置 KDUMP

在系统引导过程中为 **kdump** 保留内存。您可以在系统的 Grand Unified Bootloader (GRUB)配置文件中配置内存大小。内存大小取决于配置文件中指定的 **crashkernel=** 值，以及系统的物理内存的大小。

15.1. 估算 KDUMP 大小

在规划和构建 **kdump** 环境时，务必要知道崩溃转储文件所需的空间。

makedumpfile --mem-usage 命令估计崩溃转储文件所需的空间。它生成一个内存使用率报告。报告帮助您决定转储级别以及可以安全排除的页。

流程

- 输入以下命令生成内存用量报告：

```
# makedumpfile --mem-usage /proc/kcore


TYPE        PAGES    EXCLUDABLE    DESCRIPTION
----------------------------------------------------------------
ZERO          501635      yes        Pages filled with zero
CACHE         51657       yes        Cache pages
CACHE_PRIVATE 5442        yes        Cache pages + private
USER          16301       yes        User process pages
FREE          77738211    yes        Free pages
KERN_DATA     1333192     no         Dumpable kernel data
```

重要

makedumpfile --mem-usage 命令会以页为单位报告所需的内存。这意味着您必须根据内核页面大小计算所使用的内存大小。

默认情况下，RHEL 内核在 AMD64 和 Intel 64 CPU 构架上使用 4 KB 大小的页，在 IBM POWER 构架上使用 64 KB 大小的页。

15.2. 配置 KDUMP 内存用量

kdump-utils 软件包维护默认的 **crashkernel=** 内存保留值。**kdump** 服务使用默认值为每个内核保留崩溃内核内存。默认值也可以充当参考基础值，来在手动设置 **crashkernel=** 值时估算所需的内存大小。崩溃内核的最小大小可能会因硬件和机器规格而异。

kdump 自动内存分配根据系统硬件架构和可用内存大小而有所不同。例如，在 AMD64 和 Intel 64 位构架上，**crashkernel=** 参数的默认值仅在可用内存超过 2 GB 时才可以正常工作。 **kdump-utils** 工具在 AMD64 和 Intel 64 位构架上配置以下默认的内存保留：

```
crashkernel=2G-64G:256M,64G-:512M
```

您还可以运行 **kdumpctl estimate**，来在不触发崩溃的情况下获取大概的值。估算的 **crashkernel=** 值可能不是一个准确的值，但可作为设置合适的 **crashkernel=** 值的参考。

注意

引导命令行中的 **crashkernel=1G-4G:192M,4G-64G:256M,64G:512M** 选项在 RHEL 10 及更新版本上不再被支持。

警告

测试 **kdump** 配置的命令将导致内核崩溃，且数据丢失。请仔细按照说明操作。您不能使用活跃的生产环境系统来测试 kdump 配置。

先决条件

- 您在系统上具有 root 权限。
- 您已满足配置和目标的 **kdump** 要求。详情请查看支持的 kdump 配置和目标。
- 如果是 IBM Z 系统，您已安装了 **zipl** 工具。

流程

1. 为崩溃内核配置默认值：

```
# kdumpctl reset-crashkernel --kernel=ALL
```

在配置 **crashkernel=** 值时，请通过重启启用了 **kdump** 的系统来测试配置。如果 **kdump** 内核无法引导，请逐渐增加内存大小来设置一个可接受的值。

2. 要使用自定义 **crashkernel=** 值：

   a. 配置所需的内存保留。

```
crashkernel=192M
```

另外，您还可以使用语法 **crashkernel=*<range1>:<size1>,<range2>:<size2>***，根据安装的内存总量将保留的内存量设置为一个变量。例如：

```
crashkernel=1G-4G:192M,2G-64G:256M
```

如果系统内存总量为 1 GB 或大于 4 GB，则示例保留 192 MB 内存。如果内存量超过 4 GB，则为 **kdump** 保留 256 MB。

   b. 可选：偏移保留内存。
有些系统需要保留具有固定偏移量的内存，因为 **crashkernel** 保留非常早，其希望为特殊用途保留一些区域。如果设置了偏移，则保留内存从此偏移开始。要偏移保留的内存，请使用以下语法：

```
crashkernel=192M@16M
```

示例保留从 16 MB 开始的 192 MB 内存（物理地址 0x01000000）。如果您偏移到 0 或没有指定值，则 **kdump** 会自动偏移保留的内存。您还可以在设置变量内存保留时偏移内存，方法是将偏移指定为最后一个值。例如： **crashkernel=1G-4G:192M,2G-64G:256M@16M**。

c. 更新引导装载程序配置：

```
# grubby --update-kernel ALL --args "crashkernel=<custom-value>"
```

<custom-value>必须包含您为崩溃内核配置的自定义 **crashkernel=** 值。

3. 重启以使更改生效：

```
# reboot
```

验证

通过激活 **sysrq** 键使内核崩溃。**address-YYYY-MM-DD-HH:MM:SS/vmcore** 文件保存在 **/etc/kdump.conf** 文件中指定的目标位置。如果您选择默认的目标位置，**vmcore** 文件被保存在挂载在 **/var/crash/** 下的分区中。

1. 激活 **sysrq** 键以引导到 **kdump** 内核：

```
# echo c > /proc/sysrq-trigger
```

命令导致内核崩溃，并在需要时重启内核。

2. 显示 **/etc/kdump.conf** 文件，并检查 **vmcore** 文件是否已保存到目标位置。
如需更多信息，请参阅系统中的 **grubby (8)** 手册页。

其他资源

- 如何在系统引导红帽知识库前手动修改 GRUB 中的引导参数。

15.3. 配置 KDUMP 目标

崩溃转储通常以一个文件形式存储在本地文件系统中，直接写入设备。另外，您可以使用 **NFS** 或 **SSH** 协议通过网络发送崩溃转储。一次只能设置其中一个选项来保留崩溃转储文件。默认行为是将其存储在本地文件系统的 **/var/crash/** 目录中。

先决条件

- 您在系统上具有 root 权限。
- 满足 **kdump** 配置和目标的要求。详情请查看支持的 kdump 配置和目标。

流程

- 要将崩溃转储文件保存在本地文件系统的 **/var/crash/** 目录中，请编辑 **/etc/kdump.conf** 文件并指定路径：

```
path /var/crash
```

选项 **path /var/crash** 代表 **kdump** 在其中保存崩溃转储文件的文件系统的路径。

注意

- 当您在 **/etc/kdump.conf** 文件中指定转储目标时，路径是**相对**于指定的转储目标。
- 当您没有在 **/etc/kdump.conf** 文件中指定转储目标时，该路径表示根目录的**绝对**路径。

根据当前系统中挂载的文件系统，会自动配置转储目标和调整的转储路径。

- 要保护 **kdump** 生成的崩溃转储文件以及附带文件，您应该为目标目的地目录设置正确的属性，如用户权限和 SELinux 上下文。另外，您可以定义一个脚本，如 **kdump.conf** 文件中的 **kdump_post.sh**，如下所示：

```
kdump_post <path_to_kdump_post.sh>
```

kdump_post 指令指定 **kdump** 完成捕获并将崩溃转储保存到指定的目的地 **后** 执行的 shell 脚本或命令。您可以使用此机制扩展 **kdump** 的功能，以执行包括调整文件权限在内的操作。

- 显示并了解 **kdump** 目标配置：

  a. 通过过滤注释和空行来显示有效的配置：

```
# grep -v '^#' /etc/kdump.conf | grep -v '^$'
```

  b. 输出示例：

```
ext4 /dev/mapper/vg00-varcrashvol
path /var/crash
core_collector makedumpfile -c --message-level 1 -d 31
```

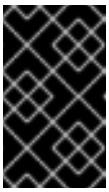
  转储目标已指定了(**ext4 /dev/mapper/vg00-varcrashvol**)，因此它被挂载到 **/var/crash**。**path** 选项也被设置为 **/var/crash**。因此，**kdump** 将 **vmcore** 文件保存在 **/var/crash/var/crash** 目录中。

- 要更改保存崩溃转储的本地目录，以 **root** 用户身份编辑 **/etc/kdump.conf** 配置文件：

  a. 从 **#path /var/crash** 行的开头删除哈希符号(**#**)。

  b. 使用预期的目录路径替换该值。例如：

```
path /usr/local/cores
```



重要

在 Red Hat Enterprise Linux 10 中，当 **kdump systemd** 服务启动时，使用 **path** 指令定义为 **kdump** 目标的目录必须存在，以避免失败。如果服务启动时目录不存在，则不再自动创建该目录。

- 要将文件写入不同的分区，请编辑 **/etc/kdump.conf** 配置文件：

  a. 根据您的选择，从 **#ext4** 行的开头删除哈希符号(**#**)。

    - 设备名称（ **#ext4 /dev/vg/lv_kdump** 行）

- 文件系统标签（**#ext4 LABEL=/boot** 行）
- UUID（**#ext4 UUID=03138356-5e61-4ab3-b58e-27507ac41937** 行）

b. 将文件系统类型和设备名称、标签或 UUID 改为所需的值。指定 UUID 值的正确语法是 **UUID="correct-uuid"** 和 **UUID=correct-uuid**。例如：

```
ext4 UUID=03138356-5e61-4ab3-b58e-27507ac41937
```

重要

您必须使用 **LABEL=** 或 **UUID=** 指定存储设备。无法保证 **/dev/sda3** 等磁盘设备名称在重启后保持一致。

当您在 IBM Z 硬件上使用 Direct Access Storage Device (DASD)时，请确保在执行 **kdump**之前，转储设备已在 **/etc/dasd.conf** 中正确指定了。

- 要将崩溃转储直接写入设备，请编辑 **/etc/kdump.conf** 配置文件：

a. 从 **#raw /dev/vg/lv_kdump** 行的开头删除哈希符号(**#**)。

b. 使用预期的设备名称替换该值。例如：

```
raw /dev/sdb1
```

- 使用 **NFS** 协议将崩溃转储保存到远程机器上：

a. 从 **#nfs my.server.com:/export/tmp** 行的开头删除哈希符号(**#**)。

b. 使用有效的主机名和目录路径替换该值。例如：

```
nfs penguin.example.com:/export/cores
```

c. 重启 **kdump** 服务以使更改生效：

```
$ sudo systemctl restart kdump.service
```

注意

当使用 NFS 指令指定 NFS 目标时，**kdump.service** 会自动尝试挂载 NFS 目标，来检查磁盘空间。不需要提前挂载 NFS 目标。要防止 **kdump.service** 挂载目标，请在 **kdump.conf** 中使用 **dracut_args --mount** 指令。这将启用 **kdump.service**，通过 **--mount** 参数指定 NFS 目标来调用 **dracut** 工具。

- 使用 SSH 协议将崩溃转储保存到远程机器上：

a. 从 **#ssh user@my.server.com** 行的开头删除哈希符号(**#**)。

b. 使用有效的用户名和密码替换该值。

c. 在配置中包含您的 SSH 密钥。

i. 从 **#sshkey /root/.ssh/kdump_id_rsa** 行的开头删除哈希符号。

ii. 将该值改为您要转储的服务器中有效密钥的位置。例如：

```
ssh john@penguin.example.com
sshkey /root/.ssh/mykey
```

其他资源

- 系统崩溃后 kdump 生成的文件

15.4. 配置 KDUMP 核心收集器

kdump 服务使用 **core_collector** 程序捕获崩溃转储镜像。在 RHEL 中， **makedumpfile** 工具是默认的内核收集器。它通过以下方式帮助缩小转储文件：

- 压缩崩溃转储文件的大小，并使用各种转储级别仅复制必要的页。
- 排除不必要的崩溃转储页。
- 过滤崩溃转储中包含的页面类型。

注意

崩溃转储文件压缩默认启用。

如果您需要自定义崩溃转储文件压缩，请按照以下流程操作。

语法

```
core_collector makedumpfile -l --message-level 1 -d 31
```

选项

- **-c**,**-l** or **-p**: 使用 **-c** 选项的 **zlib** 、 **-l** 选项的 **lzo** 、 **-p** 选项的 **snappy** 或 **-z** 选项的 **zstd**，按每个页面指定压缩转储文件格式。
- **-d (dump_level)** ：排除页面，它们不会复制到转储文件中。
- **--message-level** ：指定消息类型。您可以通过使用这个选项指定 **message_level** 来限制打印的输出。例如，把 **message_level** 设置为 7 可打印常见消息和错误消息。 **message_level** 的最大值为 31。

先决条件

- 您在系统上具有 root 权限。
- 满足 **kdump** 配置和目标的要求。详情请查看支持的 kdump 配置和目标。

流程

1. 以 **root** 用户身份编辑 **/etc/kdump.conf** 配置文件，并删除 **#core_collector makedumpfile -l --message-level 1 -d 31** 开头的哈希符号("#")。

2. 输入以下命令来启用崩溃转储文件压缩：

```
core_collector makedumpfile -l --message-level 1 -d 31
```

The **-l** 选项将压缩的文件格式设置为 LZO。**-d** 选项将转储级别设置为 31。**--message-level** 选项将消息级别设置为 1。您还可以使用 **-c**,**-p**, 或 **-z** 选项指定其他压缩格式。

如需更多信息，请参阅系统中的 **makedumpfile (8)** 手册页。

15.5. 配置 KDUMP 默认失败响应

默认情况下，当 **kdump** 不能在配置的目标位置创建崩溃转储文件时，系统会重启，转储在此过程中会丢失。您可以更改默认失败响应，并配置 **kdump**，来在无法将内核转储保存到主目标时执行不同的操作。额外的操作是：

dump_to_rootfs

将内核转储保存到 **root** 文件系统。

reboot

重启系统，在此过程中会丢失内核转储。

halt

停止系统，在此过程中会丢失内核转储。

poweroff

关闭系统，在此过程中会丢失内核转储。

shell

从 **initramfs** 中运行 shell 会话，您可以手动记录内核转储。

final_action

在 **kdump** 成功或在 shell 或 **dump_to_rootfs** 失败操作完成后，启用额外的操作，如 **reboot**、**halt** 和 **poweroff**。默认为 **reboot**。

failure_action

指定在内核崩溃中转储可能失败时要执行的操作。默认为 **reboot**。

先决条件

- root 权限。
- 满足 **kdump** 配置和目标的要求。详情请查看支持的 kdump 配置和目标。

流程

1. 以 **root** 用户身份，删除 **/etc/kdump.conf** 配置文件中 **#failure_action** 行开头的哈希符号(**#**)。
2. 将值替换为所需的操作。

```
failure_action poweroff
```

其他资源

- 配置 kdump 目标

15.6. KDUMP 的配置文件

kdump 内核的配置文件是 **/etc/sysconfig/kdump**。此文件控制 **kdump** 内核命令行参数。对于大多数配置，请使用默认选项。然而，在某些情况下，您可能需要修改某些参数来控制 **kdump** 内核行为。例如：修改 **KDUMP_COMMANDLINE_APPEND** 选项，以附加 **kdump** 内核命令行来获取详细的调试输出或修改 **KDUMP_COMMANDLINE_REMOVE** 选项，以从 **kdump** 命令行中删除参数。

KDUMP_COMMANDLINE_REMOVE

这个选项从当前 **kdump** 命令行中删除参数。它删除了可能导致 **kdump** 错误或 **kdump** 内核引导失败的参数。这些参数可能已从之前的 **KDUMP_COMMANDLINE** 进程解析了，或者从 **/proc/cmdline** 文件继承了。
如果未配置此变量，它将继承 **/proc/cmdline** 文件中的所有值。配置此选项还提供了有助于调试问题的信息。

要删除某些参数，请将其添加到 **KDUMP_COMMANDLINE_REMOVE** 中，如下所示：

```
# KDUMP_COMMANDLINE_REMOVE="hugepages hugepagesz slub_debug quiet log_buf_len swiotlb"
```

KDUMP_COMMANDLINE_APPEND

此选项将参数附加到当前命令行。这些参数可能已被之前的 **KDUMP_COMMANDLINE_REMOVE** 变量解析了。
对于 **kdump** 内核，禁用某些模块，如 **mce**、**cgroup**、**numa**，**hest_disable** 有助于防止内核错误。这些模块可能会消耗为 **kdump** 保留的大部分内核内存，或者导致 **kdump** 内核引导失败。

要在 **kdump** 内核命令行中禁用内存 **cgroups**，请运行以下命令：

```
KDUMP_COMMANDLINE_APPEND="cgroup_disable=memory"
```

如需更多信息，请参阅 **/etc/sysconfig/kdump** 文件。

15.7. 测试 KDUMP 配置

配置 **kdump** 后，您必须手动测试系统崩溃，并确保在定义的 **kdump** 目标中生成了 **vmcore** 文件。**vmcore** 文件是从新引导的内核的上下文中捕获的。因此，**vmcore** 有用于调试内核崩溃的重要信息。

警告

不要对活动状态的生产环境系统测试 **kdump**。测试 **kdump** 的命令将导致内核崩溃，且数据丢失。根据您的系统架构，确保您安排了充足的维护时间，因为 **kdump** 测试可能需要多次重启，且启动时间很长。

如果 **vmcore** 文件在 **kdump** 测试过程中没有生成，请在再次运行测试前识别并修复问题，以使 **kdump** 测试成功。

如果进行任何手动系统修改，您必须在任何系统修改的最后测试 **kdump** 配置。例如，如果您进行以下任何更改，请确保为以下情形测试 **kdump** 配置，以获取最佳 **kdump** 性能：

- 软件包升级。
- 硬件级别的更改，如存储或网络更改。
- 固件升级。
- 包括第三方模块的新安装和应用程序升级。
- 如果您使用热插拔机制在支持此机制的硬件上添加更多内存。
- 在 **/etc/kdump.conf** 或 **/etc/sysconfig/kdump** 文件中进行更改后。

先决条件

- 您在系统上具有 root 权限。
- 您已保存了所有重要数据。测试 **kdump** 的命令导致内核崩溃及数据丢失。
- 您已根据系统架构安排了大量机器维护时间。

流程

1. 启用 **kdump** 服务：

```
# kdumpctl restart
```

2. 使用 **kdumpctl** 检查 **kdump** 服务的状态：

```
# kdumpctl status
  kdump:Kdump is operational
```

另外，如果您使用 **systemctl** 命令，输出会打印在 **systemd** 日志中。

3. 启动内核崩溃来测试 **kdump** 配置。**sysrq-trigger** 组合键导致内核崩溃，并在需要时可能重启系统。

```
# echo c > /proc/sysrq-trigger
```

在内核重启时，***address-YYYY-MM-DD-HH:MM:SS*/vmcore** 文件在您在 **/etc/kdump.conf** 文件中指定的位置创建。默认值为 **/var/crash/**。

其他资源

- 配置 kdump 目标

15.8. 系统崩溃后 KDUMP 生成的文件

系统崩溃后，**kdump** 服务会将内核内存捕获在转储文件(**vmcore**)中，它还会生成额外的诊断文件，以帮助故障排除和事后分析。

kdump 生成的文件：

- **vmcore** - 崩溃时包含系统内存的主内核内存转储文件。它包含根据 **kdump** 配置中指定的 **core_collector** 程序配置的数据。默认情况下，包含内核数据结构、进程信息、堆栈跟踪和其他诊断信息。
- **vmcore-dmesg.txt** - panic 的主内核中的内核环缓冲区日志的内容(**dmesg**)。
- **kexec-dmesg.log** - 有从收集 **vmcore** 数据的辅助 **kexec** 内核执行中得到的内核和系统日志消息。

其他资源

- 什么是内核环缓冲

15.9. 启用和禁用 KDUMP 服务

您可以配置，以对特定的内核或所有安装的内核启用或禁用 **kdump** 功能。您必须定期测试 **kdump** 功能，并验证其是否正确运行。

先决条件

- 您在系统上具有 root 权限。
- 您已为配置和目标完成了 **kdump** 要求。请参阅 支持的 kdump 配置和目标。
- 用于安装 **kdump** 的所有配置都是根据需要设置的。

流程

1. 为 **multi-user.target** 启用 **kdump** 服务：

```
# systemctl enable kdump.service
```

2. 在当前会话中启动服务：

```
# systemctl start kdump.service
```

3. 停止 **kdump** 服务：

```
# systemctl stop kdump.service
```

4. 禁用 **kdump** 服务：

```
# systemctl disable kdump.service
```

警告

建议将 **kptr_restrict=1** 设置为默认。当将 **kptr_restrict** 设置为(1)作为默认时，**kdumpctl** 服务会加载崩溃内核，而不考虑内核地址空间布局(**KASLR**)是否启用了。

如果 **kptr_restrict** 没有被设置为 **1**，并且 KASLR 被启用了，则生成的 **/proc/kore** 文件的内容全为零。**kdumpctl** 服务无法访问 **/proc/kcore** 文件，并加载崩溃内核。**kexec-kdump-howto.txt** 文件显示一个要设置 **kptr_restrict=1** 的警告消息。验证 **sysctl.conf** 文件中的以下内容，以确保 **kdumpctl** 服务加载崩溃内核：

- **sysctl.conf** 文件中的内核 **kptr_restrict=1**。

15.10. 防止内核驱动程序为 KDUMP 加载

您可以通过在 **/etc/sysconfig/kdump** 配置文件中添加 **KDUMP_COMMANDLINE_APPEND=** 变量来从加载某些内核驱动程序中控制捕获内核。使用这个方法，您可以防止来自加载指定的内核模块中的 **kdump** 初始 RAM 磁盘镜像 **initramfs**。这有助于防止内存不足(OOM) killer 错误或其他崩溃内核失败。

您可以使用以下配置选项之一附加 **KDUMP_COMMANDLINE_APPEND=** 变量：

- **rd.driver.blacklist=*<modules>***
- **modprobe.blacklist=*<modules>***

先决条件

- 您在系统上具有 root 权限。

流程

1. 显示载入到当前运行内核的模块的列表。选择您要阻止其加载的内核模块：

```
$ lsmod

Module                  Size  Used by
fuse                  126976  3
xt_CHECKSUM            16384  1
ipt_MASQUERADE         16384  1
uinput                 20480  1
xt_conntrack           16384  1
```

2. 更新 **/etc/sysconfig/kdump** 文件中的 **KDUMP_COMMANDLINE_APPEND=** 变量。例如：

```
KDUMP_COMMANDLINE_APPEND="rd.driver.blacklist=hv_vmbus,hv_storvsc,hv_utils,hv_netvsc,hid-hyperv"
```

另外，考虑以下使用 **modprobe.blacklist=*<modules>*** 配置选项的示例：

```
KDUMP_COMMANDLINE_APPEND="modprobe.blacklist=emcp modprobe.blacklist=bnx2fc
modprobe.blacklist=libfcoe modprobe.blacklist=fcoe"
```

3. 重启 **kdump** 服务：

```
# systemctl restart kdump
```

如需更多信息，请参阅系统中的 **dracut.cmdline** 手册页。

15.11. 在使用加密磁盘的系统中运行 KDUMP

当您运行 LUKS 加密的分区时，系统需要一定数量的可用内存。如果系统可用内存量小于所需的可用内存量，则 **cryptsetup** 实用程序无法挂载分区。因此，在第二个内核（捕获内核）中将 **vmcore** 文件捕获到加密的目标位置会失败。

kdumpctl estimate 命令可帮助您估算 **kdump** 所需的内存量。**kdumpctl estimate** 打印 **crashkernel** 值，这是 **kdump** 所需的最合适的内存大小。

crashkernel 值是根据当前的内核大小、内核模块、initramfs 和 LUKS 加密的目标内存要求计算的。

如果您使用自定义 **crashkernel=** 选项，**kdumpctl estimate** 会打印 **LUKS required size** 的值。值是 LUKS 加密目标所需的内存大小。

流程

1. 输出估计的 **crashkernel=** 值：

```
# *kdumpctl estimate*

Encrypted kdump target requires extra memory, assuming using the keyslot with minimum
memory requirement
  Reserved crashkernel:    256M
  Recommended crashkernel: 652M

  Kernel image size:   47M
  Kernel modules size: 8M
  Initramfs size:      20M
  Runtime reservation: 64M
  LUKS required size:  512M
  Large modules: <none>
  WARNING: Current crashkernel size is lower than recommended size 652M.
```

2. 通过增加 **crashkernel=** 值来配置所需的内存量。

3. 重启系统。

注意

如果 **kdump** 服务仍无法将转储文件保存到加密的目标，请根据需要增大 **crashkernel=** 值。

第 16 章 启用 KDUMP

对于 Red Hat Enterprise Linux 系统，您可以配置在特定内核或所有安装的内核上启用或者禁用 **kdump** 功能。但是，您必须定期测试 **kdump** 功能，并验证其工作状态。

16.1. 为所有安装的内核启用 KDUMP

在安装 **kdump-utils** 后，**kdump.service** 通过启用 **kdump** 服务启动。您可以为在机器上安装的所有内核启用并启动 **kdump** 服务。

先决条件

- 您在系统上具有 root 权限。

流程

1. 将 **crashkernel=** 命令行参数添加到所有安装的内核中：

```
# grubby --update-kernel=ALL --args="crashkernel=xxM"
```

   xxM 是所需的内存（以 MB 为单位）。

2. 重启系统：

```
# reboot
```

3. 启用 **kdump** 服务：

```
# systemctl enable --now kdump.service
```

验证

- 检查 **kdump** 服务是否正在运行：

```
# systemctl status kdump.service

○ kdump.service - Crash recovery kernel arming
    Loaded: loaded (/usr/lib/systemd/system/kdump.service; enabled; vendor preset:
disabled)
    Active: active (live)
```

16.2. 为特定安装的内核启用 KDUMP

您可以为机器上的特定内核启用 **kdump** 服务。

先决条件

- 您在系统上具有 root 权限。

流程

1. 列出安装在机器上的内核：

```
# ls -a /boot/vmlinuz-*
/boot/vmlinuz-0-rescue-2930657cd0dc43c2b75db480e5e5b4a9
/boot/vmlinuz-6.12.0-55.9.1.el10_0.x86_64
/boot/vmlinuz-6.12.0-55.9.1.el10_0.x86_64
```

2. 向系统的 Grand Unified Bootloader (GRUB)配置中添加一个特定的 **kdump** 内核：例如：

```
# grubby --update-kernel=vmlinuz-6.12.0-55.9.1.el10_0.x86_64 --args="crashkernel=xxM"
```

xxM 是所需的内存保留（以 MB 为单位）。

3. 启用 **kdump** 服务：

```
# systemctl enable --now kdump.service
```

验证

- 检查 **kdump** 服务是否正在运行：

```
# systemctl status kdump.service

○ kdump.service - Crash recovery kernel arming
    Loaded: loaded (/usr/lib/systemd/system/kdump.service; enabled; vendor preset:
disabled)
    Active: active (live)
```

16.3. 禁用 KDUMP 服务

您可以在 Red Hat Enterprise Linux 系统上停止 **kdump.service**，并禁止该服务启动。

先决条件

- 满足 **kdump** 配置和目标的要求。详情请查看支持的 kdump 配置和目标。

流程

1. 要在当前会话中停止 **kdump** 服务：

```
# systemctl stop kdump.service
```

2. 要禁用 **kdump** 服务：

```
# systemctl disable kdump.service
```

警告

您必须将 **kptr_restrict=1** 设置为默认。当将 **kptr_restrict** 设置为(1)作为默认值时，**kdumpctl** 服务会加载崩溃内核，而无论是否启用了内核地址空间布局 (**KASLR**)。

如果 **kptr_restrict** 没有设置为 **1**，且 **KASLR** 被启用了，则 **/proc/kore** 文件的内容被生成为全零。**kdumpctl** 服务无法访问 **/proc/kcore** 文件，并加载崩溃内核。**kexec-kdump-howto.txt** 文件会显示一条警告信息，其建议您设置 **kptr_restrict=1**。验证 **sysctl.conf** 文件中的以下内容，以确保 **kdumpctl** 服务加载崩溃内核：

- **sysctl.conf** 文件中的内核 **kptr_restrict=1**。

其他资源

- 管理 systemd

第 17 章 支持的 KDUMP 配置和目标

kdump 机制是 Linux 内核的一个功能，它在发生内核崩溃时生成一个崩溃转储文件。内核转储文件有关键的信息，可帮助分析和确定内核崩溃的根本原因。崩溃可能是因为各种因素，举几个例子，如硬件问题或第三方内核模块问题。

通过使用提供的信息和流程，您可以执行以下操作：

- 识别 Red Hat Enterprise Linux 系统所支持的配置和目标。
- 配置 kdump。
- 验证 kdump 操作。

17.1. KDUMP 的内存要求

要让 **kdump** 捕获内核崩溃转储，并保存它以便进一步分析，应该为捕获内核永久保留系统内存的一部分。保留时，主内核无法使用系统内存的这一部分。

内存要求因某些系统参数而异。主要因素之一就是系统的硬件构架。要识别确切的机器架构，如 Intel 64 和 AMD64，也称为 x86_64，并将其输出到标准输出，请使用以下命令：

```
$ uname -m
```

使用上述最小内存要求的列表，您可以设置合适的内存大小，来在最新可用版本上为 **kdump** 自动保留内存。内存大小取决于系统的架构和总可用物理内存。

表 17.1. kdump 所需的最小保留内存量

架构	可用内存	最小保留内存
AMD64 和 Intel 64 (**x86_64**)	2 GB 到 64 GB	256 MB 内存
	64 GB 及更多	512 MB 内存
64 位 ARM (4k 页)	1 GB 到 4 GB	256 MB 内存
	4 GB 到 64 GB	320 MB RAM
	64 GB 及更多	576 MB RAM
64 位 ARM (64k 页)	1 GB 到 4 GB	356 MB RAM
	4 GB 到 64 GB	420 MB RAM
	64 GB 及更多	676 MB RAM
IBM Power 系统 (**ppc64le**)	2 GB 到 4 GB	384 MB 内存
	4 GB 到 16 GB	512 MB 内存

架构	可用内存	最小保留内存
	16 GB 到 64 GB	1 GB 内存
	64 GB 到 128 GB	2 GB 内存
	128 GB 及更多	4 GB 内存
IBM Z (**s390x**)	2 GB 到 64 GB	256 MB 内存
	64 GB 及更多	512 MB 内存

在很多系统中，**kdump** 可以估算所需内存量并自动保留。默认情况下，此行为是启用的，但仅适用于内存总量超过特定数量的系统，这些内存因系统架构而异。

重要

根据系统中内存总量自动配置保留内存是最佳工作量估算。实际需要的内存可能因其他因素（如 I/O 设备）而异。没有使用足够的内存可能会导致 debug 内核在出现内核 panic 的情况下无法作为捕获内核引导。要避免这个问题，请充分增加崩溃内核内存。

其他资源

- Red Hat Enterprise Linux 技术能力和限制

17.2. 自动内存保留的最小阈值

默认情况下，**kdump-utils** 工具配置 **crashkernel** 命令行参数，并为 **kdump** 保留一定数量的内存。但是，在某些系统上，仍可在引导装载程序配置文件中使用 **crashkernel** 参数，或者在图形配置工具中启用这个选项，来为 **kdump** 分配内存。要使此自动保留正常工作，系统中需要有一定数量的总内存。内存要求因系统架构而异。如果系统内存小于指定的阈值，则您必须手动配置内存。

表 17.2. 自动内存保留所需的最小内存量

构架	所需的内存
AMD64 和 Intel 64 (**x86_64**)	2 GB
IBM Power 系统 (**ppc64le**)	2 GB
IBM Z (**s390x**)	2 GB
64-bit ARM	2 GB

注意

RHEL 10 不再支持引导命令行中的 **crashkernel=1G-4G:192M,4G-64G:256M,64G:512M** 选项。

17.3. 支持的 KDUMP 目标

当发生内核崩溃时，操作系统会将转储文件保存在配置的或默认的目标位置上。您可以将转储文件直接保存到设备上，将其作为文件存储在本地文件系统上，或者通过网络发送转储文件。使用以下转储目标的列表，您可以知道 **kdump** 目前支持的或不支持的目标。

表 17.3. RHEL 10 上的 kdump 目标

目标类型	支持的目标	不支持的目标
物理存储	• 逻辑卷管理器(LVM)。 • 精简配置卷。 • 光纤通道(FC)磁盘，如 **qla2xxx**、**lpfc**、**bnx2fc** 和 **bfa**。 • 网络存储服务器上 **iSCSI** 软件配置的逻辑设备。 • **mdraid** 子系统作为一个软件 **RAID** 的解决方案。 • 硬件 RAID，如 **smartpqi**、**hpsa**、**megaraid**、**mpt3sas**、**ausraid** 和 **mpi3mr**。 • **SCSI** 和 **SATA** 磁盘。 • **iSCSI** 和 **HBA** 卸载。 • 硬件 **FCoE**，如 **qla2xxx** 和 **lpfc**。 • 软件 **FCoE**，如 **bnx2fc**。要使软件 **FCoE** 正常工作，可能需要额外的内存配置。	• BIOS RAID。 • 带有 **iBFT** 的软件 **iSCSI**. 目前支持的传输有 **bnx2i**、**cxgb3i** 和 **cxgb4i**。 • 带有混合设备驱动程序的软件 iSCSI，如 **be2iscsi**。 • 以太网上的光纤通道 (**FCoE**)。 • 传统 IDE。 • **GlusterFS** 服务器. • **GFS2** 文件系统。 • 集群逻辑卷管理器 (CLVM)。 • 高可用性 LVM 卷(HA-LVM)。
网络	• 使用 **igb**,**ixgbe**,**ice**,**i40e**,**e1000e**,**igc**,**tg3**,**bnx2x**,**bnxt_en**,**qede**,**cxgb4**,**be2net**, **enic**,**sfc**,**mlx4_en**,**mlx5_core**,**r8169**,**atlantic**, **nfp**, 以及仅在 64 位 ARM 架构上的 **nicvf** 等内核模块的硬件。	• 使用 **sfc SRIOV**、**cxgb4vf** 和 **pch_gbe** 等内核模块的硬件。 • IPv6 协议. • 无线连接。 • InfiniBand 网络。 • 网桥和 team 上的 VLAN 网络。

目标类型	支持的目标	不支持的目标
虚拟机监控程序（Hypervisor）	• 基于内核的虚拟机 (KVM)。 • 仅在某些配置中的 Xen hypervisor。 • ESXi 6.6, 6.7, 7.0. • 仅在 RHEL Gen1 UP 客户机及更高版本上的 Hyper-V 2012 R2。	
Filesystem	**ext[234]fs**、**XFS**、**virtiofs** 和 NFS 文件系统。	**Btrfs** 文件系统。
固件	• 基于 BIOS 的系统。 • UEFI 安全引导。	

其他资源

- 配置 kdump 目标

17.4. 支持的 KDUMP 过滤等级

要缩小转储文件的大小，**kdump** 使用 **makedumpfile** 内核收集器压缩数据，并排除不需要的信息，例如，您可以使用 **-8** 级别来删除 **hugepages** 和 **hugetlbfs** 页。**makedumpfile** 当前支持的级别可在 *Filtering levels for `kdump`* 表中看到。

表 17.4. kdump的过滤级别

选项	描述
1	零页
2	缓存页
4	缓存私有
8	用户页
16	可用页

其他资源

- 配置 kdump 核心收集器

17.5. 支持的默认故障响应

默认情况下，当 **kdump** 创建内核转储失败时，操作系统会重启。但是，当无法将内核转储保存到主目标上时，您可以将 **kdump** 配置为执行不同的操作。

dump_to_rootfs

尝试将内核转储保存到 root 文件系统。这个选项在与网络目标合并时特别有用：如果网络目标无法访问，这个选项配置 kdump 以在本地保存内核转储。之后会重启该系统。

reboot

重启系统，这个过程会丢失 core 转储文件。

halt

关闭系统，这个过程会丢失 core 转储文件。

poweroff

关闭系统，这个此过程会丢失 core 转储。

shell

从 initramfs 内运行 shell 会话，允许用户手动记录核心转储。

final_action

在 **kdump** 成功，或 **shell** 或 **dump_to_rootfs** 失败操作完成后，启用额外的操作，如 **reboot**、**halt** 和 **poweroff** 操作。默认的 **final_action** 选项为 **reboot**。

failure_action

指定在内核崩溃时转储可能会失败时要执行的操作。默认 **failure_action** 选项是 **reboot**。

其他资源

- 配置 kdump 默认失败响应

17.6. 使用 FINAL_ACTION 参数

当 **kdump** 成功或者 **kdump** 无法在配置的目标处保存 **vmcore** 文件时，您可以使用 **final_action** 参数执行额外的操作，如 **reboot**、**halt** 和 **poweroff**。如果没有指定 **final_action** 参数，则 **reboot** 是默认的响应。

流程

1. 要配置 **final_action**，请编辑 **/etc/kdump.conf** 文件并添加以下选项之一：
   - **final_action reboot**
   - **final_action halt**
   - **final_action poweroff**
2. 重启 **kdump** 服务，以使更改生效。

```
# kdumpctl restart
```

17.7. 使用 FAILURE_ACTION 参数

failure_action 参数指定在内核崩溃时转储失败时要执行的操作。**failure_action** 的默认操作是 **重启** 系统。

参数接受以下要执行的操作：

reboot

转储失败后重启系统。

dump_to_rootfs

当配置了非 root 转储目标时，将转储文件保存在 root 文件系统上。

halt

关闭系统。

poweroff

停止系统上正在运行的操作。

shell

在 **initramfs** 中启动 shell 会话，您可以从中手动执行其他恢复操作。

流程

1. 要将操作配置为在转储失败时执行的操作，请编辑 **/etc/kdump.conf** 文件并指定其中一个 **failure_action** 选项：
   - **failure_action reboot**
   - **failure_action halt**
   - **failure_action poweroff**
   - **failure_action shell**
   - **failure_action dump_to_rootfs**
2. 重启 **kdump** 服务，以使更改生效。

```
# kdumpctl restart
```

第 18 章 固件支持的转储机制

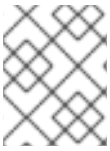
固件支持的转储 (fadump) 是一个转储捕获机制，作为 IBM POWER 系统中 **kdump** 机制的替代选择。**kexec** 和 **kdump** 机制可用于在 AMD64 和 Intel 64 系统中捕获内核转储。但是，一些硬件（如小型系统和大型机计算机）使用板上固件来隔离内存区域，并防止意外覆盖对分析崩溃很重要的数据。**fadump** 工具针对 **fadump** 机制及其与 IBM POWER 系统上 RHEL 的集成进行了优化。

18.1. IBM POWERPC 硬件支持转储固件

fadump 实用程序从带有 PCI 和 I/O 设备的完全重设系统中捕获 **vmcore** 文件。这种机制使用固件在崩溃期间保留内存区域，然后重复使用 **kdump** 用户空间脚本保存 **vmcore** 文件。内存区域由所有系统内存内容组成，但引导内存、系统注册和硬件页面表条目 (PTE) 除外。

fadump 机制通过重新引导分区并使用新内核转储之前内核崩溃中的数据，提供比传统转储类型的更高可靠性。**fadump** 需要一个基于 IBM POWER6 处理器或更高版本的硬件平台。

有关 **fadump** 机制的进一步详情，包括重置硬件的特定于 PowerPC 的方法，请查看 **/usr/share/doc/kdump-utils/fadump-howto.txt** 文件。



注意

未保留的内存区域（称为引导内存）是在崩溃事件后成功引导内核所需的 RAM 量。默认情况下，引导内存大小为 256MB 或系统 RAM 总量的 5%，以较大者为准。

与 **kexec-initiated** 事件不同，**fadump** 机制使用 production 内核恢复崩溃转储。崩溃后引导时，PowerPC 硬件使设备节点 **/proc/device-tree/rtas/ibm.kernel-dump** 可供 **proc** 文件系统 (**procfs**) 使用。**fadump-aware kdump** 脚本，检查存储的 **vmcore**，然后完全完成系统重启。

18.2. 启用固件支持的转储机制

您可以通过启用固件支持的转储(**fadump**)机制来增强 IBM POWER 系统的崩溃转储功能。

在安全引导环境中，GRUB 引导装载程序分配一个引导内存区域，称为 Real Mode Area (RMA)。RMA 有 512 MB 大小的内存，在引导组件之间分配。如果组件超过其大小分配，则 **GRUB** 会失败，并显示内存不足(**OOM**)错误。

警告

不要在 RHEL 9.1 及更早版本上的安全引导环境中启用固件支持的转储(**fadump**)机制。GRUB 引导装载程序失败，并显示以下错误：

```
error: ../../grub-core/kern/mm.c:376:out of memory.
Press any key to continue…
```

仅当您因为 **fadump** 配置而增加默认 **initramfs** 大小时，系统才是可恢复的。

有关恢复系统的临时解决方案方法的详情，请参考 GRUB 内存不足(OOM)中的系统引导结束 文章。

先决条件

- 您在系统上具有 root 权限。

流程

1. 安装 **kexec-tools**、**kdump-utils** 和 **makedumpfile** 软件包。
2. 为 **crashkernel** 配置默认值：

```
# kdumpctl reset-crashkernel --fadump=on --kernel=ALL
```

3. 可选：保留引导内存而不是默认值：

```
# grubby --update-kernel ALL --args="fadump=on crashkernel=xxM"
```

xxM 是所需的内存大小（以 MB 为单位）。

> **注意**
>
> 当指定引导选项时，请通过重启启用了 **kdump** 的内核来测试配置。如果 **kdump** 内核无法引导，请逐渐增加 **crashkernel** 值来设置适当的值。

4. 重启以使更改生效：

```
# reboot
```

18.3. IBM Z 硬件支持的固件转储机制

IBM Z 系统支持以下固件支持的转储机制：

- **独立转储 (sadump)**
- **VMDUMP**

IBM Z 系统支持并使用 **kdump** 基础架构。但是，使用 IBM Z 的固件辅助转储(fadump)方法之一有以下优点：

- 系统控制台启动并控制 **sadump** 机制，并将其存储在 **IPL** 可引导设备上。
- **VMDUMP** 机制与 **sadump** 类似。此工具也从系统控制台启动，但会从硬件检索生成的转储并将其复制到系统以进行分析。
- 这些方法（与其他基于硬件的转储机制类似）能够在 **kdump** 服务启动前捕获机器在早期启动阶段的状态。
- 虽然 **VMDUMP** 包含一种将转储文件接收到 Red Hat Enterprise Linux 系统的机制，但 **VMDUMP** 的配置和控制是从 IBM Z 硬件控制台进行管理的。

其他资源

- 独立转储

- 使用 VMDUMP 在 z/VM 中创建转储

18.4. 在 FUJITSU PRIMEQUEST 系统中使用 SADUMP

当 **kdump** 无法成功完成时，Fujitsu **sadump** 机制提供了一种 **回退** 转储捕获。您可以从系统管理板(MMB)接口手动调用 **sadump**。通过使用 MMB，为 Intel 64 或 AMD64 服务器配置 **kdump**，然后继续启用 **sadump**。

流程

1. 在 **/etc/sysctl.conf** 文件中添加或编辑以下行，以确保 **sadump** 的 **kdump** 按预期启动：

```
kernel.panic=0
kernel.unknown_nmi_panic=1
```

> **警告**
>
> 特别是，请确保在 **kdump** 后系统不会重启。如果系统在 **kdump** 保存 **vmcore** 文件失败后重启，则无法调用 **sadump**。

2. 适当地将 **/etc/kdump.conf** 中的 **failure_action** 参数设置为 **halt** 或 **shell**。

```
failure_action shell
```

如需更多信息，请参阅 FUJITSU Server PRIMEQUEST 2000 系列安装手册。

第 19 章 分析内核转储

要识别系统崩溃的原因，您可以使用 **crash** 工具，它提供了一个类似于 GNU Debugger (GDB)的交互式提示。通过使用 **crash**，您可以分析 **kdump**、**netdump**、**diskdump** 或 **xendump**，以及正在运行的 Linux 系统所创建的内核转储。另外，您可以使用 Kernel Oops Analyzer 或 Kdump Helper 工具。

19.1. 安装 CRASH 工具

使用提供的信息，了解所需的软件包以及安装 **crash** 工具的流程。默认情况下，在 Red Hat Enterprise Linux 10 系统上可能没有安装 **crash** 工具。**crash** 是一个工具，可在系统运行期间，或内核崩溃发生且内核转储文件创建后，以交互方式分析系统状态。内核转储文件也称为 **vmcore** 文件。

流程

1. 启用相关的软件仓库：

```
# subscription-manager repos --enable baseos repository
```

```
# subscription-manager repos --enable appstream repository
```

```
# subscription-manager repos --enable rhel-10-for-x86_64-baseos-debug-rpms
```

2. 安装 **crash** 软件包：

```
# dnf install crash
```

3. 安装 **kernel-debuginfo** 软件包：

```
# dnf install kernel-debuginfo
```

软件包 **kernel-debuginfo** 将对应于正在运行的内核，并提供用于转储分析所需的数据。

19.2. 运行和退出 CRASH 工具

crash 工具是一个用于分析 **kdump** 的强大工具。通过对崩溃转储文件运行 **crash**，您可以在崩溃时洞察系统的状态，识别问题的根本原因，并对与内核相关的问题进行故障排除。

先决条件

- 识别当前运行的内核（如 **6.12.0-55.9.1.el10_0.x86_64**）。

流程

1. 要启动 **crash** 工具程序，需要将两个必要的参数传递给该命令：
   - debug-info （一个解压缩的 vmlinuz 镜像），如通过特定的 **kernel-debuginfo** 软件包提供的 **/usr/lib/debug/lib/modules/6.12.0-55.9.1.el10_0.x86_64/vmlinux**。
   - 实际的 vmcore 文件，如 **/var/crash/127.0.0.1-2021-09-13-14:05:33/vmcore**
     然后，最终的 **crash** 命令如下所示：

```
# crash /usr/lib/debug/lib/modules/6.12.0-55.9.1.el10_0.x86_64/vmlinux
/var/crash/127.0.0.1-2021-09-13-14:05:33/vmcore
```

使用 **kdump** 捕获的相同 *<kernel>* 版本。

2. 运行 crash 工具。
以下示例显示了分析一个使用 6.12.0-55.9.1.el10_0.x86_64 内核创建的内核转储。

```
...
WARNING: kernel relocated [202MB]: patching 90160 gdb minimal_symbol values

      KERNEL: /usr/lib/debug/lib/modules/6.12.0-55.9.1.el10_0.x86_64/vmlinux
    DUMPFILE: /var/crash/127.0.0.1-2021-09-13-14:05:33/vmcore  [PARTIAL DUMP]
        CPUS: 2
        DATE: Mon Sep 13 14:05:16 2021
      UPTIME: 01:03:57
LOAD AVERAGE: 0.00, 0.00, 0.00
       TASKS: 586
    NODENAME: localhost.localdomain
     RELEASE: 6.12.0-55.9.1.el10_0.x86_64
     VERSION: #1 SMP Wed Aug 29 11:51:55 UTC 2018
     MACHINE: x86_64  (2904 Mhz)
      MEMORY: 2.9 GB
       PANIC: "sysrq: SysRq : Trigger a crash"
         PID: 10635
     COMMAND: "bash"
        TASK: ffff8d6c84271800  [THREAD_INFO: ffff8d6c84271800]
         CPU: 1
       STATE: TASK_RUNNING (SYSRQ)

crash>
```

3. 要退出交互式提示符并停止 **crash**，请输入 **exit** 或 **q**。

```
crash> exit
~]#
```

注意

crash 命令也作为一个调试实时系统的强大工具被使用。但是，您必须谨慎使用它，以避免系统级的问题。

其他资源

- 预期系统重启指南

19.3. 在 CRASH 工具中显示各种指示符

使用 **crash** 工具来显示各种指示符，如内核消息缓冲区、回溯追踪、进程状态、虚拟内存信息以及打开文件。

流程

- 要显示内核消息缓冲区，请在互动提示符下输入 **log** 命令：

```
crash> log
... several lines omitted ...
EIP: 0060:[<c068124f>] EFLAGS: 00010096 CPU: 2
EIP is at sysrq_handle_crash+0xf/0x20
EAX: 00000063 EBX: 00000063 ECX: c09e1c8c EDX: 00000000
ESI: c0a09ca0 EDI: 00000286 EBP: 00000000 ESP: ef4dbf24
 DS: 007b ES: 007b FS: 00d8 GS: 00e0 SS: 0068
Process bash (pid: 5591, ti=ef4da000 task=f196d560 task.ti=ef4da000)
Stack:
 c068146b c0960891 c0968653 00000003 00000000 00000002 efade5c0 c06814d0
<0> fffffffb c068150f b7776000 f2600c40 c0569ec4 ef4dbf9c 00000002 b7776000
<0> efade5c0 00000002 b7776000 c0569e60 c051de50 ef4dbf9c f196d560 ef4dbfb4
Call Trace:
 [<c068146b>] ? __handle_sysrq+0xfb/0x160
 [<c06814d0>] ? write_sysrq_trigger+0x0/0x50
 [<c068150f>] ? write_sysrq_trigger+0x3f/0x50
 [<c0569ec4>] ? proc_reg_write+0x64/0xa0
 [<c0569e60>] ? proc_reg_write+0x0/0xa0
 [<c051de50>] ? vfs_write+0xa0/0x190
 [<c051e8d1>] ? sys_write+0x41/0x70
 [<c0409adc>] ? syscall_call+0x7/0xb
Code: a0 c0 01 0f b6 41 03 19 d2 f7 d2 83 e2 03 83 e0 cf c1 e2 04 09 d0 88 41 03 f3 c3 90 c7
05 c8 1b 9e c0 01 00 00 00 0f ae f8 89 f6 <c6> 05 00 00 00 00 01 c3 89 f6 8d bc 27 00 00 00
00 8d 50 d0 83
EIP: [<c068124f>] sysrq_handle_crash+0xf/0x20 SS:ESP 0068:ef4dbf24
CR2: 0000000000000000
```

输入 **help log** 以了解有关命令用法的更多信息。

注意

内核消息缓冲区包含有关系统崩溃的最重要的信息。它总是首先转储到 **vmcore-dmesg.txt** 文件中。如果您无法获取完整的 **vmcore** 文件，例如由于目标位置上空间不足，则您可以从内核消息缓冲中获得所需的信息。默认情况下，**vmcore-dmesg.txt** 放在 **/var/crash/** 目录中。

- 要显示内核堆栈追踪，请使用 **bt** 命令：

```
crash> bt
PID: 5591   TASK: f196d560  CPU: 2   COMMAND: "bash"
 #0 [ef4dbdcc] crash_kexec at c0494922
 #1 [ef4dbe20] oops_end at c080e402
 #2 [ef4dbe34] no_context at c043089d
 #3 [ef4dbe58] bad_area at c0430b26
 #4 [ef4dbe6c] do_page_fault at c080fb9b
 #5 [ef4dbee4] error_code (via page_fault) at c080d809
    EAX: 00000063  EBX: 00000063  ECX: c09e1c8c  EDX: 00000000  EBP: 00000000
    DS:  007b      ESI: c0a09ca0  ES:  007b      EDI: 00000286  GS:  00e0
    CS:  0060      EIP: c068124f  ERR: ffffffff  EFLAGS: 00010096
 #6 [ef4dbf18] sysrq_handle_crash at c068124f
 #7 [ef4dbf24] __handle_sysrq at c0681469
 #8 [ef4dbf48] write_sysrq_trigger at c068150a
 #9 [ef4dbf54] proc_reg_write at c0569ec2
```

```
#10 [ef4dbf74] vfs_write at c051de4e
#11 [ef4dbf94] sys_write at c051e8cc
#12 [ef4dbfb0] system_call at c0409ad5
    EAX: ffffffda  EBX: 00000001  ECX: b7776000  EDX: 00000002
    DS:  007b      ESI: 00000002  ES:  007b      EDI: b7776000
    SS:  007b      ESP: bfcb2088  EBP: bfcb20b4  GS:  0033
    CS:  0073      EIP: 00edc416  ERR: 00000004  EFLAGS: 00000246
```

输入 **bt** ***<pid>*** 以显示特定进程的回溯追踪，或者输入 **help bt** 以了解有关 **bt** 用法的更多信息。

- 要显示系统中进程的状态，请使用 **ps** 命令：

```
crash> ps
   PID    PPID  CPU   TASK    ST  %MEM     VSZ    RSS  COMM
>     0      0   0  c09dc560  RU   0.0       0      0  [swapper]
>     0      0   1  f7072030  RU   0.0       0      0  [swapper]
      0      0   2  f70a3a90  RU   0.0       0      0  [swapper]
>     0      0   3  f70ac560  RU   0.0       0      0  [swapper]
      1      0   1  f705ba90  IN   0.0    2828   1424  init
... several lines omitted ...
   5566      1   1  f2592560  IN   0.0   12876    784  auditd
   5567      1   2  ef427560  IN   0.0   12876    784  auditd
   5587   5132   0  f196d030  IN   0.0   11064   3184  sshd
>  5591   5587   2  f196d560  RU   0.0    5084   1648  bash
```

使用 **ps** ***<pid>*** 显示单个进程的状态。使用 *help ps* 了解有关 **ps** 用法的更多信息。

- 要显示基本虚拟内存信息，在交互式提示符下键入 **vm** 命令：

```
crash> vm
PID: 5591   TASK: f196d560  CPU: 2   COMMAND: "bash"
   MM       PGD      RSS    TOTAL_VM
f19b5900  ef9c6000  1648k    5084k
  VMA       START      END    FLAGS  FILE
f1bb0310    242000    260000 8000875  /lib/ld-2.12.so
f26af0b8    260000    261000 8100871  /lib/ld-2.12.so
efbc275c    261000    262000 8100873  /lib/ld-2.12.so
efbc2a18    268000    3ed000 8000075  /lib/libc-2.12.so
efbc23d8    3ed000    3ee000 8000070  /lib/libc-2.12.so
efbc2888    3ee000    3f0000 8100071  /lib/libc-2.12.so
efbc2cd4    3f0000    3f1000 8100073  /lib/libc-2.12.so
efbc243c    3f1000    3f4000 100073
efbc28ec    3f6000    3f9000 8000075  /lib/libdl-2.12.so
efbc2568    3f9000    3fa000 8100071  /lib/libdl-2.12.so
efbc2f2c    3fa000    3fb000 8100073  /lib/libdl-2.12.so
f26af888    7e6000    7fc000 8000075  /lib/libtinfo.so.5.7
f26aff2c    7fc000    7ff000 8100073  /lib/libtinfo.so.5.7
efbc211c    d83000    d8f000 8000075  /lib/libnss_files-2.12.so
efbc2504    d8f000    d90000 8100071  /lib/libnss_files-2.12.so
efbc2950    d90000    d91000 8100073  /lib/libnss_files-2.12.so
f26afe00    edc000    edd000 4040075
f1bb0a18   8047000   8118000 8001875  /bin/bash
f1bb01e4   8118000   811d000 8101873  /bin/bash
f1bb0c70   811d000   8122000 100073
f26afae0   9fd9000   9ffa000 100073
... several lines omitted ...
```

使用 **vm** ***<pid>*** 显示有关单个特定进程的信息，或使用 **help vm** 了解有关 **vm** 用法的更多信息。

- 要显示打开文件的信息，请使用 **files** 命令：

```
crash> files
PID: 5591   TASK: f196d560  CPU: 2   COMMAND: "bash"
ROOT: /    CWD: /root
 FD    FILE     DENTRY    INODE    TYPE  PATH
  0  f734f640  eedc2c6c  eecd6048  CHR   /pts/0
  1  efade5c0  eee14090  f00431d4  REG   /proc/sysrq-trigger
  2  f734f640  eedc2c6c  eecd6048  CHR   /pts/0
 10  f734f640  eedc2c6c  eecd6048  CHR   /pts/0
255  f734f640  eedc2c6c  eecd6048  CHR   /pts/0
```

使用 **files** ***<pid>*** 仅显示一个选定进程打开的文件，或者使用 **help files** 来获取有关 **files** 用法的更多信息。

19.4. 使用 KERNEL OOPS ANALYZER

Kernel Oops Analyzer 工具通过将 **oops** 消息与知识库中的已知问题进行比较，来分析崩溃转储。

先决条件

- **oops** 消息被安全地提供给 Kernel Oops Analyzer。

流程

1. 访问 Kernel Oops Analyzer 工具。
2. 要诊断内核崩溃问题，请上传 **vmcore** 中生成的内核 oops 日志。
   - 或者，您可以通过提供文本消息或 **vmcore-dmesg.txt** 作为输入来诊断内核崩溃问题。
3. 点 **DETECT**，来根据 **makedumpfile** 中的信息，将 **oops** 消息与已知解决方案进行比较。

其他资源

- Kernel Oops 分析器 文章

19.5. KDUMP HELPER 工具

Kdump Helper 工具有助于使用提供的信息设置 **kdump**。kdump 帮助程序根据您的偏好生成配置脚本。在服务器中启动并运行该脚本可设置 **kdump** 服务。

其它资源

- kdump Helper

第 20 章 使用早期 KDUMP 来捕获引导时间崩溃

早期的 kdump 是 **kdump** 机制的一个特性，如果在系统服务启动前的引导过程的早期阶段发生系统或内核崩溃，它将捕获 **vmcore** 文件。早期的 kdump 更早地在内存中加载崩溃内核和崩溃内核的 **initramfs**。

在 **kdump** 服务启动前的早期引导阶段过程中，内核崩溃有时会发生，可以捕获并保存崩溃的内核内存的内容。因此，与崩溃相关的、对于进行故障排除很重要的信息会丢失。要解决这个问题，您可以使用 **early kdump** 功能，它是 **kdump** 服务的一部分。

20.1. 启用早期 KDUMP

early kdump 功能设置崩溃内核和初始 RAM 磁盘镜像(**initramfs**)，以便早期加载以捕获早期崩溃的 **vmcore** 信息。这有助于消除丢失早期引导内核崩溃信息的风险。

先决条件

- 一个有效的 Red Hat Enterprise Linux 订阅。
- 包含用于您的系统 CPU 架构的 **kexec-tools**、**kdump-utils** 和 **makedumpfile** 软件包的存储库。
- 实现了 **kdump** 配置和目标要求。如需更多信息，请参阅 支持的 kdump 配置和目标。

流程

1. 验证 **kdump** 服务是否已启用并活跃：

```
# systemctl is-enabled kdump.service && systemctl is-active kdump.service
enabled
active
```

   如果没有启用并运行 **kdump**，请设置所有必要的配置，并验证是否已启用 **kdump** 服务。

2. 使用 **早期 kdump** 功能重建引导内核的 **initramfs** 镜像：

```
# dracut -f --add earlykdump
```

3. 添加 **rd.earlykdump** 内核命令行参数：

```
# grubby --update-kernel=/boot/vmlinuz-$(uname -r) --args="rd.earlykdump"
```

4. 重启系统以反映更改：

```
# reboot
```

验证

- 验证 **rd.earlykdump** 是否已成功添加，**early kdump** 功能是否已启用：

```
# cat /proc/cmdline
BOOT_IMAGE=(hd0,msdos1)/vmlinuz-6.12.0-55.9.1.el10_0.x86_64 root=/dev/mapper/rhel-root ro crashkernel=2G-64G:256M,64G-:512M resume=/dev/mapper/rhel-swap rd.lvm.lv=rhel/root rd.lvm.lv=rhel/swap rhgb quiet rd.earlykdump
```

```
# journalctl -x | grep early-kdump
Sep 13 15:46:11 redhat dracut-cmdline[304]: early-kdump is enabled.
Sep 13 15:46:12 redhat dracut-cmdline[304]: kexec: loaded early-kdump kernel
```

如需更多信息，请参阅 **/usr/share/doc/kdump-utils/early-kdump-howto.txt** 文件。

其他资源

- 什么是早期 kdump 支持以及如何配置它？（红帽知识库）

第 21 章 为安全引导签名内核和模块

您可以使用签名的内核和签名的内核模块来加强系统的安全性。在启用了安全引导机制的基于 UEFI 的构建系统中，您可以自我签名一个私有构建的内核或内核模块。另外，您可以将公钥导入到要部署内核或内核模块的目标系统中。

如果启用了安全引导机制，则必须使用私钥签名以下所有组件，并使用对应的公钥进行身份验证：

- UEFI 操作系统引导装载程序
- Red Hat Enterprise Linux 内核
- 所有内核模块

如果这些组件中的任何一个都没有签名和验证，则系统将无法完成引导过程。

Red Hat Enterprise Linux 包括：

- 签名的引导装载程序
- 签名的内核
- 签名的内核模块

另外，签名的第一阶段引导装载程序(shim)和签名的内核包括嵌入的红帽公钥。这些签名的可执行二进制文件和嵌入的密钥可使 Red Hat Enterprise Linux 安装、引导并使用 Microsoft UEFI 安全引导认证机构密钥运行。这些密钥由支持 UEFI 安全引导的系统上的 UEFI 固件提供。

注意

- 不是所有基于 UEFI 的系统都包括对安全引导的支持。
- 构建系统（构建和签署内核模块）不需要启用 UEFI 安全引导，甚至不需要是基于 UEFI 的系统。

21.1. 先决条件

- 要能够为外部构建的内核模块签名，请从以下软件包安装工具：

```
# dnf install pesign openssl kernel-devel mokutil keyutils
```

表 21.1. 所需工具

工具	由软件包提供	用于	用途
efikeygen	**pesign**	构建系统	生成公共和专用 X.509 密钥对
openssl	**openssl**	构建系统	导出未加密的私钥
sign-file	**kernel-devel**	构建系统	用来使用私钥为内核模块签名的可执行文件
mokutil	**mokutil**	目标系统	用于手动注册公钥的可选工具

工具	由软件包提供	用于	用途
keyctl	**keyutils**	目标系统	用于在系统密钥环中显示公钥的可选工具

21.2. 什么是 UEFI 安全引导

使用 *Unified Extensible Firmware Interface* (UEFI)安全引导技术，您可以防止不是由可信密钥签名的内核空间代码的执行。系统引导装载程序使用加密密钥进行签名。固件中的公钥数据库授权密钥签名的过程。然后，您可以在下一个阶段引导装载程序和内核中验证签名。

UEFI 安全引导建立了一个从固件到签名驱动程序和内核模块的信任链，如下所示：

- UEFI 私钥签名，公钥验证 **shim** 第一阶段引导装载程序。*证书颁发机构* (CA)反过来签署公钥。CA 存储在固件数据库中。
- **shim** 文件包含红帽公钥 **Red Hat Secure Boot (CA 密钥 1)**来验证 GRUB 引导装载程序和内核。
- 内核又包含用于验证驱动程序和模块的公钥。

安全引导是 UEFI 规范的引导路径验证组件。规范定义：

- 用于非易失性存储中加密保护的 UEFI 变量的编程接口。
- 在 UEFI 变量中存储可信的 X.509 根证书。
- UEFI 应用程序的验证，如引导装载程序和驱动程序。
- 撤销已知错误的证书和应用程序哈希的流程。

UEFI 安全引导版主可以检测未经授权的更改，但**不会** ：

- 防止安装或删除第二阶段引导装载程序。
- 需要用户明确确认此类更改。
- 停止引导路径操作。在引导过程中会验证签名，但不会在安装或更新时进行验证。

如果引导装载程序或内核不是由系统可信密钥签名的，则安全引导会阻止它们启动。

21.3. UEFI 安全引导支持

如果内核和所有载入的驱动程序都使用可信密钥签名了，则您可以在启用了 UEFI 安全引导的系统上安装并运行 Red Hat Enterprise Linux 10。红帽提供由适用的红帽密钥签名和验证的内核和驱动程序。

如果要加载外部构建的内核或驱动程序，还必须给它们签名。

UEFI 安全引导施加的限制

- 系统仅在签名被正确验证后才运行 kernel-mode 代码。
- GRUB 模块加载被禁用，因为没有签名和验证 GRUB 模块的基础架构。允许模块加载将在安全引导定义的安全范围内运行不受信任的代码。

- 红帽提供了一个签名的 GRUB 二进制文件，其有 Red Hat Enterprise Linux 上所有支持的模块。

其他资源

- UEFI 安全引导施加的限制

21.4. 使用 X.509 密钥验证内核模块的要求

在 Red Hat Enterprise Linux 10 中，当加载内核模块时，内核会针对内核系统密钥环(**.builtin_trusted_keys**)和内核平台密钥环(**.platform**)中的公共 X.509 密钥检查模块的签名。**.platform** 密钥环从第三方平台提供商提供密钥和自定义公钥。内核系统 **.blacklist** 密钥环中的密钥不包括在验证中。

您需要满足某些条件，才能在启用了 UEFI 安全引导功能的系统上载入内核模块：

- 如果启用了 UEFI 安全引导，或者指定了 **module.sig_enforce** 内核参数：
  - 您只能加载那些签名的内核模块，它们的签名是针对系统密钥环(**.builtin_trusted_keys**)或平台密钥环(**.platform**)中的密钥进行验证的。
  - 公钥不能在系统中被撤销的密钥环 (**.blacklist**)。
- 如果禁用了 UEFI 安全引导且未指定 **module.sig_enforce** 内核参数：
  - 您可以加载未签名的内核模块和签名的内核模块，而无需公钥。
- 如果系统不基于 UEFI，或者禁用 UEFI 安全引导：
  - 只有内核中嵌入的密钥才会加载到 **.builtin_trusted_keys** 和 **.platform**。
  - 您无法在不重新构建内核的情况下添加这组密钥。

表 21.2. 加载内核模块的验证要求

模块已签名	找到公钥，且签名有效	UEFI 安全引导状态	sig_enforce	模块载入	内核污点
未签名	-	未启用	未启用	成功	是
		未启用	Enabled	Fails	-
		Enabled	-	Fails	-
已签名	否	未启用	未启用	成功	是
		未启用	Enabled	Fails	-
		Enabled	-	Fails	-
已签名	是	未启用	未启用	成功	否

模块已签名	找到公钥，且签名有效	UEFI 安全引导状态	sig_enforce	模块载入	内核污点
		未启用	Enabled	成功	否
		Enabled	-	成功	否

21.5. 公钥的源

在引导过程中，内核会从一组持久性密钥中加载 X.509 密钥到以下密钥环中：

- 系统密钥环 (**.builtin_trusted_keys**)
- **.platform** 密钥环
- 系统 **.blacklist** 密钥环

表 21.3. 系统密钥环源

X.509 密钥源	用户可以添加密钥	UEFI 安全引导状态	引导过程中载入的密钥
嵌入于内核中	否	-	**.builtin_trusted_keys**
UEFI **db**	有限	未启用	否
		Enabled	**.platform**
嵌入在 **shim** 引导装载程序中	否	未启用	否
		Enabled	**.platform**
Machine Owner Key（MOK）列表	是	未启用	否
		Enabled	**.platform**

.builtin_trusted_keys

- 在引导时构建的密钥环。
- 提供可信的公钥。
- 查看密钥需要 **root** 权限。

.platform

- 在引导时构建的密钥环。
- 从第三方平台提供商提供密钥和自定义公钥。

- 查看密钥需要 **root** 权限。

.blacklist

- 带有 X.509 密钥的密钥环已被撤销。
- 使用来自 **.blacklist** 的密钥签名的模块将失败，即使您的公钥位于 **.builtin_trusted_keys** 中。
- 查看密钥需要 **root** 权限。

UEFI 安全引导 db

- 签名数据库。
- 存储 UEFI 应用程序的密钥（哈希）、UEFI 驱动程序和引导装载程序。
- 密钥可以在机器上加载。

UEFI 安全引导 dbx

- 已撤销的签名数据库。
- 防止密钥被加载。
- 从此数据库撤销的密钥被添加到 **.blacklist** 密钥环中。

21.6. 生成公钥和私钥对

要在启用了安全引导的系统上使用自定义内核或自定义内核模块，您必须生成一个公钥和私有 X.509 密钥对。您可以使用生成的私钥为内核或内核模块签名。您还可以通过在向安全引导的 Machine Owner Key (MOK)中添加相应的公钥来验证签名的内核或内核模块。

先决条件

- 您在系统上具有 root 权限。

流程

- 创建一个 X.509 公钥和私钥对。
  - 如果您只想为自定义内核 *模块* 签名：

```
# efikeygen --dbdir /etc/pki/pesign \
        --self-sign \
        --module \
        --common-name 'CN=Organization signing key' \
        --nickname 'Custom Secure Boot key'
```

  - 如果要为自定义 *内核* 签名：

```
# efikeygen --dbdir /etc/pki/pesign \
        --self-sign \
        --kernel \
```

```
        --common-name 'CN=Organization signing key' \
        --nickname 'Custom Secure Boot key'
```

- 当 RHEL 系统运行 FIPS 模式时：

```
# efikeygen --dbdir /etc/pki/pesign \
        --self-sign \
        --kernel \
        --common-name 'CN=Organization signing key' \
        --nickname 'Custom Secure Boot key'
        --token 'NSS FIPS 140-2 Certificate DB'
```

注意

在 FIPS 模式下，您必须使用 **--token** 选项，以便 **efikeygen** 在 PKI 数据库中找到默认的"NSS Certificate DB"令牌。

公钥和私钥现在存储在 **/etc/pki/pesign/** 目录中。如需更多信息，请参阅系统中的 **openssl (1)** 手册页。

重要

好的安全实践是在签名密钥的有效周期内为内核和内核模块签名。但是，**sign-file** 工具不会警告您，且无论有效期如何，密钥都将在 Red Hat Enterprise Linux 10 中使用。

其他资源

- 创建并管理 TLS 密钥和证书
- 通过在 MOK 列表中添加公钥在目标系统中注册公钥

21.7. 系统密钥环输出示例

您可以使用 **keyutils** 软件包中的 **keyctl** 实用程序显示系统密钥环中的密钥信息。

密钥环输出

以下是 **.builtin_trusted_keys**、**.platform** 和来自启用了 UEFI 安全引导的 Red Hat Enterprise Linux 10 系统的 **.blacklist** 密钥环的缩短的示例输出。

```
# keyctl list %:.builtin_trusted_keys
6 keys in keyring:
...asymmetric: Red Hat Enterprise Linux Driver Update Program (key 3): bf57f3e87...
...asymmetric: Red Hat Secure Boot (CA key 1): 4016841644ce3a810408050766e8f8a29...
...asymmetric: Microsoft Corporation UEFI CA 2011: 13adbf4309bd82709c8cd54f316ed...
...asymmetric: Microsoft Windows Production PCA 2011: a92902398e16c49778cd90f99e...
...asymmetric: Red Hat Enterprise Linux kernel signing key: 4249689eefc77e95880b...
...asymmetric: Red Hat Enterprise Linux kpatch signing key: 4d38fd864ebe18c5f0b7...

# keyctl list %:.platform
4 keys in keyring:
...asymmetric: VMware, Inc.: 4ad8da0472073...
...asymmetric: Red Hat Secure Boot CA 5: cc6fafe72...
...asymmetric: Microsoft Windows Production PCA 2011: a929f298e1...
```

```
...asymmetric: Microsoft Corporation UEFI CA 2011: 13adbf4e0bd82...

# keyctl list %:.blacklist
4 keys in keyring:
...blacklist: bin:f5ff83a...
...blacklist: bin:0dfdbec...
...blacklist: bin:38f1d22...
...blacklist: bin:51f831f...
```

示例中的 **.builtin_trusted_keys** 密钥环显示从 UEFI 安全引导 **db** 密钥和嵌在 **shim** 引导装载程序中的 红**帽安全引导(CA 密钥 1)** 添加了两个密钥。

内核控制台输出

以下示例显示了内核控制台的输出结果。消息标识带有 UEFI 安全引导相关源的密钥。这包括 UEFI 安全引导 **db**、嵌入的 **shim** 和 MOK 列表。

```
# dmesg | grep -E 'integrity.*cert'
[1.512966] integrity: Loading X.509 certificate: UEFI:db
[1.513027] integrity: Loaded X.509 cert 'Microsoft Windows Production PCA 2011: a929023...
[1.513028] integrity: Loading X.509 certificate: UEFI:db
[1.513057] integrity: Loaded X.509 cert 'Microsoft Corporation UEFI CA 2011: 13adbf4309...
[1.513298] integrity: Loading X.509 certificate: UEFI:MokListRT (MOKvar table)
[1.513549] integrity: Loaded X.509 cert 'Red Hat Secure Boot CA 5: cc6fa5e72868ba494e93...
```

如需更多信息，请参阅系统中的 **keyctl (1)** 和 **dmesg (1)** 手册页。

21.8. 通过在 MOK 列表中添加公钥在目标系统中注册公钥

您必须在系统上验证您的公钥以进行内核或内核模块访问，并将其注册到目标系统的平台密钥环(**.platform**)中。当 RHEL 10 在启用了安全引导机制的基于 UEFI 的系统上引导时，内核会从 **db** key 数据库导入公钥，并从 **dbx** 数据库排除撤销的密钥。

Machine Owner Key (MOK)工具允许扩展 UEFI 安全引导密钥数据库。当在启用了安全引导机制的 UEFI 系统上引导 RHEL 10 时，MOK 列表上的密钥与来自安全引导数据库的密钥一起被添加到平台密钥环(**.platform**)中。MOK 密钥的列表以同样的方式被安全且持久地存储，但它是独立于安全引导数据库的工具。

MOK 工具被 **shim**、**MokManager**、**GRUB** 和为基于 UEFI 的系统启用安全密钥管理和身份验证的 **mokutil** 工具支持。

注意

要在您的系统上获得内核模块的身份验证服务，请考虑请求您的系统厂商将公钥合并到其工厂固件镜像中的 UEFI 安全引导密钥数据库中。

先决条件

- 您已生成了一个公钥和私钥对，并了解公钥的有效日期。详情请参阅生成公钥和私钥对。

步骤

1. 将您的公钥导出到 **sb_cert.cer** 文件中：

   –

```
# certutil -d /etc/pki/pesign \
        -n 'Custom Secure Boot key' \
        -Lr \
        > sb_cert.cer
```

2. 将您的公钥导入到 MOK 列表中：

```
# mokutil --import sb_cert.cer
```

3. 输入此 MOK 注册请求的新密码。

4. 重启机器。
   shim 引导装载程序会注意到待处理的 MOK 密钥注册请求，并启动 **MokManager.efi**，以使您从 UEFI 控制台完成注册。

5. 选择 **Enroll MOK**，在提示时输入之前与此请求关联的密码，并确认注册。
   您的公钥已添加到 MOK 列表中，这是永久的。

   密钥位于 MOK 列表中后，它将会在启用 UEFI 安全引导时自动将其传播到此列表上的 **.platform** 密钥环中。

21.9. 使用私钥签名内核

如果启用了 UEFI 安全引导机制，您可以通过载入签名的内核在系统上获得增强的安全好处。

先决条件

- 您已生成了一个公钥和私钥对，并了解公钥的有效日期。详情请参阅生成公钥和私钥对。
- 您已在目标系统上注册了公钥。详情请查看在 MOK 列表中添加公钥在目标系统中注册公钥。
- 您有一个可用于签名的 ELF 格式的内核镜像。

步骤

- 在 x64 构架上：

  a. 创建一个签名的镜像：

  ```
  # pesign --certificate 'Custom Secure Boot key' \
          --in vmlinuz-version \
          --sign \
          --out vmlinuz-version.signed
  ```

  使用 **vmlinuz** 文件的版本后缀替换 ***version***，使用您之前选择的名称替换 ***Custom Secure Boot key***。

  b. 可选：检查签名：

  ```
  # pesign --show-signature \
          --in vmlinuz-version.signed
  ```

  c. 使用签名镜像覆盖未签名的镜像：

```
# mv vmlinuz-version.signed vmlinuz-version
```

- 在 64 位 ARM 架构中：

  a. 解压缩 **vmlinuz** 文件：

  ```
  # zcat vmlinuz-version > vmlinux-version
  ```

  b. 创建一个签名的镜像：

  ```
  # pesign --certificate 'Custom Secure Boot key' \
        --in vmlinux-version \
        --sign \
        --out vmlinux-version.signed
  ```

  c. 可选：检查签名：

  ```
  # pesign --show-signature \
        --in vmlinux-version.signed
  ```

  d. 压缩 **vmlinux** 文件：

  ```
  # gzip --to-stdout vmlinux-version.signed > vmlinuz-version
  ```

  e. 删除未压缩的 **vmlinux** 文件：

  ```
  # rm vmlinux-version*
  ```

21.10. 使用私钥签名 GRUB 构建

在启用了 UEFI 安全引导机制的系统上，您可以使用自定义的现有私钥为 GRUB 构建签名。如果您使用自定义 GRUB 构建，或者已经从系统中删除 Microsoft 信任锚，则您必须执行此操作。

先决条件

- 您已生成了一个公钥和私钥对，并了解公钥的有效日期。详情请参阅生成公钥和私钥对。
- 您已在目标系统上注册了公钥。详情请查看在 MOK 列表中添加公钥在目标系统中注册公钥。
- 您有一个可用于签名的 GRUB EFI 二进制文件。

流程

- 在 x64 构架上：

  a. 创建一个签名的 GRUB EFI 二进制文件：

  ```
  # pesign --in /boot/efi/EFI/redhat/grubx64.efi \
        --out /boot/efi/EFI/redhat/grubx64.efi.signed \
        --certificate 'Custom Secure Boot key' \
        --sign
  ```

  将 ***Custom Secure Boot key*** 替换为您之前选择的名称。

b. 可选：检查签名：

```
# pesign --in /boot/efi/EFI/redhat/grubx64.efi.signed \
        --show-signature
```

c. 使用签名的二进制文件覆盖未签名的二进制文件：

```
# mv /boot/efi/EFI/redhat/grubx64.efi.signed \
    /boot/efi/EFI/redhat/grubx64.efi
```

> **警告**
>
> 当覆盖 grub 二进制文件时，您的系统可能无法正常引导，您可能需要从系统镜像重新安装 grub。

- 在 64 位 ARM 架构中：

a. 创建一个签名的 GRUB EFI 二进制文件：

```
# pesign --in /boot/efi/EFI/redhat/grubaa64.efi \
        --out /boot/efi/EFI/redhat/grubaa64.efi.signed \
        --certificate 'Custom Secure Boot key' \
        --sign
```

将 ***Custom Secure Boot key*** 替换为您之前选择的名称。

b. 可选：检查签名：

```
# pesign --in /boot/efi/EFI/redhat/grubaa64.efi.signed \
        --show-signature
```

c. 使用签名的二进制文件覆盖未签名的二进制文件：

```
# mv /boot/efi/EFI/redhat/grubaa64.efi.signed \
    /boot/efi/EFI/redhat/grubaa64.efi
```

21.11. 使用私钥签名内核模块

如果启用了 UEFI 安全引导机制，您可以通过加载签名的内核模块来提高系统的安全性。

在禁用了 UEFI 安全引导的系统上或非 UEFI 系统上，您签名的内核模块也是可以加载的。因此，您不需要提供内核模块的签名的和未签名的版本。

先决条件

- 您已生成了一个公钥和私钥对，并了解公钥的有效日期。详情请参阅生成公钥和私钥对。
- 您已在目标系统上注册了公钥。详情请查看在 MOK 列表中添加公钥在目标系统中注册公钥。

- 您有一个可以签注的 ELF 镜像格式的内核模块。

流程

1. 将您的公钥导出到 **sb_cert.cer** 文件中：

```
# certutil -d /etc/pki/pesign \
          -n 'Custom Secure Boot key' \
          -Lr \
          > sb_cert.cer
```

2. 从 NSS 数据库中提取密钥作为 PKCS #12 文件：

```
# pk12util -o sb_cert.p12 \
          -n 'Custom Secure Boot key' \
          -d /etc/pki/pesign
```

3. 当上一个命令提示时，请输入新的加密私钥的密码。

4. 导出未加密的私钥：

```
# openssl pkcs12 \
        -in sb_cert.p12 \
        -out sb_cert.priv \
        -nocerts \
        -noenc
```

重要

确保未加密的私钥的安全。

5. 为内核模块签名。以下命令将签名直接附加到内核模块文件中的 ELF 镜像中：

```
# /usr/src/kernels/$(uname -r)/scripts/sign-file \
          sha256 \
          sb_cert.priv \
          sb_cert.cer \
          my_module.ko
```

您的内核模块现在可以被加载。

重要

在 Red Hat Enterprise Linux 10 中，密钥对的有效日期很重要。这个密钥没有过期，但必须在其签名密钥的有效周期内对内核模块进行签名。**sign-file** 实用程序不会提醒您这样做。

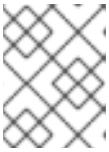
例如，一个只在 2021 年有效的密钥可用于验证在 2021 年使用该密钥签名的内核模块。但是，用户无法使用该密钥在 2022 年签发内核模块。

验证

1. 显示关于内核模块签名的信息：

```
# modinfo my_module.ko | grep signer
  signer:         Your Name Key
```

检查签名中是否列出了您在生成过程中输入的名称。



注意

附加的签名不包含在 ELF 镜像部分，不是 ELF 镜像的一个正式部分。因此，**readelf** 等工具无法在内核模块上显示签名。

2. 载入模块：

```
# insmod my_module.ko
```

3. 删除（未加载）模块：

```
# modprobe -r my_module.ko
```

其他资源

- 显示内核模块信息

21.12. 载入经过签名的内核模块

在平台密钥环中注册您的公钥(**.platform**)和 MOK 列表，并使用您的私钥签名内核模块后，您可以使用 **modprobe** 命令加载它们。

先决条件

- 您已生成了公钥和私钥对。详情请参阅生成公钥和私钥对。
- 您已将公钥注册到平台密钥环中。详情请查看在 MOK 列表中添加公钥在目标系统中注册公钥。
- 您用私钥签名了一个内核模块。详情请查看使用私钥签名内核模块。
- 安装 **kernel-modules-extra** 软件包，它会创建 **/lib/modules/$(uname -r)/extra/** 目录：

```
# dnf -y install kernel-modules-extra
```

流程

1. 验证您的公钥是否在平台密钥环中：

```
# keyctl list %:.platform
```

2. 将内核模块复制到您想要的内核的 **extra/** 目录中：

```
# cp my_module.ko /lib/modules/$(uname -r)/extra/
```

3. 更新模块依赖项列表：

```
# depmod -a
```

4. 载入内核模块：

```
# modprobe -v my_module
```

5. 可选：要在引导时载入模块，请将其添加到 **/etc/modules-loaded.d/*my_module*.conf** 文件中：

```
# echo "my_module" > /etc/modules-load.d/my_module.conf
```

验证

- 验证模块是否被成功载入：

```
# lsmod | grep my_module
```

其他资源

- 管理内核模块

第 22 章 更新安全引导撤销列表

您可以更新系统上的 UEFI 安全引导撤销列表，以便安全引导使用已知安全问题识别软件，并防止它破坏引导过程。

22.1. 安全引导撤销列表

UEFI 安全引导撤销列表或安全引导禁止签名数据库(**dbx**)是一个列表，其识别安全引导不再允许运行的软件。

当在与安全引导(Secure Boot)接口的软件中发现安全问题或稳定性问题时，比如在 GRUB 引导装载程序中，撤销列表会存储其哈希签名。带有此类可识别签名的软件在引导过程中无法运行，系统引导无法防止损害系统。

例如，一个特定版本的 GRUB 可能会包含允许攻击者绕过安全引导机制的安全问题。当找到问题时，撤销列表会添加包含这个问题的所有 GRUB 版本的哈希签名。因此，只有安全的 GRUB 版本才可以在系统上引导。

撤销列表需要常规更新以识别新发现的问题。更新撤销列表时，请确保使用一个安全更新方法，它不会导致当前安装的系统不再引导。

22.2. 应用一个在线撤销列表更新

您可以更新系统上的安全引导撤销列表，以便安全引导防止已知的安全问题。这个过程是安全的，并确保更新不会阻止系统引导。

先决条件

- 安全引导已在您的系统上启用。
- 您的系统已连接到互联网，以便获取更新。

流程

1. 确定撤销列表的当前版本：

```
# *fwupdmgr get-devices*
```

   请参阅 **UEFI dbx** 下的 **Current version** 字段。

2. 启用 LVFS Revocation List 存储库：

```
# *fwupdmgr enable-remote lvfs*
```

3. 刷新存储库元数据：

```
# *fwupdmgr refresh*
```

4. 应用撤销列表更新：
   - 在命令行上：

```
# *fwupdmgr update*
```

- 在图形界面中：
  i. 打开 **Software** 应用程序
  ii. 进入 **Updates** 选项卡。
  iii. 查找 **Secure Boot dbx Configuration Update**条目。
  iv. 单击 **Update**。

5. 在更新结束时，**fwupdmgr** 或 **Software** 会要求您重启系统。确认重启。

验证

- 重启后，再次检查撤销列表的当前版本：

```
# *fwupdmgr get-devices*
```

22.3. 应用一个离线撤销列表更新

在没有互联网连接的系统上，您可以从 Red Hat Enterprise Linux 更新安全引导撤销列表，以便安全引导防止已知的安全问题。这个过程是安全的，并确保更新不会阻止系统引导。

流程

1. 识别撤销列表的当前版本：

```
# *fwupdmgr get-devices*
```

请参阅 **UEFI dbx** 下的 **Current version** 字段。

2. 列出 RHEL 中可用的更新：

```
# *ls /usr/share/dbxtool/*
```

3. 为您的构架选择最新的更新文件。文件名采用以下格式：

```
DBXUpdate-date-architecture.cab
```

4. 安装所选更新文件：

```
# fwupdmgr install /usr/share/dbxtool/DBXUpdate-date-architecture.cab
```

5. 在更新结束时，**fwupdmgr** 会要求您重启系统。确认重启。

验证

- 重启后，再次检查撤销列表的当前版本：

```
# *fwupdmgr get-devices*
```

第 23 章 使用内核完整性子系统提高安全性

您可以使用内核完整性子系统的组件来提高系统安全性。了解有关相关组件及其配置的更多信息。

23.1. 内核完整性子系统

完整性子系统通过检测文件篡改并根据加载的策略拒绝访问来保护系统的完整性。它还收集访问日志，以便远程方可以通过远程测试来验证系统的完整性。内核完整性子系统包括 Integrity 测量架构(IMA)和扩展验证模块(EVM)。

完整性测量架构 (IMA)

IMA 维护文件内容的完整性。它包括您可以通过 IMA 策略启用的三个功能：

- **IMA-Measurement** ：收集文件内容散列或签名，并将测量保存在内核中。如果有 TPM 可用，每个测量都会扩展 TPM PCR，它启用了认证引用的远程测试。
- **IMA-Appraisal**: 通过将计算的文件哈希与已知良好的参考值进行比较，或者验证存储在 security.ima 属性中的签名来验证文件的完整性。如果验证失败，系统会拒绝访问。
- **IMA-Audit** ：将计算的文件内容散列或签名存储在系统审计日志中。

扩展验证模块 (EVM)

EVM 保护文件元数据，包括与系统安全性相关的扩展属性，如 **security.ima** 和 **security.selinux**。EVM 在 **security.evm** 中存储这些安全属性的引用哈希或 HMAC，并使用它来检测文件元数据是否已被恶意更改。

其他资源

- 安全强化
- 使用 SELinux

23.2. 通过基于 IMA 的 APPRAISAL 启用内核的运行时完整性监控

从 RHEL 9 开始，所有软件包文件都是每个文件签名的，用户可以通过启用基于签名的 IMA appraisal 来确保只访问授权的软件包文件。

启用基于签名的 IMA appraisal ：

```
ima-setup --policy=/usr/share/ima/policies/01-appraise-executable-and-lib-signatures
```

这个命令：

- 将软件包文件签名存储在 **security.ima** 中，适用于所有安装的软件包。
- 包括 **dracut** integrity 模块，以将 IMA 代码签名密钥加载到内核。
- 将策略复制到 **/etc/ima/ima-policy**，以便 systemd 在引导时加载它。

验证

- **ip** 命令可以被成功执行。

- 如果 **ip** 复制到 **/tmp**，默认情况下，它会丢失其 **security.ima**，因此 **ip** 命令不会被执行。

```
# cp /usr/sbin/ip /tmp
# /tmp/ip
-bash: /tmp/ip: Permission denied
# /tmp/ip doesn't have security.ima
# getfattr -m security.ima -d /tmp/ip
# whereas /usr/sbin/ip has
# getfattr -m security.ima /usr/sbin/ip
# file: usr/sbin/ip
security.ima=0sAwIE0zIESQBnMGUCMQCLXZ7ukyDcguLgPYwzXU16dcVrmlHxOta7vm7EUf
X07Nf0xnP1MyE//AZaqeNIKBoCMFHNDOuA4uNvS+8OOAy7YEn8oathfsF2wsDSZi+NAoum
C6RFqIB912zkRKxraSX8sA==
```

如果示例策略 **01-appraise-executable-and-lib-signatures** 没有满足您的要求，您可以创建并使用自定义策略。

23.3. 使用 IMA 测量启用远程测试

您可以使用 IMA 测量启用远程测试，以验证系统的完整性。要通过 **Keylime** 等工具使用远程认证，您必须启用 IMA-Measurement。签名的测量策略位于 **/usr/share/ima/policies/02-keylime-remote-attestation** 中。部署并运行符合您的要求的示例策略。

先决条件

- 签名的测量策略位于 **/usr/share/ima/policies/02-keylime-remote-attestation** 中。

流程

1. 部署策略：

```
# cp --preserve=xattr /usr/share/ima/policies/02-keylime-remote-attestation /etc/ima/ima-policy
```

2. 加载策略：

```
# echo /etc/ima/ima-policy > /sys/kernel/security/integrity/ima/policy
```

如果示例策略没有满足您的要求，或者您要确保只加载签名的 IMA 策略，请参阅为 UEFI 系统部署自定义签名的 IMA 策略。

验证

- 验证策略是否已载入：

```
# cat /sys/kernel/security/integrity/ima/policy
```

其他资源

- 使用 Keylime 确保系统完整性

第 24 章 扩展、自定义并故障排除内核完整性子系统

扩展、自定义内核完整性子系统并对内核完整性子系统进行故障排除，以支持各种安全要求和操作环境。

24.1. 为 IMA APPRAISAL 生成良好的参考值

在部署包含 IMA-appraisal 规则的 IMA 策略前，请确保所有由这些规则管理的文件都有存储在 **security.ima** 扩展属性中的有效引用值。如果缺少这些引用值，IMA 可能会阻止系统正确引导或拒绝访问文件。

```
# ima-appraise-file </path/to/file>
```

24.1.1. 添加 IMA 签名作为不可变文件的良好引用

使用 IMA 签名作为不可变文件的可信引用值来支持完整性验证。这种方法有助于确保仅访问具有有效签名的文件，从而增强系统安全性和合规性。

先决条件

- 您已创建了包含 IMA-appraisal 规则的 IMA 策略。

流程

1. 安装 **rpm-plugin-ima** ：

```
$ sudo dnf install rpm-plugin-ima -yq
```

这样可确保软件包文件在软件包安装、重新安装或升级过程中自动存储在 **security.xattr** 中。

2. 重新安装所有软件包：

```
$ sudo dnf reinstall "*" -y
```

这样可确保为所有软件包更新了 **security.xattr** 扩展属性。

3. 启用 dracut 完整性模块，以便在引导时自动在 **/etc/keys/ima** 载入中的官方 IMA code-signing 密钥：

```
$ sudo dracut -f
```

验证

- 验证签名是否已正确存储在 **security.ima** 扩展属性中：

```
$ # evmctl ima_verify -k /etc/keys/ima/redhatimarelease-10.der /usr/lib/systemd/systemd
keyid d3320449 (from /etc/keys/ima/redhatimarelease-10.der)
key 1: d3320449 /etc/keys/ima/redhatimarelease-10.der
/usr/lib/systemd/systemd: verification is OK

$ # evmctl ima_verify -k /etc/keys/ima/redhatimarelease-10.der /bin/bash
keyid d3320449 (from /etc/keys/ima/redhatimarelease-10.der)
```

```
key 1: d3320449 /etc/keys/ima/redhatimarelease-10.der
/bin/bash: verification is OK
...
```

24.1.2. 为可变文件生成良好的参考值

要保持可能随时间变化的文件的完整性，请根据需要生成和更新参考值。这样可确保系统准确验证可变文件的真实性，并防止未经授权的修改。

先决条件

- 您在系统上具有 root 权限。
- 您已创建了包含 IMA-appraisal 规则的 IMA 策略。
- 您已为 IMA appraisal 生成了很好的参考值。
- 禁用安全引导。

流程

1. 可选：启用您选择的 IMA-appraisal 策略，或者如果您只使用自定义策略，请跳过这一步。使用内置 **ima_policy=appraise_tcb** 作为示例：

```
# grubby --update-kernel=/boot/vmlinuz-$(uname -r) --args="ima_policy=appraise_tcb"
```

   - 另外，**s390x** 系统：

```
# zipl
```

2. 通过添加 **ima_appraise=fix** 内核命令行参数来启用 IMA-appraisal 修复模式：

```
# grubby --update-kernel=/boot/vmlinuz-$(uname -r) --args="ima_appraise=fix"
```

   - 另外，**s390x** 系统：

```
# zipl
```

3. 重启系统：

```
# reboot
```

4. 可选：载入您的自定义 IMA 策略：

```
# echo <path_to_your_custom_ima_policy> > /sys/kernel/security/ima/policy
```

5. 重新标记整个系统：

```
# find / -fstype xfs -type f -uid 0 -exec head -c 0 '{}' \;
```

6. 通过删除 **ima_appraise=fix** 内核命令行参数来关闭 IMA-appraisal 修复模式：

```
# grubby --update-kernel=/boot/vmlinuz-$(uname -r) --remove-args="ima_appraise=fix"
```

- 另外，**s390x** 系统：

```
# zipl
```

7. 如果禁用了安全引导，则启用安全引导。

其他资源

- 为单一引导条目更改内核命令行参数

24.2. 编写自定义 IMA 策略

如果您使用内核命令行参数启用的内置 IMA 策略，如 **ima_policy=tcb** 或 **ima_policy=critical_data**，或者 **/usr/share/ima/policies/** 中的示例策略不满足您的要求，您可以创建自定义 IMA 策略规则。当 systemd 从 **/etc/ima/ima-policy** 加载策略时，它会替换内置的 IMA 策略。

> **警告**
>
> 定义 IMA 策略后，如果策略在部署前包含 IMA-appraisal 规则，请生成很好的引用值。如果您的策略不包含 IMA-appraisal 规则，您可以通过运行 **echo /PATH-TO-YOUR-DRAFT-IMA-POLICY > /sys/kernel/security/integrity/ima/policy** 来验证策略。这种方法有助于防止系统引导失败。
>
> 请参阅 为 IMA appraisal 生成良好的参考值。

流程

- 检查规则格式和示例策略。
  IMA 策略规则使用格式 **操作 [condition ...]** 指定在特定条件下触发的操作。例如，**/usr/share/ima/policies/01-appraise-executable-and-lib-signatures** 中的示例策略包括以下规则：

```
# Skip some unsupported filesystems
# For a list of these filesystems, see
# https://www.kernel.org/doc/Documentation/ABI/testing/ima_policy
# PROC_SUPER_MAGIC
dont_appraise fsmagic=0x9fa0
...
appraise func=BPRM_CHECK appraise_type=imasig
```

  第一个规则 **dont_appraise fsmagic=0x9fa0** 指示 IMA 跳过 **PROC_SUPER_MAGIC** 文件系统中的文件。最后一条规则是 **appraise func=BPRM_CHECK appraise_type=imasig**，在执行文件时强制执行签名验证。

其他资源

- ABI 测试符号 - Linux 内核文档

24.3. 使用 OPENSSL 创建自定义 IMA 密钥

您可以使用 **OpenSSL** 为您的数字证书生成一个 CSR，以保护您的代码。

内核搜索代码签名密钥的 **.ima** keyring，以验证 IMA 签名。在向 **.ima** keyring 中添加代码签名密钥前，您需要确保 IMA CA 密钥在 **.builtin_trusted_keys** 或 **.secondary_trusted_keys** keyring 中签名了这个密钥。

先决条件

- 自定义 IMA CA 密钥有以下扩展：
  - 具有 CA 布尔值断言的基本约束扩展。
  - 带有 **keyCertSign** 位断言，但 **没有 digitalSignature** 断言的 **KeyUsage** 扩展。
- 自定义 IMA 代码签名密钥符合以下条件：
  - IMA CA 密钥签名了这个自定义 IMA 代码签名密钥。
  - 自定义密钥包括 **subjectKeyIdentifier** 扩展。
- **x86_64** 或 **aarch64** 系统或 **ppc64le** 系统上的 PowerVM 安全引导启用了 UEFI 安全引导。

流程

1. 要生成一个自定义 IMA CA 密钥对，请运行：

```
# openssl req -new -x509 -utf8 -sha256 -days 3650 -batch -config ima_ca.conf -outform
DER -out custom_ima_ca.der -keyout custom_ima_ca.priv
```

2. 可选：要检查 **ima_ca.conf** 文件的内容，请运行：

```
# cat ima_ca.conf
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
prompt = no
string_mask = utf8only
x509_extensions = ca

[ req_distinguished_name ]
O = YOUR_ORG
CN =  YOUR_COMMON_NAME IMA CA
emailAddress = YOUR_EMAIL

[ ca ]
basicConstraints=critical,CA:TRUE
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid:always,issuer
keyUsage=critical,keyCertSign,cRLSign
```

3. 要为 IMA 代码签名密钥生成一个私钥和一个签名请求的证书(CSR)，请运行：

```
# openssl req -new -utf8 -sha256 -days 365 -batch -config ima.conf -out
custom_ima.csr -keyout custom_ima.priv
```

4. 可选：要检查 **ima.conf** 文件的内容，请运行：

```
# cat ima.conf
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
prompt = no
string_mask = utf8only
x509_extensions = code_signing

[ req_distinguished_name ]
O = YOUR_ORG
CN = YOUR_COMMON_NAME IMA signing key
emailAddress = YOUR_EMAIL

[ code_signing ]
basicConstraints=critical,CA:FALSE
keyUsage=digitalSignature
subjectKeyIdentifier=hash
authorityKeyIdentifier=keyid:always,issuer
```

5. 使用 IMA CA 私钥签名 CSR，来创建 IMA 代码签名证书：

```
# openssl x509 -req -in custom_ima.csr -days 365 -extfile ima.conf -extensions
code_signing -CA custom_ima_ca.der -CAkey custom_ima_ca.priv -CAcreateserial -
outform DER -out ima.der
```

24.4. 加载由自定义 IMA 密钥签名的 IMA 策略

要保持您的系统完整性并满足您的机构的安全要求，您可以加载使用您自己的自定义 IMA 密钥签名的 IMA 策略。这种方法可确保在系统启动或运行时仅应用可信的经过身份验证的策略。

注意

此流程只适用于启用了 UEFI 安全引导的 **x86_64** 和 **aarch64** 系统，以及运行 PowerVM 安全引导的 **ppc64le** 系统。

先决条件

- 在您的系统中必须具有 root 权限。
- 为 Red Hat Enterprise Linux 启用 UEFI 安全引导，或者内核使用 **ima_policy=secure_boot** 参数引导，以确保只能加载签名的 IMA 策略。
- 自定义 IMA CA 密钥已添加到 MOK 列表中。如需更多信息，请参阅在 MOK 列表中添加公钥在目标系统上注册公钥。
- 内核版本为 5.14 或更高版本。
- 为 IMA 策略生成了很好的参考值。如需更多信息，请参阅 为 IMA appraisal 生成良好的参考值。

流程

1. 将自定义 IMA 代码签名密钥添加到 **.ima** 密钥环中：

```
# keyctl padd asymmetric <KEY_SUBJECT> %:.ima <
<PATH_TO_YOUR_CUSTOM_IMA_KEY>
```

2. 准备 IMA 策略，并使用您的自定义 IMA 代码签名密钥对其进行签名：

```
# evmctl ima_sign <PATH_TO_YOUR_CUSTOM_IMA_POLICY> -k
<PATH_TO_YOUR_CUSTOM_IMA_KEY>
```

3. 加载签名的 IMA 策略：

```
# echo <PATH_TO_YOUR_CUSTOM_SIGNED_IMA_POLICY> >
/sys/kernel/security/ima/policy
# echo $?
0
```

0

表示 IMA 策略已被成功加载。如果命令返回非零值，则 IMA 策略没有成功加载。

> **警告**
>
> 不要跳过这一步。如果这样做，您的系统可能无法引导，您需要恢复您的系统。

如果 IMA 策略无法加载，请重复步骤 2 和 3 来修复问题。

4. 将签名的 IMA 策略复制到 **/etc/ima/ima-policy** 中，以便在引导时自动启用它：

```
# cp --preserve=xattr <PATH_TO_YOUR_CUSTOM_IMA_POLICY> /etc/ima/ima-policy
```

5. 使用 **dracut** 完整性模块，自动将自定义 IMA 代码签名密钥添加到引导时 **.ima** 密钥环中：

```
# cp <PATH_TO_YOUR_CUSTOM_IMA_KEY> /etc/keys/ima/
# cp --preserve=xattr /usr/share/ima/dracut-98-integrity.conf /etc/dracut.conf.d/98-
integrity.conf
# dracut -f
```

- 另外，**s390x** 系统：

```
# zipl
```

验证

- 验证 IMA 策略是否已成功载入：

```
# cat /sys/kernel/security/ima/policy
```

输出应包含自定义 IMA 策略中的规则。

24.5. 对 SYSTEMD 无法加载 IMA 策略的故障排除

如果 systemd 没有加载 **/etc/ima/ima-policy**，系统挂起并显示错误 **systemd[1]: Freezing execution**。

```
[    5.829882] ima: policy update failed
[    5.830094] ima: signed policy file (specified as an absolute pathname) required
[!!!!!!] Failed to load IMA policy.
…
[    5.859994] systemd[1]: Freezing execution.
```

您可以使用三种方法来恢复您的系统。

24.5.1. 关闭安全引导

如果因为未签名策略而无法加载策略，您可能会看到类似以下示例的错误。

```
[    5.661906] ima: policy update failed
[    5.662290] ima: signed policy file (specified as an absolute pathname) required
[    5.662496] systemd[1]: Failed to load the IMA custom policy file /etc/ima/ima-policy1: Permission denied
[    5.662663] ima: policy update failed
[    5.662856] audit: type=1800 audit(1744968172.925:7): pid=1 uid=0 auid=4294967295 ses=4294967295 subj=system_u:system_r:init_t:s0 op=appraise_data cause=IMA-signature-required comm="systemd" name="/etc/ima/ima-policy" dev="vda3" ino=25679834 res=0 errno=0
[    5.663205] audit: type=1802 audit(1744968172.925:8): pid=1 uid=0 auid=4294967295 ses=4294967295 subj=system_u:system_r:init_t:s0 op=policy_update cause=failed comm="systemd" res=0 errno=0
[!!!!!!] Failed to load IMA policy.
```

作为临时解决方案，您可以临时关闭安全引导，并遵循 为 UEFI 系统部署自定义签名的 IMA 策略 来解决这个问题。

24.5.2. 使用 **init=/bin/bash** 内核参数引导系统

要使用 **init=/bin/bash** 内核参数引导系统，您可以使用以下步骤：

1. 修改 bootloader 条目并添加 **init=/bin/bash** 内核参数。
2. 访问 shell 后，使用写权限重新挂载系统：

   ```
   # mount -o remount,rw /
   ```

3. 将 **/etc/ima/ima-policy** 重命名为 **/etc/ima/ima-policy.bak** ：

   ```
   # mv /etc/ima/ima-policy /etc/ima/ima-policy.bak
   ```

4. 重启系统：

```
# echo 1 > /proc/sys/kernel/sysrq
# printf "s\nb" > /proc/sysrq-trigger
```

5. 解决 **/etc/ima/ima-policy.bak** 中的任何问题，并验证是否可以载入该策略：

```
# echo /etc/ima/ima-policy.bak >> /sys/kernel/security/integrity/ima/policy
```

6. 将 **/etc/ima/ima-policy.bak** 重命名为 **/etc/ima/ima-policy**:

```
# mv /etc/ima/ima-policy.bak /etc/ima/ima-policy
```

24.5.3. 使用 initcall_blacklist=init_ima 内核参数引导系统

如果系统挂起并显示错误 **systemd[1]: Freezing execution**，您可以使用 **initcall_blacklist=init_ima** 内核参数引导系统，以禁用 IMA 策略。

1. 修改引导装载程序条目并添加 **initcall_blacklist=init_ima** 内核参数。
2. 将 **/etc/ima/ima-policy** 重命名为 **/etc/ima/ima-policy.bak** ：

```
# mv /etc/ima/ima-policy /etc/ima/ima-policy.bak
```

3. 重启系统：

```
# systemctl reboot
```

4. 解决 **/etc/ima/ima-policy.bak** 中的任何问题，并验证是否可以载入该策略：

```
# echo /etc/ima/ima-policy.bak >> /sys/kernel/security/integrity/ima/policy
```

5. 将 **/etc/ima/ima-policy.bak** 重命名为 **/etc/ima/ima-policy**:

```
# mv /etc/ima/ima-policy.bak /etc/ima/ima-policy
```

24.6. 签名自定义构建的软件包

为保持系统的完整性，在部署前务必要为自定义构建的软件包签名。使用 **rpm-sign** 工具和 IMA 代码签名密钥，您可以签署您的自定义构建软件包。

先决条件

- 您在系统中具有 root 权限。
- 您有一个要签名的自定义构建软件包。
- 您有 IMA 代码签名密钥。
- 已安装 **rpm-sign** 工具。
- 创建自定义 IMA 密钥。请参阅使用 OpenSSL 创建自定义 IMA 密钥。

流程

流程

1. 使用 **rpmsign -signfiles** 为软件包文件签名：

```
# rpmsign --define "gpg_name _<GPG_KEY_NAME>" --addsign --signfiles --fskpass --
fskpath=<PATH_TO_YOUR_PRIVATE_IMA_CODE_SIGNING_KEY>
<PATH_TO_YOUR_RPM>
```

--define "*gpg_name _<GPG_KEY_NAME>*"

GPG 密钥为软件包签名，IMA 代码签名密钥签署软件包中的每个文件。

--addsign

在软件包中添加签名。

--signfiles

对软件包中的每个文件进行签名。

--fskpass

避免重复输入 IMA 代码签名密钥的密码。

--fskpath

指定 IMA 代码签名密钥的路径。

验证

- 要验证软件包是否已签名，您可以使用以下命令：

```
# rpm -q --queryformat "[%{FILENAMES} %{FILESIGNATURES}\n]
<PATH_TO_YOUR_RPM>"
/usr/bin/YOUR_BIN 030204...
/usr/lib/YOUR_LIB.so 030204...
...
```

其他资源

- 打包软件

24.7. 在 IMA 和 FAPOLICYD 间选择

IMA 和 **fapolicyd** 是两个不同的工具，用于强制文件的完整性。IMA 是一个内核模块，它通过在引导时验证文件的完整性来强制实施文件完整性。**fapolicyd** 是一个守护进程，它通过在运行时验证文件的完整性来强制实施文件完整性。

以下列表可帮助您确定哪个工具满足您的要求：

- IMA 验证数字签名以确保完整性，而 **fapolicyd** 目前只支持基于哈希的验证。
- IMA 在内核空间中运行，**fapolicyd** 在用户空间中运行。
- **fapolicyd** 通过检查文件大小来支持基本的完整性验证，也可以验证存储在 **security.ima** 中的引用哈希值。
- IMA 和 **fapolicyd** 使用不同的策略语法。例如，**fapolicyd** 支持基于路径的策略，但 IMA 不支持。

第 25 章 使用 SYSTEMD 管理应用程序使用的资源

Red Hat Enterprise Linux 10 通过将 **cgroup** 层次结构的系统与 **systemd** 单元树绑定，将资源管理设置从进程级别移到应用程序级别。因此，您可以使用 **systemctl** 命令或通过修改 **systemd** 单元文件来管理系统资源。

要做到这一点，**systemd** 从单元文件或者直接通过 **systemctl** 命令获取各种配置选项。然后，**systemd** 使用 Linux 内核系统调用及 **cgroups**和**namespaces** 这样的功能将这些选项应用到特定的进程组中。

注意

您可以在以下手册页中查看 **systemd** 的完整配置选项：

- **systemd.resource-control (5)**
- **systemd.exec (5)**

25.1. 资源管理中的 SYSTEMD 角色

systemd 的核心功能是服务管理和监管。**systemd** 系统和服务管理器：

- 确保受管服务在正确时间启动，并在启动过程中按正确的顺序启动。
- 确保受管服务平稳运行，以最优地使用底层硬件平台。
- 提供定义资源管理策略的能力。
- 提供调整各种选项的能力，这可以提高服务的性能。

重要

通常，建议您使用 **systemd** 来控制系统资源的使用。除非是特殊情况，否则不得手动配置 **cgroups** 虚拟文件系统。

25.2. 系统源的分发模型

要修改系统资源的发布，您可以应用一个或多个以下分发模型：

Weights（权重）

您可以通过增加所有子组的权重并为每个子组群分配资源，使其与总和总的比例匹配。
例如，如果您有 10 个 cgroups，则每个权重值为 100，sum 为 1000。每个 cgroup 会收到十分之一的资源。

权重通常用于分发无状态资源。例如， *CPUWeight=* 选项是此资源分布模型的实现。

Limits

cgroup 可以最多消耗配置的资源量。子组限值总和不能超过父 cgroup 的限值。因此，可以过量使用此模型中的资源。
例如， *MemoryMax=* 选项是此资源分发模型的实现。

Protections（保护）

您可以为 cgroup 设置受保护的资源量。如果资源使用量低于保护边界，内核将尝试不以竞争同一资源的 cgroup 替代其他 cgroup。可以过量使用。
例如，*MemoryLow=* 选项是此资源分发模型的实现。

Allocations（分配）

独占分配有限资源的绝对数量。不能过量使用。Linux 中这种资源类型的一个示例就是实时预算。

单元文件选项

资源控制配置的设置。
例如，您可以使用 *CPUAccounting=* 或 *CPUQuota=* 等选项配置 CPU 资源。同样，您可以使用 *AllowedMemoryNodes=* 和 *IOAccounting=* 等选项配置内存或 I/O 资源。

25.3. 使用 SYSTEMD 分配系统资源

使用 systemd 分配系统资源涉及创建和管理 systemd 服务和单元。这可以被配置为在特定时间启动、停止或重新启动，或者响应某些系统事件。您可以更改服务的单元文件选项的值，或使用 **systemctl** 命令。

流程

- 使用 **systemctl** 命令。

  a. 检查为您选择的服务分配的值：

  ```
  # systemctl show --property <unit file option> <service name>
  ```

  b. 设置 CPU 时间分配策略选项的必要值：

  ```
  # systemctl set-property <service name> <unit file option>=<value>
  ```

  如需更多信息，请参阅 **systemd.resource-control (5)** 和 **systemd.exec (5)** 手册页。

验证

- 检查为您选择的服务新分配的值：

  ```
  # systemctl show --property <unit file option> <service name>
  ```

25.4. CGROUPS 的 SYSTEMD 层次结构概述

在后端，**systemd** 系统和服务管理器使用 **slice**、**scope**，以及 **service** 单元来整理和构建控制组中的进程。您可以通过创建自定义单元文件或使用 **systemctl** 命令来进一步修改此层次结构。另外，**systemd** 会在 **/sys/fs/cgroup/** 目录中自动挂载重要内核资源控制器的层次结构。

对于资源控制，您可以使用以下三种 **systemd** 单元类型：

service

systemd 根据单元配置文件启动的一个进程或一组进程，。
服务封装指定的进程，以便它们可以作为一个集启动和停止。服务使用以下方法命名：

```
<name>.service
```

影响范围

外部创建的一组进程。范围封装通过 **fork()** 函数由任意进程启动和停止的进程，然后在运行时由 **systemd** 注册。例如，用户会话、容器和虚拟机被视为范围。范围命名如下：

> *<name>*.scope

slice

一组分级组织的单元。片段组织了一个分级，其中放置范围和服务。
实际的进程包含在范围或服务中。slice 单元的每个名称对应层次结构中的位置的路径。

短划线(**-**)字符充当路径组件与 **-.slice** root 片段中片段的分隔符。在以下示例中：

> *<parent-name>*.slice

parent-name.slice 是 **parent.slice** 的子分片，它是 **-.slice** root 片段的子分片。**parent-name.slice** 可以有自己的子slice 名为 **parent-name-name2.slice**，以此类推。

service、**scope** 和 **slice** 单元直接映射到控制组层次结构中的对象。激活这些单元后，它们直接映射到从单元名称构建的控制组路径。

以下是控制组群分级的缩写示例：

```
Control group /:
-.slice
 ├─user.slice
 │ ├─user-42.slice
 │ │ ├─session-c1.scope
 │ │ │ ├─ 967 gdm-session-worker [pam/gdm-launch-environment]
 │ │ │ ├─1035 /usr/libexec/gdm-x-session gnome-session --autostart
/usr/share/gdm/greeter/autostart
 │ │ │ ├─1054 /usr/libexec/Xorg vt1 -displayfd 3 -auth /run/user/42/gdm/Xauthority -background
none -noreset -keeptty -verbose 3
 │ │ │ ├─1212 /usr/libexec/gnome-session-binary --autostart /usr/share/gdm/greeter/autostart
 │ │ │ ├─1369 /usr/bin/gnome-shell
 │ │ │ ├─1732 ibus-daemon --xim --panel disable
 │ │ │ ├─1752 /usr/libexec/ibus-dconf
 │ │ │ ├─1762 /usr/libexec/ibus-x11 --kill-daemon
 │ │ │ ├─1912 /usr/libexec/gsd-xsettings
 │ │ │ ├─1917 /usr/libexec/gsd-a11y-settings
 │ │ │ ├─1920 /usr/libexec/gsd-clipboard
…
 ├─init.scope
 │ └─1 /usr/lib/systemd/systemd --switched-root --system --deserialize 18
 └─system.slice
   ├─rngd.service
   │ └─800 /sbin/rngd -f
   ├─systemd-udevd.service
   │ └─659 /usr/lib/systemd/systemd-udevd
   ├─chronyd.service
   │ └─823 /usr/sbin/chronyd
   ├─auditd.service
   │ ├─761 /sbin/auditd
   │ └─763 /usr/sbin/sedispatch
```

```
├──accounts-daemon.service
│  └──876 /usr/libexec/accounts-daemon
├──example.service
│  ├── 929 /bin/bash /home/jdoe/example.sh
│  └──4902 sleep 1
…
```

本例显示服务和范围包含进程，并放置在不包含自身进程的片段中。

其他资源

- 使用 systemctl 管理系统服务
- 内核资源控制器简介

25.5. 列出 SYSTEMD 单元

使用 **systemd** 系统和服务管理器列出其单元。

流程

- 使用 **systemctl** 工具列出系统上所有活动的单元。终端返回一个类似于以下示例的输出：

```
# systemctl
UNIT                                    LOAD   ACTIVE SUB       DESCRIPTION
…
init.scope                              loaded active running   System and Service Manager
session-2.scope                         loaded active running   Session 2 of user jdoe
abrt-ccpp.service                       loaded active exited    Install ABRT coredump hook
abrt-oops.service                       loaded active running   ABRT kernel log watcher
abrt-vmcore.service                     loaded active exited    Harvest vmcores for ABRT
abrt-xorg.service                       loaded active running   ABRT Xorg log watcher
…
-.slice                                 loaded active active    Root Slice
machine.slice                           loaded active active    Virtual Machine and Container
Slice system-getty.slice                                        loaded active active
system-getty.slice
system-lvm2\x2dpvscan.slice             loaded active active    system-
lvm2\x2dpvscan.slice
system-sshd\x2dkeygen.slice             loaded active active    system-
sshd\x2dkeygen.slice
system-systemd\x2dhibernate\x2dresume.slice   loaded active active    system-
systemd\x2dhibernate\x2dresume>
system-user\x2druntime\x2ddir.slice     loaded active active    system-
user\x2druntime\x2ddir.slice
system.slice                            loaded active active    System Slice
user-1000.slice                         loaded active active    User Slice of UID 1000
user-42.slice                           loaded active active    User Slice of UID 42
user.slice                              loaded active active    User and Session Slice
…
```

UNIT

还反映控制组层次结构中单元位置的单元名称。与资源控制相关的单元是 *slice*、*scope* 和 *service*

LOAD

指示单元配置文件是否被正确加载。如果单元文件加载失败，字段会提供状态 *error*，而不是 *loaded*。其他单元负载状态为：*stub*、*merge* 和 *masked*。

ACTIVE

高级单元激活状态，其是 **SUB** 的一个泛论。

SUB

低级单元激活状态。可能的值的范围取决于单元类型。

DESCRIPTION

单元内容和功能的描述。

- 列出所有活跃的和不活跃的单元：

```
# systemctl --all
```

- 限制输出中的信息量：

```
# systemctl --type service,masked
```

--type 选项需要一个以逗号分隔的单元类型列表，如 *service* 和 *slice*，或者单元载入状态，如 *loaded* 和 *masked*。

如需更多信息，请参阅系统上的 **systemd.resource-control (5)** 和 **systemd.exec (5)** 手册页。

其他资源

- 使用 systemctl 管理系统服务

25.6. 查看 SYSTEMD CGROUPS 层次结构

显示控制组(**cgroup**)层次结构以及在特定 **cgroups** 中运行的进程。

流程

- 使用 **systemd-cgls** 命令显示系统上整个 **cgroups** 层次结构。

```
# systemd-cgls
Control group /:
-.slice
 ├─user.slice
 │ ├─user-42.slice
 │ │ ├─session-c1.scope
 │ │ │ ├─ 965 gdm-session-worker [pam/gdm-launch-environment]
 │ │ │ ├─1040 /usr/libexec/gdm-x-session gnome-session --autostart
/usr/share/gdm/greeter/autostart
…
 ├─init.scope
 │ └─1 /usr/lib/systemd/systemd --switched-root --system --deserialize 18
 └─system.slice
  …
  ├─example.service
  │ ├─6882 /bin/bash /home/jdoe/example.sh
```

```
  |   └─6902 sleep 1
  ├─systemd-journald.service
   └─629 /usr/lib/systemd/systemd-journald
 …
```

示例输出返回整个 **cgroups** 层次结构，其中最高级别由 *slices* 组成。

- 使用 **systemd-cgls <*resource_controller*>** 命令显示资源控制器过滤的 **cgroups** 层次结构。

```
# systemd-cgls memory
Controller memory; Control group /:
 ├─1 /usr/lib/systemd/systemd --switched-root --system --deserialize 18
 ├─user.slice
 |   ├─user-42.slice
 |   |   ├─session-c1.scope
 |   |   |   ├─ 965 gdm-session-worker [pam/gdm-launch-environment]
…
 └─system.slice
  |
  …
   ├─chronyd.service
   |   └─844 /usr/sbin/chronyd
   ├─example.service
   |   ├─8914 /bin/bash /home/jdoe/example.sh
   |   └─8916 sleep 1
  …
```

示例输出列出了与所选控制器交互的服务。

- 使用 **systemctl status <*system_unit*>** 命令显示某个单元及其 **cgroups** 层次结构部分的详细信息。

```
# systemctl status example.service
● example.service - My example service
   Loaded: loaded (/usr/lib/systemd/system/example.service; enabled; vendor preset:
disabled)
   Active: active (running) since Tue 2019-04-16 12:12:39 CEST; 3s ago
 Main PID: 17737 (bash)
    Tasks: 2 (limit: 11522)
   Memory: 496.0K (limit: 1.5M)
   CGroup: /system.slice/example.service
           ├─17737 /bin/bash /home/jdoe/example.sh
           └─17743 sleep 1
Apr 16 12:12:39 redhat systemd[1]: Started My example service.
Apr 16 12:12:39 redhat bash[17737]: The current time is Tue Apr 16 12:12:39 CEST 2019
Apr 16 12:12:40 redhat bash[17737]: The current time is Tue Apr 16 12:12:40 CEST 2019
```

25.7. 查看进程的 CGROUP

您可以了解进程属于哪一个 *控制组* (**cgroup**)。然后，您可以检查 **cgroup**，以查找其使用哪个控制器和特定于控制器的配置。

流程

1. 要查看某个进程所属的 **cgroup**，请运行 **# cat proc/<*PID*>/cgroup** 命令：

```
# cat /proc/2467/cgroup
0::/system.slice/example.service
```

输出示例与关注进程相关。在这种情况下，它是由 **PID 2467** 来标识的进程，它属于 **example.service** 单元。您可以检查进程是否被放置在 **systemd** 单元文件规范所定义的正确控制组中。

2. 要显示 **cgroup** 使用哪些控制器和配置文件，请检查 **cgroup** 目录：

```
# cat /sys/fs/cgroup/system.slice/example.service/cgroup.controllers
memory pids

# ls /sys/fs/cgroup/system.slice/example.service/
cgroup.controllers
cgroup.events
…
cpu.pressure
cpu.stat
io.pressure
memory.current
memory.events
…
pids.current
pids.events
pids.max
```

cgroup 版本 1 层次结构使用每个控制器模型。因此，**/proc/*PID*/cgroup** 文件中的输出显示，PID 所属的每个控制器下的 **cgroups**。您可以在 **/sys/fs/cgroup/*<controller_name>*/** 控制器目录下找到 **cgroups**。

如需更多信息，请参阅 **/usr/share/doc/kernel-doc-<kernel_version>/Documentation/admin-guide/cgroup-v2.rst** 文件（安装 **kernel-doc** 软件包）。

其他资源

- 内核资源控制器简介

25.8. 监控资源消耗

查看当前运行的控制组(**cgroup**)的列表及其实时资源消耗。

流程

1. 使用 **systemd-cgtop** 命令显示当前运行的 **cgroup** 的动态帐户：

```
# systemd-cgtop
Control Group                          Tasks   %CPU   Memory  Input/s Output/s
/                                        607   29.8    1.5G       -       -
/system.slice                            125      -  428.7M       -       -
/system.slice/ModemManager.service         3      -    8.6M       -       -
/system.slice/NetworkManager.service       3      -   12.8M       -       -
/system.slice/accounts-daemon.service      3      -    1.8M       -       -
```

```
/system.slice/boot.mount                    -    -   48.0K     -     -
/system.slice/chronyd.service               1    -    2.0M     -     -
/system.slice/cockpit.socket                -    -    1.3M     -     -
/system.slice/colord.service                3    -    3.5M     -     -
/system.slice/crond.service                 1    -    1.8M     -     -
/system.slice/cups.service                  1    -    3.1M     -     -
/system.slice/dev-hugepages.mount           -    -  244.0K     -     -
/system.slice/dev-mapper-rhel\x2dswap.swap  -    -  912.0K     -     -
/system.slice/dev-mqueue.mount              -    -   48.0K     -     -
/system.slice/example.service               2    -    2.0M     -     -
/system.slice/firewalld.service             2    -   28.8M     -     -
...
```

示例输出显示当前运行的 **cgroups**，按照资源使用量排序（CPU、内存、磁盘 I/O 负载）。这个列表默认每 1 秒刷新一次。因此，它提供了一个动态洞察每个控制组的实际资源使用情况。

如需更多信息，请参阅系统中的 **systemd-cgtop (1)** 手册页。

25.9. 使用 SYSTEMD 单元文件为应用程序设置限制

systemd 服务管理器监督每个现有或正在运行的单元，并为它们创建控制组。该单元在 **/usr/lib/systemd/system/** 目录中有配置文件。

您可以手动将单元文件修改为：

- 设置限制。
- 优先顺序。
- 控制对进程组的硬件资源的访问。

先决条件

- 您在系统上具有 root 权限。

流程

1. 编辑 **/usr/lib/systemd/system/example.service** 文件，来限制服务的内存使用：

```
…
[Service]
MemoryMax=1500K
…
```

配置限制控制组中的进程不能超过的最大内存。**example.service** 服务是此类控制组群的一部分，它有一定的限制。使用后缀 K、M、G 或 T 将 Kilobyte、Megabyte、Gigabyte 或 Terabyte 作为一个测量单位。

2. 重新载入所有单元配置文件：

```
# systemctl daemon-reload
```

3. 重启服务：

```
# systemctl restart example.service
```

验证

1. 检查更改是否生效：

```
# cat /sys/fs/cgroup/system.slice/example.service/memory.max
1536000
```

此输出显示内存消耗被限制在大约 1,500 KB。

其他资源

- 使用 systemctl 管理系统服务

25.10. 使用 SYSTEMCTL 命令将限制设置为应用程序

CPU 关联性设置可帮助您将特定进程的访问限制到某些 CPU。实际上，CPU 调度程序永远不会将进程调度到不在进程的关联性掩码中的 CPU 上运行。

默认 CPU 关联性掩码应用到 **systemd** 管理的所有服务。

要为特定的 **systemd** 服务配置 CPU 关联性掩码， **systemd** 提供了 **CPUAffinity=** 作为：

- 单元文件选项。
- **/etc/systemd/system.conf** 文件的 [Manager] 部分中的配置选项。

CPUAffinity= 单元文件选项设置 CPU 或 CPU 范围列表，它们被合并并用作关联性掩码。使用 **CPUAffinity** 单元文件选项为特定 **systemd** 服务设置 CPU 关联性掩码。

流程

1. 在您选择的服务中检查 **CPUAffinity** 单元文件选项的值：

```
$ systemctl show --property <CPU affinity configuration option> <service name>
```

2. 以 root 用户身份，为用作关联性掩码的 CPU 范围设置 **CPUAffinity** 单元文件选项所需的值：

```
# systemctl set-property <service name> CPUAffinity=<value>
```

3. 重新启动服务以应用更改。

```
# systemctl restart <service name>
```

如需更多信息，请参阅系统上的 **systemd.resource-control (5)**、**systemd.exec (5)** 和 **cgroups (7)** man page。

25.11. 通过管理器配置设置全局默认 CPU 关联性

/etc/systemd/system.conf 文件中的 **CPUAffinity** 选项为进程识别号(PID) 1 和从 PID1 分叉的所有进程定义一个关联性掩码。然后，您可以基于每个服务覆盖 **CPUAffinity**。

使用 **/etc/ systemd /system.conf** 文件为所有 systemd 服务设置默认 CPU 关联性掩码。

流程

1. 在 **/etc/systemd/system.conf** 文件的 **[Manager]** 部分中设置 **CPUAffinity=** 选项的 CPU 号。
2. 保存编辑的文件并重新载入 **systemd** 服务：

```
# systemctl daemon-reload
```

3. 重启服务器以应用更改。
   详情请查看系统中的 **systemd.resource-control (5)** 手册页。

25.12. 使用 SYSTEMD 配置 NUMA 策略

非统一内存访问(NUMA)是一种计算机内存子系统设计，其中内存访问时间取决于相对于处理器的物理内存位置。

靠近 CPU 的内存比不同 CPU 的本地内存(外部内存)或一组 CPU 之间共享的内存具有更低的延迟（本地内存）。

就 Linux 内核而言，NUMA 策略管理内核为进程分配物理内存页面的位置（例如，在哪些 NUMA 节点上）。

systemd 提供单元文件选项 **NUMAPolicy** 和 **NUMAMask**，以控制服务的内存分配策略。

重要

当您配置严格的 NUMA 策略时，例如 **bind**，请确保您也正确地设置了 **CPUAffinity=** 单元文件选项。

流程

- 通过 **NUMAPolicy** 单元文件选项设置 NUMA 内存策略：
  a. 在您选择的服务中检查 **NUMAPolicy** 单元文件选项的值：

```
$ systemctl show --property <NUMA policy configuration option> <service name>
```

  b. 作为根目录，设置 **NUMAPolicy** 单元文件选项所需的策略类型：

```
# systemctl set-property <service name> NUMAPolicy=<value>
```

  c. 重启服务以应用更改：

```
# systemctl restart <service name>
```

- 使用 **[Manager]** 配置选项设置全局 **NUMAPolicy** 设置：
  a. 在 **/etc/systemd/system.conf** 文件中搜索文件 **[Manager]** 部分中的 **NUMAPolicy** 选项。
  b. 编辑策略类型并保存文件。

c. 重新载入 **systemd** 配置：

```
# systemd daemon-reload
```

d. 重启服务器。

其他资源

- 使用 systemctl 命令将限制设置为应用程序

25.13. SYSTEMD 的 NUMA 策略配置选项

Systemd 提供以下选项来配置 NUMA 策略：

NUMAPolicy

控制已执行进程的 NUMA 内存策略。您可以使用这些策略类型：

- default
- preferred
- bind
- interleave
- local

NUMAMask

控制与所选的 NUMA 策略关联的 NUMA 节点列表。
请注意，您不必为以下策略指定 **NUMAMask** 选项：

- default
- local
  对于首选策略，列表仅指定单个 NUMA 节点。

如需更多信息，请参阅系统上的 **systemd.resource-control (5)**、**systemd.exec (5)** 和 **set_mempolicy (2)** man page。

25.14. 使用 SYSTEMD-RUN 命令创建临时 CGROUP

临时 **cgroup** 设置运行时期间由单元（服务或范围）消耗的资源的限制。

流程

- 要创建一个临时控制组群，使用以下格式的 **systemd-run** 命令：

```
# systemd-run --unit=<name> --slice=<name>.slice <command>
```

此命令会创建并启动临时服务或范围单元，并在此类单元中运行自定义命令。

  - **--unit=<name>** 选项为单元取一个名称。如果未指定 **--unit**，则会自动生成名称。

  - **--slice=<*name*>.slice** 选项使您的服务或范围单元成为指定片段的成员。将 **<*name*>.slice** 替换为现有片段的名称（如 **systemctl -t slice** 输出中所示），或通过传递唯一名称来创建新片段。默认情况下，服务和范围作为 **system.slice** 的成员创建。
  - 使用您要在服务或范围单元中输入的命令替换 **<*command*>**。
    此时会显示以下信息，以确认您已创建并启动了该服务，或者已成功启动范围：

    ```
    # Running as unit <name>.service
    ```

- *可选*：在其进程完成后保持单元运行，以收集运行时信息：

  ```
  # systemd-run --unit=<name> --slice=<name>.slice --remain-after-exit <command>
  ```

  命令会创建并启动临时服务单元，并在单元中运行自定义命令。**--remain-after-exit** 选项可确保服务在其进程完成后继续运行。

25.15. 删除临时控制组群

如果您不再需要限制、确定或控制对进程组的硬件资源的访问，您可以使用 **systemd** 系统和服务管理器删除临时控制组 (**cgroup**)。

当服务或范围单元包含的所有进程完成时，临时 **cgroup** 会被自动释放。

流程

- 要停止带有所有进程的服务单元，请输入：

  ```
  # systemctl stop <name>.service
  ```

- 要终止一个或多个单元进程，请输入：

  ```
  # systemctl kill <name>.service --kill-who=PID,... --signal=<signal>
  ```

  命令使用 **--kill-who** 选项从您要终止的控制组中选择进程。要同时终止多个进程，请传递以逗号分隔的 PID 列表。**--signal** 决定要发送到指定进程的 POSIX 信号的类型。默认信号是 *SIGTERM*。

其他资源

- 控制组简介
- 内核资源控制器简介
- 控制组简介
- 管理 systemd

第 26 章 使用控制组群为应用程序设置系统资源限制

使用控制组(**cgroups**)内核功能，您可以控制应用程序的资源使用情况来更有效地使用它们。

您可以为以下任务使用 **cgroups** ：

- 为系统资源分配设置限制。
- 将硬件资源优先分配给特定的进程。
- 防止某些进程获取硬件资源。

26.1. 控制组简介

使用 *控制组* Linux 内核功能，您可以将进程组织为按层排序的组 - **cgroups**。您可以通过为 **cgroup** 虚拟文件系统提供结构来定义层次结构（控制组树），默认挂载到 **/sys/fs/cgroup/** 目录。

systemd 服务管理器使用 **cgroups** 来组织它管理的所有单元和服务。您可以通过创建和删除 **/sys/fs/cgroup/** 目录中的子目录来手动管理 **cgroups** 的层次结构。

然后，内核中的资源控制器通过限制、优先处理或分配这些进程的系统资源来在 **cgroups** 中修改进程的行为。这些资源包括以下内容：

- CPU 时间
- 内存
- 网络带宽
- 这些资源的组合

cgroups 的主要用例是聚合系统进程，并在应用程序和用户之间划分硬件资源。这样可以提高环境的效率、稳定性和安全性。

控制组群版本 1

控制组版本 1 (**cgroups-v1**)为每个资源控制器提供单独的层次结构。CPU、内存或 I/O 等资源具有自己的控制组层次结构。您可以组合不同的控制组层次结构，以便一个控制器可以在管理其单个资源时相互协调。但是，当两个控制器属于不同的进程层次结构时，协调受到限制。
cgroups-v1 控制器的开发时间跨度很长，从而导致它们的控制文件的行为和命名不一致。

控制组群版本 2

控制组版本 2 (**cgroups-v2**) 提供单一控制组层次结构，用于挂载所有资源控制器。
控制文件行为和命名在不同控制器之间保持一致。

重要

默认情况下，RHEL 10 挂载并使用 **cgroups-v2**。

有关 **cgroups-v1** 和 **cgroups-v2** 的详情，请安装 **kernel-doc** RPM 软件包。安装后，文档位于本地系统的 **/usr/share/doc/kernel-doc- &*lt;version*> /Documentation** 目录中。**cgroups-v1** 文档文件位于 **Documentation/admin-guide/cgroup-v1/** 目录中。此目录在不同控制器中有多个文件。**cgroups-v2** 文档位于 **Documentation/admin-guide/cgroup-v2.rst** 文件中。

其他资源

- 内核资源控制器简介

26.2. 内核资源控制器简介

内核资源控制器启用控制组的功能。RHEL 10 支持用于 *控制组版本 1* (**cgroups-v1**)和 *控制组版本 2* (**cgroups-v2**)的各种控制器。

资源控制器也称为控制组子系统，是一个代表单一资源的内核子系统，如 CPU 时间、内存、网络带宽或磁盘 I/O。Linux 内核提供由 **systemd** 服务管理器自动挂载的一系列资源控制器。

您可以在 **/proc/cgroups** 文件中找到当前挂载的资源控制器的列表。

可用于 cgroups-v1的控制器

- **blkio** ：设置对块设备的输入/输出访问的限制。
- **CPU** ：调整控制组任务的默认调度程序的参数。**cpu** 控制器与 **cpuacct** 控制器一起挂载在同一挂载上。
- **cpuacct** ：创建控制组中任务使用的 CPU 资源自动报告。 **cpuacct** 控制器与 **cpu** 控制器一起挂载在同一挂载上。
- **cpuset**:Restricts 控制组任务仅在指定 CPU 子集上运行，并指示任务仅在指定的内存节点上使用内存。
- **设备** ：控制对控制组中任务的访问。
- **freezer** ：暂停或恢复控制组中的任务。
- Memory ：设置控制组中任务的 **内存使用** 限制，并针对这些任务使用的内存资源生成自动报告。
- **net_cls** ：使用类标识符(**classid**)标记网络数据包，使 Linux 流量控制器(**tc** 命令)能够识别源自特定控制组任务的数据包。**net_cls** 子系统 **net_filter** (iptables) 也可使用此标签对此类数据包执行操作。
- **net_filter** ：使用防火墙标识符(**fwid**)标记网络套接字，允许 Linux 防火墙识别源自特定控制组任务的数据包（通过使用 **iptables** 命令）。
- **net_prio** ：设置网络流量的优先级。
- **PIDs** ：为控制组群中的多个进程及其子进程设置限值。
- **perf_event** ：通过 **perf** 性能监控和报告工具对任务进行分组。
- **RDMA** ：设置控制组群中的 Remote Direct Memory Access/InfiniBand 特定资源的限制。
- **HugeTLB** ：根据控制组群中的任务限制大量虚拟内存页面的使用。

可用于 cgroups-v2的控制器

- **io** ：设置对块设备的输入/输出访问的限制。

- Memory ：设置控制组中任务的 **内存使用** 限制，并针对这些任务使用的内存资源生成自动报告。
- **PIDs** ：为控制组群中的多个进程及其子进程设置限值。
- **RDMA** ：设置控制组群中的 Remote Direct Memory Access/InfiniBand 特定资源的限制。
- CPU：调整控制组任务的默认调度程序的参数，并创建控制组中任务使用的 CPU 资源自动报告。
- **cpuset: 限制控制组任务仅在指定 CPU 子集上运行，并指示任务仅在指定的内存节点上使用内存。仅支持具有新分区功能的核心功能(cpus{,.effective}, mems{,.effective})。**
- **perf_event ：通过 perf 性能监控和报告工具对任务进行分组。perf_event 在 v2 层次结构上自动启用。**

重要

资源控制器可以在 cgroups-v1 层次结构或 cgroups-v2 层次结构中使用，不能同时在两者中使用。

26.3. 命名空间简介

命名空间为组织和识别软件对象创建单独的空间。这使得它们不会相互影响。因此，每个软件对象都包含其自己的一组资源，如挂载点、网络设备或主机名，即使它们共享同样的系统。

使用命名空间的最常见技术是容器。

对特定全局资源的更改仅对该命名空间中的进程可见，不影响系统或其他命名空间的其余部分。

要检查进程所属的命名空间，您可以在 /proc/<*PID*>/ns/ 目录中检查符号链接。

表 26.1. 支持的命名空间以及它们隔离的资源：

Namespace	Isolates
Mount	挂载点
UTS	主机名和 NIS 域名
IPC	系统 V IPC, POSIX 消息队列
PID	进程 ID
Network	网络设备、堆栈、端口等
User	用户和组群 ID
Control groups	控制组群根目录

如需更多信息，请参阅系统中的 namespaces (7) 和 cgroup_namespaces (7) 手册页。

第 27 章 使用 CGROUPFS 手动管理 CGROUP

您可以通过在 cgroupfs 虚拟文件系统中创建目录来管理系统上的 cgroup 层次结构。文件系统默认挂载到 /sys/fs/cgroup/ 目录，您可以在专用的控制文件中指定所需的配置。

注意

cgroups-v1 支持被 systemd 弃用，因此 cgroup-v1 将从以后的 Red Hat Enterprise Linux 10 版本中删除。在以后的 RHEL 10 版本中必须使用 cgroups-v2。

重要

您必须使用 systemd 来控制系统资源的使用。除非是特殊情况，否则不得手动配置 cgroups 虚拟文件系统。

27.1. 在 CGROUPS-V2 文件系统中创建 CGROUP 和启用控制器

您可以通过创建和删除目录，并通过写入 cgroup 虚拟文件系统中的文件来管理 *控制组* (cgroups)。文件系统默认挂载到 /sys/fs/cgroup/ 目录中。要使用 cgroups 控制器中的设置，您还需要为子 cgroup 启用所需的控制器。在默认情况下，root cgroup 会为其子 cgroups 启用 memory 和 pids。因此，您必须在 /sys/fs/cgroup/ root cgroup 中创建至少两级子 cgroups。这样，您可以选择从子 cgroup 中删除 memory 和 pids 控制器，并更好地组织 cgroup 文件。

先决条件

- 您在系统上具有 root 权限。

流程

1. 创建 /sys/fs/cgroup/Example/ 目录：

```
# mkdir /sys/fs/cgroup/Example/
```

/sys/fs/cgroup/Example/ 目录定义一个子组。当您创建 /sys/fs/cgroup/Example/ 目录时，目录中会自动创建一些 cgroups-v2 接口文件。/sys/fs/cgroup/Example/ 目录还包含 memory 和 pids 控制器的特定于控制器的文件。

2. 可选：检查新创建的子控制组：

```
# ll /sys/fs/cgroup/Example/
-r—r—r--. 1 root root 0 Jun  1 10:33 cgroup.controllers
-r—r—r--. 1 root root 0 Jun  1 10:33 cgroup.events
-rw-r—r--. 1 root root 0 Jun  1 10:33 cgroup.freeze
-rw-r--r--. 1 root root 0 Jun  1 10:33 cgroup.procs
…
-rw-r—r--. 1 root root 0 Jun  1 10:33 cgroup.subtree_control
-r—r—r--. 1 root root 0 Jun  1 10:33 memory.events.local
-rw-r—r--. 1 root root 0 Jun  1 10:33 memory.high
-rw-r—r--. 1 root root 0 Jun  1 10:33 memory.low
…
-r—r—r--. 1 root root 0 Jun  1 10:33 pids.current
-r—r—r--. 1 root root 0 Jun  1 10:33 pids.events
-rw-r—r--. 1 root root 0 Jun  1 10:33 pids.max
```

示例输出显示常规 cgroup 控制接口文件，如 cgroup.procs 或 cgroup.controllers。无论启用控制器是什么，这些文件都是所有控制组通用的。

memory.high 和 pids.max 等文件与 memory 和 pids 控制器有关，它们是 root 控制组 (/sys/fs/cgroup/)，默认情况下会被 systemd 启用。

默认情况下，新创建的子组从父 cgroup 继承所有设置。在这种情况下，来自 root cgroup 没有限制。

3. 验证 /sys/fs/cgroup/cgroup.controllers 文件中是否有所需的控制器：

```
# cat /sys/fs/cgroup/cgroup.controllers
cpuset cpu io memory hugetlb pids rdma
```

4. 启用所需的控制器。在本例中是 cpu 和 cpuset 控制器：

```
# echo "+cpu" >> /sys/fs/cgroup/cgroup.subtree_control
# echo "+cpuset" >> /sys/fs/cgroup/cgroup.subtree_control
```

这些命令为 /sys/fs/cgroup/ root 控制组的直接子组启用 cpu 和 cpuset 控制器。包含新创建的 Example 控制组。*子组* 是可以指定进程，并根据标准对每个进程应用控制检查的位置。

用户可以在任意级别读取 cgroup.subtree_control 文件的内容，以了解即时子组中哪些控制器可用于启用。

注意

默认情况下，根控制组中的 /sys/fs/cgroup/cgroup.subtree_control 文件包含 memory 和 pids 控制器。

5. 为 Example 控制组群的子 cgroup 启用所需的控制器：

```
# echo "+cpu +cpuset" >> /sys/fs/cgroup/Example/cgroup.subtree_control
```

这些命令可确保，直接的子组仅具有与 CPU 时间分发相关的控制器，而不是 memory 或 pids 控制器。

6. 创建 /sys/fs/cgroup/Example/tasks/ 目录：

```
# mkdir /sys/fs/cgroup/Example/tasks/
```

/sys/fs/cgroup/Example/tasks/ 目录定义了一个子组，它带有只与 cpu 和 cpuset 控制器相关的文件。现在，您可以将进程分配给此控制组，并将 cpu 和 cpuset 控制器选项用于您的进程。

7. 可选：检查子控制组：

```
# ll /sys/fs/cgroup/Example/tasks
-r—r—r--. 1 root root 0 Jun  1 11:45 cgroup.controllers
-r—r—r--. 1 root root 0 Jun  1 11:45 cgroup.events
-rw-r—r--. 1 root root 0 Jun  1 11:45 cgroup.freeze
-rw-r—r--. 1 root root 0 Jun  1 11:45 cgroup.max.depth
-rw-r—r--. 1 root root 0 Jun  1 11:45 cgroup.max.descendants
-rw-r—r--. 1 root root 0 Jun  1 11:45 cgroup.procs
-r—r—r--. 1 root root 0 Jun  1 11:45 cgroup.stat
-rw-r—r--. 1 root root 0 Jun  1 11:45 cgroup.subtree_control
-rw-r—r--. 1 root root 0 Jun  1 11:45 cgroup.threads
-rw-r—r--. 1 root root 0 Jun  1 11:45 cgroup.type
-rw-r—r--. 1 root root 0 Jun  1 11:45 cpu.max
-rw-r—r--. 1 root root 0 Jun  1 11:45 cpu.pressure
-rw-r—r--. 1 root root 0 Jun  1 11:45 cpuset.cpus
-r—r—r--. 1 root root 0 Jun  1 11:45 cpuset.cpus.effective
```

```
-rw-r—r--. 1 root root 0 Jun  1 11:45 cpuset.cpus.partition
-rw-r—r--. 1 root root 0 Jun  1 11:45 cpuset.mems
-r—r—r--. 1 root root 0 Jun  1 11:45 cpuset.mems.effective
-r—r—r--. 1 root root 0 Jun  1 11:45 cpu.stat
-rw-r—r--. 1 root root 0 Jun  1 11:45 cpu.weight
-rw-r—r--. 1 root root 0 Jun  1 11:45 cpu.weight.nice
-rw-r—r--. 1 root root 0 Jun  1 11:45 io.pressure
-rw-r—r--. 1 root root 0 Jun  1 11:45 memory.pressure
```

重要

cpu 控制器只有在相关子控制组至少有 2 个在单个 CPU 上竞争时间的进程时，才会被激活 。

验证

- 可选：确认您是否已创建了一个只有所需的控制器处于活跃状态的新的 cgroup ：

```
# cat /sys/fs/cgroup/Example/tasks/cgroup.controllers
cpuset cpu
```

其他资源

- 内核资源控制器简介

27.2. 通过调整 CPU 权重来控制应用程序的 CPU 时间分布

您需要为 cpu 控制器的相关文件分配值，以控制分发到特定 cgroup 树下应用程序的 CPU 时间。

先决条件

- 您在系统上具有 root 权限。
- 您有要控制 CPU 时间分布的应用程序。
- 您已挂载了 cgroups-v2 文件系统。

- 您在 /sys/fs/cgroup/ *根控制组* 中创建了两级 *子控制组*，如下例所示：

```
...
├── Example
│   ├── g1
│   ├── g2
│   └── g3
...
```

- 您已在父控制组和子控制组中启用 cpu 控制器，类似于在 cgroups-v2 文件系统中创建 cgroups 并启用控制器。

流程

1. 配置所需的 CPU 权重，以便在控制组内实现资源限制：

```
# echo "150" > /sys/fs/cgroup/Example/g1/cpu.weight
# echo "100" > /sys/fs/cgroup/Example/g2/cpu.weight
# echo "50" > /sys/fs/cgroup/Example/g3/cpu.weight
```

2. 将应用程序的 PID 添加到 g1、g2 和 g3 子组中：

```
# echo "33373" > /sys/fs/cgroup/Example/g1/cgroup.procs
# echo "33374" > /sys/fs/cgroup/Example/g2/cgroup.procs
# echo "33377" > /sys/fs/cgroup/Example/g3/cgroup.procs
```

这些命令确保所需的应用程序成为 Example/g*/ 子 cgroup 的成员，并根据这些 cgroup 的配置获得其分配的 CPU 时间。

已运行进程的子 cgroup (g1, g2, g3) 的权重在父 cgroup（Example）级别上相加。然后根据分配的权重按比例分配 CPU 资源。

因此，当所有进程在同一时间运行时，内核会根据分配的 cgroup 的 cpu.weight 文件，为每个进程分配相应的 CPU 时间：

子 cgroup	cpu.weight 文件	CPU 时间分配
g1	150	~50% (150/300)

子 cgroup	cpu.weight 文件	CPU 时间分配
g2	100	~33% (100/300)
g3	50	~16% (50/300)

cpu.weight 控制器文件的值不是一个百分比。

如果一个进程停止运行，使 cgroup g2 没有运行进程，则计算将省略 cgroup g2，仅计算 cgroup g1 和 g3 的帐户权重：

子 cgroup	cpu.weight 文件	CPU 时间分配
g1	150	~75% (150/200)
g3	50	~25% (50/200)

重要

如果子 cgroup 有多个正在运行的进程，则分配给 cgroup 的 CPU 时间会在其成员进程间平均分配。

验证

1. 验证应用程序是否运行在指定的控制组中：

```
# cat /proc/33373/cgroup /proc/33374/cgroup /proc/33377/cgroup
0::/Example/g1
0::/Example/g2
0::/Example/g3
```

命令输出显示了运行在 Example/g*/ 子 cgroup 中指定的应用程序的进程。

2. 检查节流应用程序的当前 CPU 消耗：

```
# top
top - 05:17:18 up 1 day, 18:25,  1 user,  load average: 3.03, 3.03, 3.00
```

```
Tasks:  95 total,   4 running,  91 sleeping,   0 stopped,   0 zombie
%Cpu(s): 18.1 us, 81.6 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.3 hi,  0.0 si,  0.0 st
MiB Mem :   3737.0 total,   3233.7 free,    132.8 used,    370.5 buff/cache
MiB Swap:   4060.0 total,   4060.0 free,      0.0 used.   3373.1 avail Mem

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 33373 root      20   0   18720   1748   1460 R  49.5   0.0 415:05.87 sha1sum
 33374 root      20   0   18720   1756   1464 R  32.9   0.0 412:58.33 sha1sum
 33377 root      20   0   18720   1860   1568 R  16.3   0.0 411:03.12 sha1sum
   760 root      20   0  416620  28540  15296 S   0.3   0.7   0:10.23 tuned
     1 root      20   0  186328  14108   9484 S   0.0   0.4   0:02.00 systemd
     2 root      20   0       0      0      0 S   0.0   0.0   0:00.01 kthread
...
```

注意

为了清楚地说明，所有进程都在单个 CPU 上运行。当在多个 CPU 上使用时，CPU 权重应用相同的原则。

请注意，PID 33373、PID 33374 和 PID 33377 的 CPU 资源是根据您分配给子 cgroup 的 150、100 和 50 权重分配的。权重对应于每个应用程序分配的 CPU 时间的大约 50%、33% 和 16%。

第 28 章 使用 EBPF 分析系统性能

您可以使用 bfptrace 和 BPF Compiler Collection (BCC)库创建用于分析 Linux 操作系统性能及收集信息的工具，这些信息可能很难通过其他接口获得。

28.1. 使用 BPFTRACE 软件包

bpftrace 是使用 eBPF 技术的，一个用于 RHEL 系统的强大的追踪工具。您可以动态跟踪并分析内核和用户空间事件，而不用修改内核代码。

流程

1. 安装 bpftrace 软件包：

```
$ sudo dnf install bpftrace
```

2. 运行测试：

```
$ sudo bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @ = count(); } interval:s:1 {
print(@); clear(@); }'
```

此命令通过观察系统调用的速率来显示系统活动的高级概述。

您现在准备使用 bpftrace。您可以浏览 /usr/share/bpftrace/tools/ 处的示例脚本，在线学习脚本，或创建自己的脚本来跟踪事件并分析系统行为。

其他资源

- bpftrace
- 一行命令示例
- 官方工具

- 用户工具

28.2. 安装 BCC-TOOLS 软件包

安装 bcc-tools 软件包，该软件包还会将 BPF Compiler Collection (BCC)库作为依赖项安装。

流程

- 安装 bcc-tools ：

```
# dnf install bcc-tools
```

BCC 工具安装在 /usr/share/bcc/tools/ 目录中。

验证

- 检查安装的工具：

```
# ls -l /usr/share/bcc/tools/
...
-rwxr-xr-x. 1 root root  4198 Dec 14 17:53 dcsnoop
-rwxr-xr-x. 1 root root  3931 Dec 14 17:53 dcstat
-rwxr-xr-x. 1 root root 20040 Dec 14 17:53 deadlock_detector
-rw-r--r--. 1 root root  7105 Dec 14 17:53 deadlock_detector.c
drwxr-xr-x. 3 root root  8192 Mar 11 10:28 doc
-rwxr-xr-x. 1 root root  7588 Dec 14 17:53 execsnoop
-rwxr-xr-x. 1 root root  6373 Dec 14 17:53 ext4dist
-rwxr-xr-x. 1 root root 10401 Dec 14 17:53 ext4slower
...
```

上表中的 doc 目录包含每个工具的文档。

28.3. 使用所选 BCC-TOOLS 进行性能调整

使用 BPF Compiler Collection (BCC)库中的某些预先创建的程序来在每个事件基础上高效、安全地分析系统性能。BCC 库中预创建的程序集可作为创建其他程序的示例。

先决条件

- 您在系统上具有 root 权限。
- 您已安装了 bcc-tools 软件包。

流程

- 使用 execsnoop 检查新系统进程。

  a. 在一个终端中运行 execsnoop 程序：

  ```
  # /usr/share/bcc/tools/execsnoop
  ```

  b. 要创建 ls 命令的短期进程，请在另一个终端中输入：

  ```
  $ ls /usr/share/bcc/tools/doc/
  ```

  c. 运行 execsnoop 的终端显示类似如下的输出：

  ```
  PCOMM PID    PPID   RET ARGS
  ls    8382   8287     0 /usr/bin/ls --color=auto /usr/share/bcc/tools/doc/
  ...
  ```

  execsnoop 程序为每个消耗系统资源的新进程的打印一行输出。它甚至会检测很快运行的程序（如 ls ）的进程，大多数监控工具也不会进行注册。

  execsnoop 输出显示以下字段：

  PCOMM

  进程名称。(ls)

  PID

进程 ID。(8382)

PPID

父进程 ID。(8287)

RET

exec（） 系统调用的返回值(0)，其将程序代码加载到新进程中。

ARGS

启动的程序的参数的位置。

要查看 execsnoop 的详情、示例和选项，请参阅 /usr/share/bcc/tools/doc/execsnoop_example.txt 文件。有关 exec () 的详情，请查看 exec (3) 手册页。

- 使用 opensnoop 跟踪命令打开的文件。

a. 在一个终端中，运行 opensnoop 程序，来打印仅由 uname 命令的进程打开的文件的输出：

```
# /usr/share/bcc/tools/opensnoop -n uname
```

b. 在另一个终端中，输入以下命令来打开某些文件：

```
$ uname
```

c. 运行 opensnoop 的终端显示类似如下的输出：

```
PID    COMM  FD ERR PATH
8596   uname  3  0   /etc/ld.so.cache
8596   uname  3  0   /lib64/libc.so.6
8596   uname  3  0   /usr/lib/locale/locale-archive
...
```

opensnoop 程序在整个系统中监视 open () 系统调用，并为 uname 尝试打开的每个文

件打印一行输出。

opensnoop 输出显示以下字段：

PID

进程 ID。(8596)

COMM

进程名称。(uname)

FD

文件描述符 - open（） 返回的值，以指向打开的文件。(3)

ERR

任何错误。

PATH

open（） 试图打开的文件的位置。

如果命令尝试读取不存在的文件，则 FD 列返回 -1，ERR 列将打印与相关错误对应的值。因此，Opennoop 可以帮助您识别行为不正确的应用程序。

要查看 opensnoop 的更多详细信息、示例和选项，请参阅 /usr/share/bcc/tools/doc/opensnoop_example.txt 文件。有关 open () 的更多信息，请参阅 open (2) 手册页。

- 使用 biotop 监控磁盘上执行 I/O 操作的主要进程。

a. 在一个终端中使用参数 30 运行 biotop 程序，来生成 30 秒概述：

```
# /usr/share/bcc/tools/biotop 30
```

注意

如果未提供任何参数，则默认情况下输出屏幕会每 1 秒刷新一次。

b. **在另一个终端中，输入命令来从本地硬盘设备读取内容，并将输出写到 /dev/zero 文件中：**

```
# dd if=/dev/vda of=/dev/zero
```

此步骤会生成特定的 I/O 流量来演示 biotop。

c. **运行 biotop 的终端显示类似如下的输出：**

```
PID    COMM             D MAJ MIN DISK       I/O  Kbytes     AVGms
9568   dd               R 252 0   vda      16294 14440636.0  3.69
48     kswapd0          W 252 0   vda       1763 120696.0    1.65
7571   gnome-shell      R 252 0   vda        834 83612.0     0.33
1891   gnome-shell      R 252 0   vda       1379 19792.0     0.15
7515   Xorg             R 252 0   vda        280  9940.0     0.28
7579   llvmpipe-1       R 252 0   vda        228  6928.0     0.19
9515   gnome-control-c  R 252 0   vda         62  6444.0     0.43
8112   gnome-terminal-  R 252 0   vda         67  2572.0     1.54
7807   gnome-software   R 252 0   vda         31  2336.0     0.73
9578   awk              R 252 0   vda         17  2228.0     0.66
7578   llvmpipe-0       R 252 0   vda        156  2204.0     0.07
9581   pgrep            R 252 0   vda         58  1748.0     0.42
7531   InputThread      R 252 0   vda         30  1200.0     0.48
7504   gdbus            R 252 0   vda          3  1164.0     0.30
1983   llvmpipe-1       R 252 0   vda         39   724.0     0.08
1982   llvmpipe-0       R 252 0   vda         36   652.0     0.06
...
```

biotop 输出显示以下字段：

PID

进程 ID。(9568)

COMM

进程名称。(dd)

DISK

执行读操作的磁盘。(vda)

I/O

执行的读操作的数量。(16294)

Kbytes

读操作达到的 Kbytes 量。(14,440,636)

AVGms

读操作的平均 I/O 时间。(3.69)

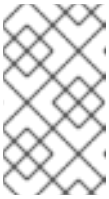
有关 biotop 的详情、示例和选项，请查看 /usr/share/bcc/tools/doc/biotop_example.txt 文件。有关 dd 的更多信息，请参阅 dd (1) 手册页。

- 使用 xfsslower 来公开意外慢的文件系统操作。

xfsslower 测量 XFS 文件系统执行读、写、打开或同步(fsync)操作所花费的时间。1 参数可确保程序仅显示比 1 ms 较慢的操作。

a. 在一个终端中运行 xfsslower 程序：

```
# /usr/share/bcc/tools/xfsslower 1
```



注意

如果未提供任何参数，xfsslower 默认会显示比 10 ms 慢的操作。

b. 在另一个终端中，输入以下命令在 vim 编辑器中创建一个文本文件，来开始与 XFS 文件系统进行交互：

```
$ vim text
```

c. 运行 **xfsslower** 的终端显示在保存上一步中的文件时：

```
TIME     COMM          PID    T BYTES   OFF_KB   LAT(ms) FILENAME
13:07:14 b'bash'       4754   R 256     0          7.11 b'vim'
13:07:14 b'vim'        4754   R 832     0          4.03 b'libgpm.so.2.1.0'
13:07:14 b'vim'        4754   R 32      20         1.04 b'libgpm.so.2.1.0'
13:07:14 b'vim'        4754   R 1982    0          2.30 b'vimrc'
13:07:14 b'vim'        4754   R 1393    0          2.52 b'getscriptPlugin.vim'
13:07:45 b'vim'        4754   S 0       0          6.71 b'text'
13:07:45 b'pool'       2588   R 16      0          5.58 b'text'
...
```

每行代表文件系统中的一个操作，它花费的时间超过特定阈值。**xfsslower** 检测可能的文件系统问题，其表现为意外的慢操作。

xfsslower 输出显示以下字段：

COMM

进程名称。(b'bash')

T

操作类型。(R)

- **Read**
- **Write**
- **Sync**

OFF_KB

KB 为单位的文件偏移。(0)

FILENAME

读、写或同步的文件。

要查看 xfsslower 的详情、示例和选项，请参阅 /usr/share/bcc/tools/doc/xfsslower_example.txt 文件。有关 fsync 的详情，请参考 fsync (2) 手册页。